Lis ユーザガイド

バージョン 2.1.21



The Scalable Software Infrastructure Project
http://www.ssisc.org/

Copyright © 2005 The Scalable Software Infrastructure Project, supported by “Development of Software Infrastructure for Large Scale Scientific Simulation” Team, CREST, JST.
Akira Nishida, CREST team director, JST.
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the project nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE SCALABLE SOFTWARE INFRASTRUCTURE PROJECT “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE SCALABLE SOFTWARE INFRASTRUCTURE PROJECT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

表紙: 尾形光琳. 燕子花図. 1705 年頃. 根津美術館所蔵.

目 次

1 はじめに **5**

2 導入 **6**

2.1 システム要件 6
2.2 UNIX 及び互換システムへの導入 6
2.2.1 アーカイブの展開 6
2.2.2 ソースツリーの設定 7
2.2.3 実行ファイルの生成 7
2.2.4 導入 11
2.3 Windows システムへの導入 12
2.4 検証 12
2.4.1 test1 12
2.4.2 test2 13
2.4.3 test2b 13
2.4.4 test3 13
2.4.5 test3b 14
2.4.6 test3c 14
2.4.7 test4 14
2.4.8 test5 14
2.4.9 test6 15
2.4.10 test7 15
2.4.11 test8f 15
2.4.12 etest1 15
2.4.13 getest1 15
2.4.14 etest2 16
2.4.15 etest3 16
2.4.16 etest4 16
2.4.17 etest5 16
2.4.18 etest5b 17
2.4.19 getest5 17
2.4.20 getest5b 17
2.4.21 etest6 17
2.4.22 etest7 18
2.4.23 spmvtest1 18
2.4.24 spmvtest2 18
2.4.25 spmvtest2b 18
2.4.26 spmvtest3 19
2.4.27 spmvtest3b 19
2.4.28 spmvtest4 19
2.4.29 spmvtest5 20
2.5 制限事項 20

3 基本操作 **22**
3.1 初期化・終了処理 . . . 22
3.2 ベクトルの操作 . . . 23
3.3 行列の操作 . . . 26
3.4 線型方程式の求解 . . . 32
3.5 固有値問題の求解 . . . 38
3.6 プログラムの作成 . . . 42
3.7 前処理とソルバの分離 . . . 45
3.8 マトリックスフリー計算 . . . 48
3.8.1 ユーザ定義シェル行列 (LIS_MATRIX_USER) . . . 48
3.8.2 線型結合行列演算子 (LIS_MATRIX_OPERATOR) . . . 48
3.8.3 行列演算子を用いたシフト付き一般化固有値ソルバ . . . 48
3.8.4 ネイティブリスタート付きクリロフソルバ . . . 48
3.8.5 不完全 LU 分解前処理の半精度保持 . . . 50
3.8.6 ユーザ定義行列における前処理の再利用と互換性 . . . 50
3.8.7 ソルバ管理機能の拡張 . . . 51
3.8.8 MFAMG 前処理 . . . 51
3.8.9 ネイティブ FEAST 固有値ソルバ . . . 51
3.9 実行ファイルの生成 . . . 52
3.10 実行 . . . 53
3.11 プロセス上での自由度について . . . 54

4 4 倍精度演算 **55**
4.1 4 倍精度演算の使用 . . . 55

5 行列格納形式 **57**
5.1 Compressed Sparse Row (CSR) . . . 57
5.1.1 行列の作成 (逐次・マルチスレッド環境) . . . 57
5.1.2 行列の作成 (マルチプロセス環境) . . . 58
5.1.3 関連する関数 . . . 58
5.2 Compressed Sparse Column (CSC) . . . 59
5.2.1 行列の作成 (逐次・マルチスレッド環境) . . . 59
5.2.2 行列の作成 (マルチプロセス環境) . . . 60
5.2.3 関連する関数 . . . 60
5.3 Modified Compressed Sparse Row (MSR) . . . 61
5.3.1 行列の作成 (逐次・マルチスレッド環境) . . . 61
5.3.2 行列の作成 (マルチプロセス環境) . . . 62
5.3.3 関連する関数 . . . 62
5.4 Diagonal (DIA) . . . 63
5.4.1 行列の作成 (逐次環境) . . . 63
5.4.2 行列の作成 (マルチスレッド環境) . . . 64
5.4.3 行列の作成 (マルチプロセス環境) . . . 65
5.4.4 関連する関数 . . . 65

5.5 Ellpack-Itpack Generalized Diagonal (ELL) . . . 66
5.5.1 行列の作成 (逐次・マルチスレッド環境) . . . 66
5.5.2 行列の作成 (マルチプロセス環境) . . . 67
5.5.3 関連する関数 . . . 67
5.6 Jagged Diagonal (JAD) . . . 68
5.6.1 行列の作成 (逐次環境) . . . 69
5.6.2 行列の作成 (マルチスレッド環境) . . . 70
5.6.3 行列の作成 (マルチプロセス環境) . . . 71
5.6.4 関連する関数 . . . 71
5.7 Block Sparse Row (BSR) . . . 73
5.7.1 行列の作成 (逐次・マルチスレッド環境) . . . 73
5.7.2 行列の作成 (マルチプロセス環境) . . . 74
5.7.3 関連する関数 . . . 74
5.8 Block Sparse Column (BSC) . . . 75
5.8.1 行列の作成 (逐次・マルチスレッド環境) . . . 75
5.8.2 行列の作成 (マルチプロセス環境) . . . 76
5.8.3 関連する関数 . . . 76
5.9 Variable Block Row (VBR) . . . 77
5.9.1 行列の作成 (逐次・マルチスレッド環境) . . . 78
5.9.2 行列の作成 (マルチプロセス環境) . . . 79
5.9.3 関連する関数 . . . 80
5.10 Coordinate (COO) . . . 81
5.10.1 行列の作成 (逐次・マルチスレッド環境) . . . 81
5.10.2 行列の作成 (マルチプロセス環境) . . . 82
5.10.3 関連する関数 . . . 82
5.11 Dense (DNS) . . . 83
5.11.1 行列の作成 (逐次・マルチスレッド環境) . . . 83
5.11.2 行列の作成 (マルチプロセス環境) . . . 84
5.11.3 関連する関数 . . . 84

6 関数 **85**

6.1 ベクトルの操作 . . . 86
6.1.1 lis_vector_create . . . 86
6.1.2 lis_vector_destroy . . . 86
6.1.3 lis_vector_duplicate . . . 87
6.1.4 lis_vector_set_size . . . 88
6.1.5 lis_vector_get_size . . . 89
6.1.6 lis_vector_get_range . . . 89
6.1.7 lis_vector_set_value . . . 90
6.1.8 lis_vector_get_value . . . 91
6.1.9 lis_vector_set_values . . . 92
6.1.10 lis_vector_get_values . . . 93
6.1.11 lis_vector_scatter . . . 94

6.1.12 lis_vector_gather . . . 94
6.1.13 lis_vector_is_null . . . 95
6.2 行列の操作 . . . 96
6.2.1 lis_matrix_create . . . 96
6.2.2 lis_matrix_destroy . . . 96
6.2.3 lis_matrix_duplicate . . . 97
6.2.4 lis_matrix_malloc . . . 97
6.2.5 lis_matrix_set_value . . . 98
6.2.6 lis_matrix_assemble . . . 99
6.2.7 lis_matrix_set_size . . . 99
6.2.8 lis_matrix_get_size . . . 100
6.2.9 lis_matrix_get_range . . . 100
6.2.10 lis_matrix_get_nnz . . . 101
6.2.11 lis_matrix_set_type . . . 102
6.2.12 lis_matrix_get_type . . . 103
6.2.13 lis_matrix_set_csr . . . 104
6.2.14 lis_matrix_set_csc . . . 104
6.2.15 lis_matrix_set_msr . . . 105
6.2.16 lis_matrix_set_dia . . . 106
6.2.17 lis_matrix_set_ell . . . 106
6.2.18 lis_matrix_set_jad . . . 107
6.2.19 lis_matrix_set_bsr . . . 108
6.2.20 lis_matrix_set_bsc . . . 109
6.2.21 lis_matrix_set_vbr . . . 110
6.2.22 lis_matrix_set_coo . . . 111
6.2.23 lis_matrix_set_dns . . . 111
6.2.24 lis_matrix_unset . . . 112
6.3 ベクトルと行列を用いた計算 . . . 113
6.3.1 lis_vector_swap . . . 113
6.3.2 lis_vector_copy . . . 113
6.3.3 lis_vector_axpy . . . 114
6.3.4 lis_vector_xpay . . . 114
6.3.5 lis_vector_axpyz . . . 115
6.3.6 lis_vector_scale . . . 115
6.3.7 lis_vector_pmul . . . 116
6.3.8 lis_vector_pdiv . . . 116
6.3.9 lis_vector_set_all . . . 117
6.3.10 lis_vector_abs . . . 117
6.3.11 lis_vector_reciprocal . . . 118
6.3.12 lis_vector_conjugate . . . 118
6.3.13 lis_vector_shift . . . 119
6.3.14 lis_vector_dot . . . 120

6.3.15 lis_vector_nhdot . . . 120
6.3.16 lis_vector_nrm1 . . . 121
6.3.17 lis_vector_nrm2 . . . 121
6.3.18 lis_vector_nrmi . . . 122
6.3.19 lis_vector_sum . . . 122
6.3.20 lis_matrix_set_blocksize . . . 123
6.3.21 lis_matrix_convert . . . 124
6.3.22 lis_matrix_copy . . . 125
6.3.23 lis_matrix_axpy . . . 125
6.3.24 lis_matrix_xpay . . . 126
6.3.25 lis_matrix_axpyz . . . 126
6.3.26 lis_matrix_scale . . . 127
6.3.27 lis_matrix_get_diagonal . . . 127
6.3.28 lis_matrix_shift_diagonal . . . 128
6.3.29 lis_matrix_shift_matrix . . . 128
6.3.30 lis_matvec . . . 129
6.3.31 lis_matvech . . . 129
6.4 線型方程式の求解 . . . 130
6.4.1 lis_solver_create . . . 130
6.4.2 lis_solver_destroy . . . 130
6.4.3 lis_precon_create . . . 131
6.4.4 lis_precon_destroy . . . 131
6.4.5 lis_solver_set_option . . . 132
6.4.6 lis_solver_set_optionC . . . 136
6.4.7 lis_solve . . . 137
6.4.8 lis_solve_kernel . . . 138
6.4.9 lis_solver_get_status . . . 139
6.4.10 lis_solver_get_iter . . . 139
6.4.11 lis_solver_get_iterex . . . 140
6.4.12 lis_solver_get_time . . . 140
6.4.13 lis_solver_get_timeex . . . 141
6.4.14 lis_solver_get_residualnorm . . . 141
6.4.15 lis_solver_get_rhistory . . . 142
6.4.16 lis_solver_get_solver . . . 143
6.4.17 lis_solver_get_precon . . . 143
6.4.18 lis_solver_get_solvername . . . 144
6.4.19 lis_solver_get_preconname . . . 144
6.5 固有値問題の求解 . . . 145
6.5.1 lis_esolver_create . . . 145
6.5.2 lis_esolver_destroy . . . 145
6.5.3 lis_esolver_set_option . . . 146
6.5.4 lis_esolver_set_optionC . . . 150

6.5.5 lis_esolve . . . 151
6.5.6 lis_gesolve . . . 152
6.5.7 lis_esolver_get_status . . . 153
6.5.8 lis_esolver_get_iter . . . 153
6.5.9 lis_esolver_get_iterex . . . 154
6.5.10 lis_esolver_get_time . . . 154
6.5.11 lis_esolver_get_timeex . . . 155
6.5.12 lis_esolver_get_residualnorm . . . 155
6.5.13 lis_esolver_get_rhistory . . . 156
6.5.14 lis_esolver_get_evalues . . . 156
6.5.15 lis_esolver_get_evectors . . . 157
6.5.16 lis_esolver_get_residualnorms . . . 157
6.5.17 lis_esolver_get_iters . . . 158
6.5.18 lis_esolver_get_specific_evalue . . . 158
6.5.19 lis_esolver_get_specific_evector . . . 159
6.5.20 lis_esolver_get_specific_residualnorm . . . 159
6.5.21 lis_esolver_get_specific_iter . . . 160
6.5.22 lis_esolver_get_esolver . . . 160
6.5.23 lis_esolver_get_esolvername . . . 161
6.6 配列を用いた計算 . . . 162
6.6.1 lis_array_swap . . . 162
6.6.2 lis_array_copy . . . 162
6.6.3 lis_array_axpy . . . 163
6.6.4 lis_array_xpay . . . 163
6.6.5 lis_array_axpyz . . . 164
6.6.6 lis_array_scale . . . 164
6.6.7 lis_array_pmul . . . 165
6.6.8 lis_array_pdiv . . . 165
6.6.9 lis_array_set_all . . . 166
6.6.10 lis_array_abs . . . 166
6.6.11 lis_array_reciprocal . . . 167
6.6.12 lis_array_conjugate . . . 167
6.6.13 lis_array_shift . . . 168
6.6.14 lis_array_dot . . . 168
6.6.15 lis_array_nhdot . . . 169
6.6.16 lis_array_nrm1 . . . 169
6.6.17 lis_array_nrm2 . . . 170
6.6.18 lis_array_nrmi . . . 170
6.6.19 lis_array_sum . . . 171
6.6.20 lis_array_matvec . . . 172
6.6.21 lis_array_matvech . . . 173
6.6.22 lis_array_matvec_ns . . . 174

6.6.23 lis_array_matmat . . . 175
6.6.24 lis_array_matmat_ns . . . 176
6.6.25 lis_array_ge . . . 177
6.6.26 lis_array_solve . . . 177
6.6.27 lis_array_cgs . . . 178
6.6.28 lis_array_mgs . . . 178
6.6.29 lis_array_qr . . . 179
6.7 ファイルの操作 . . . 180
6.7.1 lis_input . . . 180
6.7.2 lis_input_vector . . . 181
6.7.3 lis_input_matrix . . . 182
6.7.4 lis_output . . . 183
6.7.5 lis_output_vector . . . 184
6.7.6 lis_output_matrix . . . 185
6.8 その他 . . . 186
6.8.1 lis_initialize . . . 186
6.8.2 lis_finalize . . . 186
6.8.3 lis_wtime . . . 187
6.8.4 CHKERR . . . 187
6.8.5 lis_printf . . . 188

参考文献 **189**

A ファイル形式 **195**
A.1 拡張 Matrix Market 形式 . . . 195
A.2 Harwell-Boeing 形式 . . . 196
A.3 ベクトル用拡張 Matrix Market 形式 . . . 197
A.4 ベクトル用 PLAIN 形式 . . . 197

バージョン 1.0 からの変更点

1. double-double 型 4 倍精度演算に対応.
2. Fortran コンパイラに対応.
3. Autotools に対応.
4. (a) ソルバの構造を変更.
   (b) 関数 `lis_matrix_create()`, `lis_vector_create()` の引数を変更.
   (c) コマンドラインオプションの記法を変更.

バージョン 1.1 からの変更点

1. 標準固有値問題に対応.
2. 64 ビット整数型に対応.
3. (a) 関数 `lis_output_residual_history()`, `lis_get_residual_history()` の名称をそれぞれ `lis_solver_output_rhistory()`, `lis_solver_get_rhistory()` に変更.
   (b) Fortran インタフェース `lis_vector_set_value()`, `lis_vector_get_value()` の配列起点を 1 に変更.
   (c) Fortran インタフェース `lis_vector_set_size()` の配列起点を 1 に変更.
   (d) 演算精度に関するオプションの名称を `-precision` から `-f` に変更.
4. 関数 `lis_solve_kernel()` の仕様を `lis_solve_execute()` で計算された残差を返すよう変更.
5. 整数型の仕様を変更.
   (a) C プログラムにおける整数型を `LIS_INT` に変更. `LIS_INT` の既定値は `int`. プリプロセッサマクロ `_LONGLONG` が定義された場合には, `long long int` に置き換えられる.
   (b) Fortran プログラムにおける整数型を `LIS_INTEGER` に変更. `LIS_INTEGER` の既定値は `integer`. プリプロセッサマクロ `LONGLONG` が定義された場合には, `integer*8` に置き換えられる.
6. 行列格納形式 CRS (Compressed Row Storage), CCS (Compressed Column Storage) の名称をそれぞれ CSR (Compressed Sparse Row), CSC (Compressed Sparse Column) に変更.
7. 関数 `lis_get_solvername()`, `lis_get_preconname()`, `lis_get_esolvername()` の名称をそれぞれ `lis_solver_get_solvername()`, `lis_solver_get_preconname()`, `lis_esolver_get_esolvername()` に変更.

バージョン 1.2 からの変更点

1. nmake に対応.
2. ファイル lis_config_win32.h の名称を lis_config_win.h に変更.
3. 行列格納形式 JDS (Jagged Diagonal Storage) の名称を JAD (Jagged Diagonal) に変更.
4. 関数 lis_fscan_double(), lis_bswap_double() の名称をそれぞれ lis_fscan_scalar(), lis_bswap_scalar() に変更.

バージョン 1.3 からの変更点

1. long double 型 4 倍精度演算に対応.
2. Fortran でのポインタ操作に対応.
3. 構造体 LIS_SOLVER, LIS_ESOLVER のメンバ residual の名称を rhistory に変更.
4. 構造体 LIS_SOLVER, LIS_ESOLVER のメンバ iters, iters2 の名称をそれぞれ iter, iter2 に変更.
5. 関数 lis_solver_get_iters(), lis_solver_get_itersex(), lis_esolver_get_iters(), lis_esolver_get_itersex() の名称をそれぞれ lis_solver_get_iter(), lis_solver_get_iterex(), lis_esolver_get_iter(), lis_esolver_get_iterex() に変更.
6. 構造体 LIS_SOLVER, LIS_ESOLVER のメンバ*times の名称をそれぞれ*time に変更.
7. 構造体 LIS_VECTOR にメンバ intvalue を追加.
8. 関数 lis_output_vector*(), lis_output_mm_vec() の仕様を整数値を格納できるよう変更.
9. 関数 lis_matrix_scaling*() の名称をそれぞれ lis_matrix_scale*() に変更.
10. 関数 lis_array_dot2(), lis_array_invGauss() の名称をそれぞれ lis_array_dot(), lis_array_ge() に変更.

バージョン 1.4 からの変更点

1. 配列操作に対応.
2. 前処理とソルバの分離に対応.
3. 関数 lis_array_qr() の仕様を QR 法の反復回数及び誤差を返すよう変更.
4. 関数 lis_array_matvec2(), lis_array_matmat2() の名称をそれぞれ lis_array_matvec_ns(), lis_array_matmat_ns() に変更.
5. プリプロセッサマクロ_LONGLONG, LONGLONG の名称をそれぞれ_LONG__LONG, LONG__LONG に変更.

バージョン 1.5 からの変更点

1. 複素数演算に対応.

バージョン 1.6 からの変更点

1. 一般化固有値問題に対応.
2. GCC libquadmath に対応.
3. 固有値解法のシフト量の符号を慣例に合わせて変更.
4. `lis_matrix_shift_diagonal()`, `lis_vector_shift()`, `lis_array_shift()` のシフト量の符号をそれぞれ変更.

バージョン 1.7 からの変更点

1. マルチステップ線型方程式解法に対応.
2. オプション `-ssor_w` 及び `-hybrid_w` の名称を `-ssor_omega` 及び `-hybrid_omega` にそれぞれ変更.

バージョン 1.8 からの変更点

1. 線型方程式解法を複素演算向けに一般化.
2. 非対称固有値問題に対応.
3. エルミート内積の定義 $(x, y) = y^H x$ を $(x, y) = x^H y$ に変更.
4. A^T に関する関数 `lis_lis_matvect*()`, `lis_lis_array_matvect*()`, `lis_matrix_solvet*()`, `lis_psolvet*()` の名称と定義を, A^H に関する `lis_lis_matvech*()`, `lis_lis_array_matvech*()`, `lis_matrix_solveh*()`, `lis_psolveh*()` にそれぞれ変更.
5. 関数 `lis_matrix_shift_general()` の名称を `lis_matrix_shift_matrix()` に変更.

バージョン 2.0 からの変更点

1. 特定の固有対に対する操作に対応.
2. 関数 `lis_matrix_shift_general()` の名称を `lis_matrix_shift_matrix()` に変更.

1 はじめに

Lis (Library of Iterative Solvers for linear systems, 発音は [lis]) は, 偏微分方程式の数値計算に現れる離散化された線型方程式

$$Ax = b$$

及び固有値問題

$$Ax = \lambda Bx$$

を解くための並列反復法ソフトウェアライブラリである [1]. 対応する線型方程式解法, 固有値解法の一覧を表 1-2, 前処理を表 3 に示す. また行列格納形式の一覧を表 4 に示す.

表 1: 線型方程式解法

CG[2, 3]	CR[2]
BiCG[4, 5, 6]	BiCR[7]
CGS[8]	CRS[9]
BiCGSTAB[10]	BiCRSTAB[9]
GPBiCG[11]	GPBiCR[9]
BiCGSafe[12]	BiCRSafe[13]
BiCGSTAB(l)[14]	TFQMR[15]
Jacobi[16]	Orthomin(m)[17]
Gauss-Seidel[18, 19]	GMRES(m)[20]
SOR[21, 22]	FGMRES(m)[23]
IDR(s)[24]	MINRES[25]
COCG[26]	COCR[27]
GCRO-DR[28]	FGCRO-DR[29]

表 2: 固有値解法

Power[30]
Inverse[31]
Rayleigh Quotient[32]
CG[33]
CR[34]
Subspace[36]
Lanczos[37]
Arnoldi[38]
FEAST[39]
Krylov

表 3: 前処理

Jacobi[40]
SSOR[40]
ILU(k)[41, 42]
ILUT[43, 44]
Crout ILU[44, 45]
I+S[46]
SA-AMG[47]
MFAMG[48]
Hybrid[49]
SAINV[50]
Additive Schwarz[51, 52]
ユーザ定義

表 4: 行列格納形式

Compressed Sparse Row	(CSR)
Compressed Sparse Column	(CSC)
Modified Compressed Sparse Row	(MSR)
Diagonal	(DIA)
Ellpack-Itpack Generalized Diagonal	(ELL)
Jagged Diagonal	(JAD)
Block Sparse Row	(BSR)
Block Sparse Column	(BSC)
Variable Block Row	(VBR)
Coordinate	(COO)
Dense	(DNS)

2 導入

本節では, 導入, 検証の手順について述べる.

2.1 システム要件

Lis の導入には C コンパイラが必要である. また, Fortran インタフェースを使用する場合は Fortran コンパイラ, AMG 前処理ルーチンを使用する場合は Fortran 90 コンパイラが必要である. 並列計算環境では, OpenMP ライブラリ [94] または MPI-1 ライブラリ [88] を使用する [53, 54]. データの入出力には, Harwell-Boeing 形式 [80], Matrix Market 形式 [84] が利用可能である. 表 5 に主な動作確認環境を示す (表 7 も参照のこと).

表 5: 主な動作確認環境

C コンパイラ (必須)	OS
Intel C/C++ Compiler 7.0, 8.0, 9.1, 10.1, 11.1, 12.1, 14.0, 16.0, 17.0, 18.0, 19.0, 2021.8.0	Linux Windows
IBM XL C/C++ V7.0, 9.0	AIX Linux
Sun WorkShop 6, Sun ONE Studio 7, Sun Studio 11, 12	Solaris
PGI C++ 6.0, 7.1, 10.5, 16.10	Linux
gcc 3.3, 4.4, 5.4, 6.4, 8.2, 9.3, 10.2, 11.3	Linux macOS Windows
Clang 3.3, 3.4, 3.7, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.1, 12.0	macOS FreeBSD
Microsoft Visual C++ 2008, 2010, 2012, 2013, 2015, 2017, 2019, 2022	Windows
Fortran コンパイラ (オプション)	**OS**
Intel Fortran Compiler 8.1, 9.1, 10.1, 11.1, 12.1, 14.0, 16.0, 17.0, 18.0, 19.0, 2021.8.0	Linux Windows
IBM XL Fortran V9.1, 11.1	AIX Linux
Sun WorkShop 6, Sun ONE Studio 7, Sun Studio 11, 12	Solaris
PGI Fortran 6.0, 7.1, 10.5, 16.10	Linux
g77 3.3 gfortran 4.4, 5.4, 6.4, 8.2, 10.1, 11.3	Linux macOS Windows

2.2 UNIX 及び互換システムへの導入

2.2.1 アーカイブの展開

次のコマンドを入力し, アーカイブを展開する. (`$VERSION`) はバージョンを示す.

```
> unzip lis-($VERSION).zip
```

これにより, ディレクトリ `lis-($VERSION)` に図 1 に示すサブディレクトリが作成される.

```
lis-($VERSION)
 + config
 |  設定ファイル
 + doc
 |  説明書
 + graphics
 |  描画用サンプルファイル
 + include
 |  ヘッダファイル
 + src
 |  ソースファイル
 + test
 |  検証プログラム
 + win
    Windows システム用設定ファイル
```

図 1: lis-($VERSION).zip のファイル構成

2.2.2 ソースツリーの設定

ディレクトリ lis-($VERSION) において次のコマンドを実行し, ソースツリーを設定する.

- 既定の設定を使用する場合 : `> ./configure`
- 導入先を指定する場合 : `> ./configure --prefix=<install-dir>`

表 6 に主な設定オプションを示す. また, 表 7 に TARGET として指定できる主な計算機環境を示す.

2.2.3 実行ファイルの生成

ディレクトリ lis-($VERSION) において次のコマンドを入力し, 実行ファイルを生成する.

```
> make
```

実行ファイルが正常に生成されたかどうかを確認するには, ディレクトリ lis-($VERSION) において次のコマンドを入力し, ディレクトリ lis-($VERSION)/test に生成された実行ファイルを用いて検証を行う.

```
> make check
```

このコマンドでは, Matrix Market 形式のファイル test/testmat.mtx から行列, ベクトルデータを読み込み, BiCG 法を用いて線型方程式 $Ax = b$ の解を求める. 以下に SGI Altix 3700 上での実行結果を示す. なおオプション --enable-omp と --enable-mpi は組み合わせて使用することができる.

表 6: 主な設定オプション (一覧は `./configure --help` を参照)

`--enable-omp`	OpenMP ライブラリを使用
`--enable-mpi`	MPI ライブラリを使用
`--enable-fortran`	FORTRAN 77 互換インタフェースを使用
`--enable-f90`	Fortran 90 互換インタフェースを使用
`--enable-saamg`	SA-AMG 前処理を使用
`--enable-quad`	double-double 型 4 倍精度演算を使用
`--enable-longdouble`	long double 型 4 倍精度演算を使用
`--enable-longlong`	64 ビット整数型を使用
`--enable-complex`	スカラ型として複素数型を使用
`--enable-debug`	デバッグモードを使用
`--enable-shared`	動的リンクを使用
`--enable-gprof`	プロファイラを使用
`--disable-test`	検証プログラムを作成しない
`--prefix=<install-dir>`	導入先を指定
`TARGET=<target>`	計算機環境を指定
`CC=<c_compiler>`	C コンパイラを指定
`CFLAGS=<c_flags>`	C コンパイラオプションを指定
`F77=<f77_compiler>`	FORTRAN 77 コンパイラを指定
`F77FLAGS=<f77_flags>`	FORTRAN 77 コンパイラオプションを指定
`FC=<f90_compiler>`	Fortran 90 コンパイラを指定
`FCFLAGS=<f90_flags>`	Fortran 90 コンパイラオプションを指定
`LDFLAGS=<ld_flags>`	リンクオプションを指定

表 7: TARGET の例 (詳細は lis-($VERSION)/configure.ac を参照)

<target>	等価なオプション
cray_xt3_cross	./configure CC=cc FC=ftn CFLAGS="-O3 -B -fastsse -tp k8-64" FCFLAGS="-O3 -fastsse -tp k8-64 -Mpreprocess" FCLDFLAGS="-Mnomain" ac_cv_sizeof_void_p=8 cross_compiling=yes ax_f77_mangling="lower case, no underscore, extra underscore"
fujitsu_fx10_cross	./configure CC=fccpx FC=frtpx CFLAGS="-Kfast,ocl,preex" FCFLAGS="-Kfast,ocl,preex -Cpp -fs" FCLDFLAGS="-mlcmain=main" ac_cv_sizeof_void_p=8 cross_compiling=yes ax_f77_mangling="lower case, underscore, no extra underscore"
hitachi_sr16k	./configure CC=cc FC=f90 CFLAGS="-Os -noparallel" FCFLAGS="-Oss -noparallel" FCLDFLAGS="-lf90s" ac_cv_sizeof_void_p=8 ax_f77_mangling="lower case, underscore, no extra underscore"
ibm_bgl_cross	./configure CC=blrts_xlc FC=blrts_xlf90 CFLAGS="-O3 -qarch=440d -qtune=440 -qstrict" FCFLAGS="-O3 -qarch=440d -qtune=440 -qsuffix=cpp=F90" ac_cv_sizeof_void_p=4 cross_compiling=yes ax_f77_mangling="lower case, no underscore, no extra underscore"
intel_mic_cross	./configure CC=icc F77=ifort FC=ifort MPICC=mpiicc MPIF77=mpiifort MPIFC=mpiifort CFLAGS="-mmic" FFLAGS="-mmic" FCFLAGS="-mmic" LDFLAGS="-mmic" FCLDFLAGS="-mmic" cross_compiling=yes host=x86_64-pc-linux-gnu host_alias=x86_64-linux-gnu host_cpu=x86_64 host_os=linux-gnu host_vendor=pc target=k1om-mpss-linux-gnu target_alias=k1om-mpss-linux target_cpu=k1om target_os=linux-gnu target_vendor=mpss
nec_sx9_cross	./configure CC=sxmpic++ FC=sxmpif90 AR=sxar RANLIB=true ac_cv_sizeof_void_p=8 ax_vector_machine=yes cross_compiling=yes ax_f77_mangling="lower case, no underscore, extra underscore"

既定

```
matrix size = 100 x 100 (460 nonzero entries)

initial vector x      : all components set to 0
precision             : double
linear solver         : BiCG
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
matrix storage format : CSR
linear solver status  : normal end

BiCG: number of iterations = 15 (double = 15, quad = 0)
BiCG: elapsed time         = 5.178690e-03 sec.
BiCG:   preconditioner     = 1.277685e-03 sec.
BiCG:     matrix creation  = 1.254797e-03 sec.
BiCG:   linear solver      = 3.901005e-03 sec.
BiCG: relative residual    = 6.327297e-15
```

--enable-omp

```
max number of threads = 32
number of threads = 2
matrix size = 100 x 100 (460 nonzero entries)

initial vector x      : all components set to 0
precision             : double
linear solver         : BiCG
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
matrix storage format : CSR
linear solver status  : normal end

BiCG: number of iterations = 15 (double = 15, quad = 0)
BiCG: elapsed time         = 8.960009e-03 sec.
BiCG:   preconditioner     = 2.297878e-03 sec.
BiCG:     matrix creation  = 2.072096e-03 sec.
BiCG:   linear solver      = 6.662130e-03 sec.
BiCG: relative residual    = 6.221213e-15
```

--enable-mpi

```
number of processes = 2
matrix size = 100 x 100 (460 nonzero entries)

initial vector x      : all components set to 0
precision             : double
linear solver         : BiCG
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
matrix storage format : CSR
linear solver status  : normal end

BiCG: number of iterations = 15 (double = 15, quad = 0)
BiCG: elapsed time         = 2.911400e-03 sec.
BiCG:   preconditioner     = 1.560780e-04 sec.
BiCG:     matrix creation  = 1.459997e-04 sec.
BiCG:   linear solver      = 2.755322e-03 sec.
BiCG: relative residual    = 6.221213e-15
```

2.2.4 導入

ディレクトリ lis-($VERSION) において次のコマンドを入力し, 導入先のディレクトリにファイルを複製する.

```
> make install
```

これにより, ディレクトリ ($INSTALLDIR) に以下のファイルが複製される.

```
($INSTALLDIR)
 +bin
 |   +lsolve esolve esolver gesolve gesolver hpcg_kernel hpcg_spmvtest spmvtest*
 +include
 |   +lis_config.h lis.h lisf.h
 +lib
 |   +liblis.a
 +share
     +doc/lis examples/lis man
```

lis_config.h はライブラリを生成する際に, また lis.h は C, lisf.h は Fortran でライブラリを使用する際に必要なヘッダファイルである. liblis.a は生成されたライブラリである. ライブラリが正常に導入されたかどうかを確認するには, ディレクトリ lis-($VERSION) において次のコマンドを入力し, ディレクトリ examples/lis に生成された実行ファイルを用いて検証を行う.

```
> make installcheck
```

examples/lis 下の test1, etest5, etest5b, getest5, getest5b, test3b, spmvtest3b は, lsolve,

esolve, esolver, gesolve, gesolver, hpcg_kernel, hpcg_spmvtest の別名で ($INSTALLDIR)/bin に複製される. examples/lis/spmvtest*も, それぞれ ($INSTALLDIR)/bin に複製される.

($INSTALLDIR) に複製されたファイルを削除するには，次のコマンドを入力する.

```
> make uninstall
```

lis-($VERSION) に生成されたライブラリ, 及び実行ファイルを削除するには，次のコマンドを入力する.

```
> make clean
```

生成された設定ファイルを合わせて削除するには，次のコマンドを入力する.

```
> make distclean
```

2.3 Windows システムへの導入

適当なツールを用いてアーカイブを展開した後, Microsoft Build Engine を使用する場合は, ディレクトリ lis-($VERSION)\win において次のコマンドを入力し, 設定ファイル Makefile を生成する (詳細は configure.bat --help を参照).

```
> configure.bat
```

Makefile の既定値は Makefile.in で定義される. 実行ファイルを生成するには, lis-($VERSION)\win において次のコマンドを入力する.

```
> nmake
```

実行ファイルが正常に生成されたかどうかを確認するには, 次のコマンドを入力し, 生成された実行ファイルを用いて検証を行う.

```
> nmake check
```

生成されたライブラリ, 実行ファイル, ヘッダファイル, 及び PDF 文書は, 以下のコマンドにより ($INSTALLDIR)\lib, ($INSTALLDIR)\bin, ($INSTALLDIR)\include, 及び ($INSTALLDIR)\doc にそれぞれ格納される.

```
> nmake install
```

($INSTALLDIR) に複製されたファイルを削除するには，次のコマンドを入力する.

```
> nmake uninstall
```

($INSTALLDIR)\win に生成されたライブラリ, 及び実行ファイルを削除するには，次のコマンドを入力する.

```
> nmake clean
```

生成された設定ファイルを合わせて削除するには，次のコマンドを入力する.

```
> nmake distclean
```

UNIX 互換環境を使用する場合は前節を参照のこと.

2.4 検証

検証プログラムは lis-($VERSION)/test に格納される.

2.4.1 test1

ディレクトリ lis-($VERSION)/test において

```
> test1 matrix_filename rhs_setting solution_filename rhistory_filename [options]
```

と入力すると, matrix_filename から行列データを読み込み, 線型方程式 $Ax = b$ を options で指定された解法で解く. また, 解を拡張 Matrix Market 形式で solution_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す (付録 A を参照). 入力可能な行列データ形式は拡張 Matrix Market 形式, Harwell-Boeing 形式のいずれかである. rhs_setting には

0	行列データファイルに含まれる右辺ベクトルを用いる
1	$b = (1, \ldots, 1)^T$ を用いる
2	$b = A \times (1, \ldots, 1)^T$ を用いる
rhs_filename	右辺ベクトルのファイル名

のいずれかを指定できる. rhs_filename は PLAIN 形式, Matrix Market 形式に対応する. test1f.F は test1.c の Fortran 版である.

2.4.2 test2

ディレクトリ lis-($VERSION)/test において

```
> test2 m n matrix_type solution_filename rhistory_filename [options]
```

と入力すると, 2 次元 Laplace 作用素を 5 点中心差分により離散化して得られる次数 mn の行列 A を係数とする線型方程式 $Ax = b$ を, matrix_type で指定された行列格納形式, options で指定された解法で解く. また, 解を拡張 Matrix Market 形式で solution_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. m, n は各次元の格子点数である. test2f.F90 は test2.c の Fortran 90 版である.

2.4.3 test2b

ディレクトリ lis-($VERSION)/test において

```
> test2b m n matrix_type solution_filename rhistory_filename [options]
```

と入力すると, 2 次元 Laplace 作用素を 9 点中心差分により離散化して得られる次数 mn の行列 A を係数とする線型方程式 $Ax = b$ を, matrix_type で指定された行列格納形式, options で指定された解法で解く. また, 解を拡張 Matrix Market 形式で solution_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. m, n は各次元の格子点数である.

2.4.4 test3

ディレクトリ lis-($VERSION)/test において

```
> test3 l m n matrix_type solution_filename rhistory_filename [options]
```

と入力すると, 3 次元 Laplace 作用素を 7 点中心差分により離散化して得られる次数 lmn の行列 A を係数とする線型方程式 $Ax = b$ を, matrix_type で指定された行列格納形式, options で指定された解法で解く. また, 解を拡張 Matrix Market 形式で solution_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. l, m, n は各次元の格子点数である.

2.4.5 test3b

ディレクトリ lis-($VERSION)/test において

```
> test3b l m n matrix_type solution_filename rhistory_filename [options]
```

と入力すると, 3 次元 Laplace 作用素を 27 点中心差分により離散化して得られる次数 lmn の行列 A を係数とする線型方程式 $Ax = b$ を, matrix_type で指定された行列格納形式, options で指定された解法で解く. また, 解を拡張 Matrix Market 形式で solution_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. l, m, n は各次元の格子点数である.

2.4.6 test3c

ディレクトリ lis-($VERSION)/test において

```
> test3c l m n step [options]
```

と入力すると, 3 次元 Laplace 作用素を 7 点中心差分により離散化して得られる次数 lmn の行列 A を係数とする線型方程式 $Ax = b$ を, options で指定された解法で step ステップ分解く. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. 行列, 右辺ベクトルの値はステップ毎に更新される. l, m, n は各次元の格子点数である.

2.4.7 test4

線型方程式 $Ax = b$ を指定された解法で解き, 解を標準出力に書き出す. 行列 A は次数 12 の 3 重対角行列

$$A = \begin{pmatrix} 2 & -1 & & & & \\ -1 & 2 & -1 & & & \\ & \ddots & \ddots & \ddots & & \\ & & -1 & 2 & -1 & \\ & & & -1 & 2 & \end{pmatrix}$$

である. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される. test4f.F は test4.c の Fortran 版である.

2.4.8 test5

ディレクトリ lis-($VERSION)/test において

```
> test5 n gamma [options]
```

と入力すると, 線型方程式 $Ax = b$ を指定された解法で解く. 行列 A は次数 n の Toepliz 行列

$$A = \begin{pmatrix} 2 & 1 & & & & \\ 0 & 2 & 1 & & & \\ \gamma & 0 & 2 & 1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & \gamma & 0 & 2 & 1 \\ & & & \gamma & 0 & 2 \end{pmatrix}$$

である. 右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される.

2.4.9 test6

test6.c は test2.c の配列版である．ディレクトリ lis-($VERSION)/test において

```
> test6 m n
```

と入力すると, 2 次元 Laplace 作用素を 5 点中心差分により離散化して得られる次数 mn の行列 A を係数とする線型方程式 $Ax = b$ を直接法で解く．右辺ベクトル b は解ベクトル x の値がすべて 1 となるよう設定される．m, n は各次元の格子点数である．test6f.F90 は test6.c の Fortran 90 版である．

2.4.10 test7

ディレクトリ lis-($VERSION)/test において

```
> test7
```

と入力すると, 複素演算の使用例を示す．test7f.F は test7.c の Fortran 版である．

2.4.11 test8f

ディレクトリ lis-($VERSION)/test において

```
> mpiexec -n m test8f
```

と入力すると, Newton-Raphson 法を用いて非線型偏微分方程式を解く．Newton-Raphson 法において, 前処理行列はソルバと切り離して更新される（3.7 節を参照のこと）．

2.4.12 etest1

ディレクトリ lis-($VERSION)/test において

```
> etest1 matrix_filename evector_filename rhistory_filename [options]
```

と入力すると, matrix_filename から行列データを読み込み, 標準固有値問題 $Ax = \lambda x$ を options で指定された解法で解いて, 指定された固有値を標準出力に書き出す．また, 対応する固有ベクトルを拡張 Matrix Market 形式で evector_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す．入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである．etest1f.F は etest1.c の Fortran 版である．複数の固有対を取得する場合は etest5 を参照のこと．

2.4.13 getest1

ディレクトリ lis-($VERSION)/test において

```
> getest1 matrix_a_filename matrix_b_filename evector_filename rhistory_filename
[options]
```

と入力すると, matrix_a_filename 及び matrix_b_filename から行列データを読み込み, 一般化固有値問題 $Ax = \lambda Bx$ を options で指定された解法で解いて, 指定された固有値を標準出力に書き出す．また, 対応する固有ベクトルを拡張 Matrix Market 形式で evector_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す．入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである．複数の固有対を取得する場合は getest5 を参照のこと．

2.4.14 etest2

ディレクトリ lis-($VERSION)/test において

```
> etest2 m n matrix_type evector_filename rhistory_filename [options]
```

と入力すると, 2 次元 Laplace 作用素を 5 点中心差分により離散化して得られる次数 mn の行列 A に関する固有値問題 $Ax = \lambda x$ を, matrix_type で指定された行列格納形式, options で指定された解法で解き, 指定された固有値を標準出力に書き出す. また, 対応する固有ベクトルを evector_filename に, 残差履歴を rhistory_filename に書き出す. m, n は各次元の格子点数である.

2.4.15 etest3

ディレクトリ lis-($VERSION)/test において

```
> etest3 l m n matrix_type evector_filename rhistory_filename [options]
```

と入力すると, 3 次元 Laplace 作用素を 7 点中心差分により離散化して得られる次数 lmn の行列 A に関する固有値問題 $Ax = \lambda x$ を, matrix_type で指定された行列格納形式, options で指定された解法で解き, 指定された固有値を標準出力に書き出す. また, 対応する固有ベクトルを拡張 Matrix Market 形式で evector_filename に, 残差履歴を PLAIN 形式で rhistory_filename に書き出す. l, m, n は各次元の格子点数である. 複数の固有対を取得する場合は etest6 を参照のこと.

2.4.16 etest4

ディレクトリ lis-($VERSION)/test において

```
> etest4 n [options]
```

と入力すると, 固有値問題 $Ax = \lambda x$ を指定された解法で解き, 指定された固有値を標準出力に書き出す. 行列 A は次数 n の 3 重対角行列

$$A = \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}$$

である. etest4f.F は etest4.c の Fortran 版である.

2.4.17 etest5

ディレクトリ lis-($VERSION)/test において

```
> etest5 matrix_filename evalues_filename evectors_filename residuals_filename
 iters_filename [options]
```

と入力すると, matrix_filename から行列データを読み込み, 標準固有値問題 $Ax = \lambda x$ を options で指定された解法で解く. また, オプション -ss により指定された個数の固有値を evalues_filename に, 対応する固有ベクトル, 残差ノルム及び反復回数を evectors_filename, residuals_filename 及び iters_filename に拡張 Matrix Market 形式で書き出す. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである.

2.4.18 etest5b

ディレクトリ lis-($VERSION)/test において

```
> etest5b matrix_filename evalues_filename [options]
```

と入力すると, matrix_filename から行列データを読み込み, 標準固有値問題 $Ax = \lambda x$ を options で指定された解法で解く. また, オプション -ss により指定された個数の Ritz 値を evalues_filename に拡張 Matrix Market 形式で書き出す. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである.

2.4.19 getest5

ディレクトリ lis-($VERSION)/test において

```
> getest5 matrix_a_filename matrix_b_filename evalues_filename evectors_filename
residuals_filename iters_filename [options]
```

と入力すると, matrix_a_filename 及び matrix_b_filename から行列データを読み込み, 一般化固有値問題 $Ax = \lambda Bx$ を options で指定された解法で解く. また, オプション -ss により指定された個数の固有値を evalues_filename に, 対応する固有ベクトル, 残差ノルム及び反復回数を evectors_filename, residuals_filename 及び iters_filename に拡張 Matrix Market 形式で書き出す. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである.

2.4.20 getest5b

ディレクトリ lis-($VERSION)/test において

```
> getest5b matrix_a_filename matrix_b_filename evalues_filename [options]
```

と入力すると, matrix_a_filename 及び matrix_b_filename から行列データを読み込み, 一般化固有値問題 $Ax = \lambda Bx$ を options で指定された解法で解く. また, オプション -ss により指定された個数の Ritz 値を evalues_filename に拡張 Matrix Market 形式で書き出す. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである.

2.4.21 etest6

ディレクトリ lis-($VERSION)/test において

```
> etest6 l m n matrix_type evalues_filename evectors_filename residuals_filename
iters_filename [options]
```

と入力すると, 3 次元 Laplace 作用素を 7 点中心差分により離散化して得られる次数 lmn の行列 A に関する固有値問題 $Ax = \lambda x$ を, matrix_type で指定された行列格納形式, options で指定された解法で解く. また, オプション -ss により指定された個数の固有値を evalues_filename に, 対応する固有ベクトル, 残差ノルム及び反復回数を evectors_filename, residuals_filename 及び iters_filename に拡張 Matrix Market 形式で書き出す. l, m, n は各次元の格子点数である.

2.4.22 etest7

etest7.c は etest2.c の配列版である．ディレクトリ lis-($VERSION)/test において

```
> etest7 m n
```

と入力すると, 2 次元 Laplace 作用素を 5 点中心差分により離散化して得られる次数 mn の行列 A に関する固有値問題 $Ax = \lambda x$ を QR 法で解く．m, n は各次元の格子点数である．

2.4.23 spmvtest1

ディレクトリ lis-($VERSION)/test において

```
> spmvtest1 n iter [matrix_type]
```

と入力すると, 1 次元 Laplace 作用素を 3 点中心差分により離散化して得られる次数 n の行列

$$A = \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}$$

とベクトル $(1, \ldots, 1)^T$ との積を iter で指定された回数実行し, FLOPS 値を算出する．必要なら matrix_type により,

0	実行可能なすべての行列格納形式について測定する
1-11	行列格納形式の番号

のいずれかを指定する．

2.4.24 spmvtest2

ディレクトリ lis-($VERSION)/test において

```
> spmvtest2 m n iter [matrix_type]
```

と入力すると, 2 次元 Laplace 作用素を 5 点中心差分により離散化して得られる次数 mn の 5 重対角行列とベクトル $(1, \ldots, 1)^T$ との積を iter で指定された回数実行し, FLOPS 値を算出する．必要なら matrix_type により,

0	実行可能なすべての行列格納形式について測定する
1-11	行列格納形式の番号

のいずれかを指定する．m, n は各次元の格子点数である．

2.4.25 spmvtest2b

ディレクトリ lis-($VERSION)/test において

```
> spmvtest2b m n iter [matrix_type]
```

と入力すると, 2 次元 Laplace 作用素を 9 点中心差分により離散化して得られる次数 mn の 9 重対角行列とベ

クトル $(1,\ldots,1)^T$ との積を iter で指定された回数実行し, FLOPS 値を算出する. 必要なら matrix_type により,

0	実行可能なすべての行列格納形式について測定する
1-11	行列格納形式の番号

のいずれかを指定する. m, n は各次元の格子点数である.

2.4.26 spmvtest3

ディレクトリ lis-($VERSION)/test において

```
> spmvtest3 l m n iter [matrix_type]
```

と入力すると, 3 次元 Laplace 作用素を 7 点中心差分により離散化して得られる次数 lmn の 7 重対角行列とベクトル $(1,\ldots,1)^T$ との積を iter で指定された回数実行し, FLOPS 値を算出する. 必要なら matrix_type により,

0	実行可能なすべての行列格納形式について測定する
1-11	行列格納形式の番号

のいずれかを指定する. l, m, n は各次元の格子点数である.

2.4.27 spmvtest3b

ディレクトリ lis-($VERSION)/test において

```
> spmvtest3b l m n iter [matrix_type]
```

と入力すると, 3 次元 Laplace 作用素を 27 点中心差分により離散化して得られる次数 lmn の 27 重対角行列とベクトル $(1,\ldots,1)^T$ との積を iter で指定された回数実行し, FLOPS 値を算出する. 必要なら matrix_type により,

0	実行可能なすべての行列格納形式について測定する
1-11	行列格納形式の番号

のいずれかを指定する. l, m, n は各次元の格子点数である.

2.4.28 spmvtest4

ディレクトリ lis-($VERSION)/test において

```
> spmvtest4 matrix_filename_list iter [block]
```

と入力すると, matrix_filename_list の示す行列データファイルリストから行列データを読み込み, 各行列とベクトル $(1,\ldots,1)^T$ との積を実行可能な行列格納形式について iter で指定された回数実行し, FLOPS 値を算出する. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである. 必要なら block により, BSR, BSC 形式のブロックサイズを指定する.

2.4.29 spmvtest5

ディレクトリ lis-($VERSION)/test において

```
> spmvtest5 matrix_filename matrix_type iter [block]
```

と入力すると, matrix_filename の示す行列データファイルから行列データを読み込み, 行列とベクトル $(1,\ldots,1)^T$ との積を行列格納形式 matrix_type について iter で指定された回数実行し, FLOPS 値を算出する. 入力可能な行列データ形式は Matrix Market 形式, もしくは Harwell-Boeing 形式のいずれかである. 必要なら block により, BSR, BSC 形式のブロックサイズを指定する.

2.5 制限事項

現バージョンには以下の制限がある.

- 行列格納形式
  - VBR 形式はマルチプロセス環境では使用できない.
  - CSR 形式以外の格納形式は SA-AMG 前処理では使用できない.
  - マルチプロセス環境において必要な配列を直接定義する場合は, CSR 形式を使用しなければならない. 他の格納形式を使用する場合は, 関数 lis_matrix_convert() を用いて変換を行う.
- double-double 型 4 倍精度演算 (4 節を参照)
  - 線型方程式解法のうち, Jacobi, Gauss-Seidel, SOR, IDR(s), COCG, COCR 法では使用できない.
  - 固有値解法では使用できない.
  - Hybrid 前処理での内部反復解法のうち, Jacobi, Gauss-Seidel, SOR 法では使用できない.
  - I+S, SA-AMG 前処理では使用できない.
  - double-double 型 4 倍精度演算は複素演算には対応していない.
  - long-double 型 4 倍精度演算と併用することはできない.
- 前処理
  - ILU(k) 前処理のアルゴリズムは, ブロック対角要素を並列に分解する局所 ILU 前処理 [42] に基づく. スレッド数またはプロセス数が増加するにつれて収束特性が Jacobi 前処理に近づく点に注意のこと.
  - Jacobi, SSOR 以外の前処理が選択され, かつ行列 A が CSR 形式でない場合, 前処理作成時に CSR 形式の行列 A が作成される.
  - 非対称線型方程式解法として BiCG 法が選択された場合, SA-AMG 前処理は使用できない.
  - SA-AMG 前処理はマルチスレッド計算には対応していない.
  - SA-AMG 前処理は複素演算には対応していない.
  - SAINV 前処理の前処理行列作成部分は逐次実行される.
  - ユーザ定義前処理は使用できない.

- 固有値解法
  - 複素固有値を計算する場合には, 複素演算を有効にする必要がある. 従って, 非対称行列の固有値を計算する場合は, 常に複素演算を有効にしなければならない.

3 基本操作

本節では, ライブラリの使用方法について述べる. プログラムでは, 以下の処理を行う必要がある.

- 初期化処理
- 行列の作成
- ベクトルの作成
- ソルバ (解法の情報を格納する構造体) の作成
- 行列, ベクトルへの値の代入
- 解法の設定
- 求解
- 終了処理

また, プログラムの先頭には以下のコンパイラ指示文を記述しなければならない.

- C `#include "lis.h"`
- Fortran `#include "lisf.h"`

`lis.h`, `lisf.h` は, 導入時に `($INSTALLDIR)/include` 下に格納される.

3.1 初期化・終了処理

初期化, 終了処理は以下のように記述する. 初期化処理はプログラムの最初に, 終了処理は最後に実行しなければならない.

C

```
1: #include "lis.h"
2: LIS_INT main(LIS_INT argc, char* argv[])
3: {
4:     lis_initialize(&argc, &argv);
5:     ...
6:     lis_finalize();
7: }
```

Fortran

```
1: #include "lisf.h"
2:      call lis_initialize(ierr)
3:     ...
4:      call lis_finalize(ierr)
```

初期化処理

初期化処理を行うには, 関数

- C `LIS_INT lis_initialize(LIS_INT* argc, char** argv[])`

- Fortran subroutine lis_initialize(LIS_INTEGER ierr)

を用いる. この関数は, MPI の初期化, コマンドライン引数の取得等の初期化処理を行う.

LIS_INT の既定値 int は, プリプロセッサマクロ_LONG__LONG が定義された場合には long long int に, また LIS_INTEGER の既定値 integer は, プリプロセッサマクロ LONG__LONG が定義された場合には integer*8 に置き換えられる.

終了処理

終了処理を行うには, 関数

- C　　LIS_INT lis_finalize()
- Fortran subroutine lis_finalize(LIS_INTEGER ierr)

を用いる.

3.2　ベクトルの操作

ベクトル v の次数を $global_n$ とする. ベクトル v を $nprocs$ 個のプロセスで行ブロック分割する場合の各部分ベクトルの行数を $local_n$ とする. $global_n$ が $nprocs$ で割り切れる場合は $local_n = global_n$ / $nprocs$ となる. 例えば, ベクトル v を (3.1) 式のように 2 プロセスで行ブロック分割する場合, $global_n$ と $local_n$ はそれぞれ 4 と 2 となる.

$$v = \begin{pmatrix} 0 \\ 1 \\ \hline 2 \\ 3 \end{pmatrix} \begin{matrix} \text{PE0} \\ \\ \text{PE1} \end{matrix} \tag{3.1}$$

(3.1) 式のベクトル v を作成する場合, 逐次, マルチスレッド環境ではベクトル v そのものを, マルチプロセス環境では各プロセスにプロセス数で行ブロック分割した部分ベクトルを作成する.

ベクトル v を作成するプログラムは以下のように記述する. ただし, マルチプロセス環境のプロセス数は 2 とする.

C (逐次・マルチスレッド環境)

```
 1: LIS_INT i,n;
 2: LIS_VECTOR v;
 3: n = 4;
 4: lis_vector_create(0,&v);
 5: lis_vector_set_size(v,0,n);              /* or lis_vector_set_size(v,n,0); */
 6:
 7: for(i=0;i<n;i++)
 8: {
 9:     lis_vector_set_value(LIS_INS_VALUE,i,(double)i,v);
10: }
```

C (マルチプロセス環境)

```
 1: LIS_INT i,n,is,ie;                        /* or LIS_INT i,ln,is,ie; */
 2: LIS_VECTOR v;
 3: n = 4;                                    /* ln = 2; */
 4: lis_vector_create(MPI_COMM_WORLD,&v);
 5: lis_vector_set_size(v,0,n);               /* lis_vector_set_size(v,ln,0); */
 6: lis_vector_get_range(v,&is,&ie);
 7: for(i=is;i<ie;i++)
 8: {
 9:     lis_vector_set_value(LIS_INS_VALUE,i,(double)i,v);
10: }
```

Fortran (逐次・マルチスレッド環境)

```
 1: LIS_INTEGER i,n
 2: LIS_VECTOR v
 3: n = 4
 4: call lis_vector_create(0,v,ierr)
 5: call lis_vector_set_size(v,0,n,ierr)
 6:
 7: do i=1,n
 9:     call lis_vector_set_value(LIS_INS_VALUE,i,DBLE(i),v,ierr)
10: enddo
```

Fortran (マルチプロセス環境)

```
 1: LIS_INTEGER i,n,is,ie
 2: LIS_VECTOR v
 3: n = 4
 4: call lis_vector_create(MPI_COMM_WORLD,v,ierr)
 5: call lis_vector_set_size(v,0,n,ierr)
 6: call lis_vector_get_range(v,is,ie,ierr)
 7: do i=is,ie-1
 8:     call lis_vector_set_value(LIS_INS_VALUE,i,DBLE(i),v,ierr);
 9: enddo
```

ベクトルの作成

ベクトル v の作成には, 関数

- C　　LIS_INT lis_vector_create(LIS_Comm comm, LIS_VECTOR *v)
- Fortran subroutine lis_vector_create(LIS_Comm comm, LIS_VECTOR v, LIS_INTEGER ierr)

を用いる. comm には MPI コミュニケータを指定する. 逐次, マルチスレッド環境では comm の値は無視される.

次数の設定

次数の設定には, 関数

- C　　LIS_INTEGER lis_vector_set_size(LIS_VECTOR v, LIS_INT local_n,
  LIS_INT global_n)
- Fortran subroutine lis_vector_set_size(LIS_VECTIR v, LIS_INTEGER local_n,
  LIS_INTEGER global_n, LIS_INTEGER ierr)

を用いる. $local_n$ か $global_n$ のどちらか一方を与えなければならない.

逐次, マルチスレッド環境では, $local_n$ は $global_n$ に等しい. したがって, `lis_vector_set_size(v,n,0)` と `lis_vector_set_size(v,0,n)` は, いずれも次数 n のベクトルを作成する.

マルチプロセス環境においては, `lis_vector_set_size(v,n,0)` は各プロセス上に次数 n の部分ベクトルを作成する. 一方, `lis_vector_set_size(v,0,n)` は各プロセス p 上に次数 m_p の部分ベクトルを作成する. m_p はライブラリ側で決定される.

値の代入

ベクトル v の第 i 行に値を代入するには, 関数

- C `LIS_INT lis_vector_set_value(LIS_INT flag, LIS_INT i, LIS_SCALAR value, LIS_VECTOR v)`
- Fortran `subroutine lis_vector_set_value(LIS_INTEGER flag, LIS_INTEGER i, LIS_SCALAR value, LIS_VECTOR v, LIS_INTEGER ierr)`

を用いる. マルチプロセス環境では, 部分ベクトルの第 i 行ではなく, 全体ベクトルの第 i 行を指定する. `flag` には

`LIS_INS_VALUE` 挿入: $v[i] = value$

`LIS_ADD_VALUE` 加算代入: $v[i] = v[i] + value$

のどちらかを指定する.

ベクトルの複製

既存のベクトルと同じ情報を持つベクトルを作成するには, 関数

- C `LIS_INT lis_vector_duplicate(LIS_VECTOR vin, LIS_VECTOR *vout)`
- Fortran `subroutine lis_vector_duplicate(LIS_VECTOR vin, LIS_VECTOR vout, LIS_INTEGER ierr)`

を用いる. 第 1 引数 `LIS_VECTOR vin` は `LIS_MATRIX` を指定することも可能である. この関数はベクトルの要素の値は複製しない. 値も複製する場合は, この関数の後に

- C `LIS_INT lis_vector_copy(LIS_VECTOR vsrc, LIS_VECTOR vdst)`
- Fortran `subroutine lis_vector_copy(LIS_VECTOR vsrc, LIS_VECTOR vdst, LIS_INTEGER ierr)`

を呼び出す.

ベクトルの破棄

不要になったベクトルをメモリから破棄するには,

- C `LIS_INT lis_vector_destroy(LIS_VECTOR v)`
- Fortran `subroutine lis_vector_destroy(LIS_VECTOR v, LIS_INTEGER ierr)`

を用いる.

3.3 行列の操作

行列 A の次数を $global_n \times global_n$ とする. 行列 A を $nprocs$ 個のプロセスで行ブロック分割する場合の各ブロックの行数を $local_n$ とする. $global_n$ が $nprocs$ で割り切れる場合は $local_n = global_n$ / $nprocs$ となる. 例えば, 行列 A を (3.2) 式のように 2 個のプロセスで行ブロック分割する場合, $global_n$ と $local_n$ はそれぞれ 4 と 2 となる.

$$A = \left(\begin{array}{cccc} 2 & 1 & & \\ 1 & 2 & 1 & \\ \hline & 1 & 2 & 1 \\ & & 1 & 2 \end{array}\right) \begin{array}{l} \text{PE0} \\ \\ \text{PE1} \\ \\ \end{array} \tag{3.2}$$

目的の格納形式の行列を作成するには以下の 3 つの方法がある.

方法 1: ライブラリ関数を用いて目的の格納形式の配列を定義する場合

(3.2) 式の行列 A を CSR 形式で作成する場合, 逐次, マルチスレッド環境では行列 A そのものを, マルチプロセス環境では各プロセスにプロセス数で行ブロック分割した部分行列を作成する.

行列 A を CSR 形式で作成するプログラムは以下のように記述する. ただし, マルチプロセス環境のプロセス数は 2 とする.

C (逐次・マルチスレッド環境)

```
 1: LIS_INT i,n;
 2: LIS_MATRIX A;
 3: n = 4;
 4: lis_matrix_create(0,&A);
 5: lis_matrix_set_size(A,0,n);              /* or lis_matrix_set_size(A,n,0); */
 6: for(i=0;i<n;i++) {
 7:     if( i>0   ) lis_matrix_set_value(LIS_INS_VALUE,i,i-1,1.0,A);
 8:     if( i<n-1 ) lis_matrix_set_value(LIS_INS_VALUE,i,i+1,1.0,A);
 9:     lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0,A);
10: }
11: lis_matrix_set_type(A,LIS_MATRIX_CSR);
12: lis_matrix_assemble(A);
```

C (マルチプロセス環境)

```
 1: LIS_INT i,n,gn,is,ie;
 2: LIS_MATRIX A;
 3: gn = 4;                                  /* or n=2 */
 4: lis_matrix_create(MPI_COMM_WORLD,&A);
 5: lis_matrix_set_size(A,0,gn);             /* lis_matrix_set_size(A,n,0); */
 6: lis_matrix_get_size(A,&n,&gn);
 7: lis_matrix_get_range(A,&is,&ie);
 8: for(i=is;i<ie;i++) {
 9:     if( i>0    ) lis_matrix_set_value(LIS_INS_VALUE,i,i-1,1.0,A);
10:     if( i<gn-1 ) lis_matrix_set_value(LIS_INS_VALUE,i,i+1,1.0,A);
11:     lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0,A);
12: }
13: lis_matrix_set_type(A,LIS_MATRIX_CSR);
14: lis_matrix_assemble(A);
```

Fortran (逐次・マルチスレッド環境)

```
 1: LIS_INTEGER i,n
 2: LIS_MATRIX A
 3: n = 4
 4: call lis_matrix_create(0,A,ierr)
 5: call lis_matrix_set_size(A,0,n,ierr)
 6: do i=1,n
 7:     if( i>1 ) call lis_matrix_set_value(LIS_INS_VALUE,i,i-1,1.0d0,A,ierr)
 8:     if( i<n ) call lis_matrix_set_value(LIS_INS_VALUE,i,i+1,1.0d0,A,ierr)
 9:     call lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0d0,A,ierr)
10: enddo
11: call lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
12: call lis_matrix_assemble(A,ierr)
```

Fortran (マルチプロセス環境)

```
 1: LIS_INTEGER i,n,gn,is,ie
 2: LIS_MATRIX A
 3: gn = 4
 4: call lis_matrix_create(MPI_COMM_WORLD,A,ierr)
 5: call lis_matrix_set_size(A,0,gn,ierr)
 6: call lis_matrix_get_size(A,n,gn,ierr)
 7: call lis_matrix_get_range(A,is,ie,ierr)
 8: do i=is,ie-1
 9:     if( i>1  ) call lis_matrix_set_value(LIS_INS_VALUE,i,i-1,1.0d0,A,ierr)
10:     if( i<gn ) call lis_matrix_set_value(LIS_INS_VALUE,i,i+1,1.0d0,A,ierr)
11:     call lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0d0,A,ierr)
12: enddo
13: call lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
14: call lis_matrix_assemble(A,ierr)
```

行列の作成

行列 A の作成には, 関数

- C　　LIS_INT lis_matrix_create(LIS_Comm comm, LIS_MATRIX *A)
- Fortran subroutine lis_matrix_create(LIS_Comm comm, LIS_MATRIX A, LIS_INTEGER ierr)

を用いる. comm には MPI コミュニケータを指定する. 逐次, マルチスレッド環境では, comm の値は無視される.

次数の設定

次数の設定には, 関数

- C　　LIS_INT lis_matrix_set_size(LIS_MATRIX A, LIS_INT local_n, LIS_INT global_n)
- Fortran subroutine lis_matrix_set_size(LIS_MATRIX A, LIS_INTEGER local_n, LIS_INTEGER global_n, LIS_INTEGER ierr)

を用いる. *local_n* か *global_n* のどちらか一方を与えなければならない.

逐次, マルチスレッド環境では, *local_n* は *global_n* に等しい. したがって, lis_matrix_set_size(A,n,0) と lis_matrix_set_size(A,0,n) は, いずれも次数 $n \times n$ の行列を作成する.

マルチプロセス環境においては, lis_matrix_set_size(A,n,0) は各プロセス上に次数 $n \times N$ の部分行列を作成する. N は n の総和である.
一方, lis_matrix_set_size(A,0,n) は各プロセス p 上に次数 $m_p \times n$ の部分行列を作成する. m_p はライブラリ側で決定される.

値の代入

行列 A の第 i 行第 j 列に値を代入するには, 関数

- C
```
LIS_INT lis_matrix_set_value(LIS_INT flag, LIS_INT i, LIS_INT j,
 LIS_SCALAR value, LIS_MATRIX A)
```
- Fortran
```
subroutine lis_matrix_set_value(LIS_INTEGER flag, LIS_INTEGER i,
 LIS_INTEGER j, LIS_SCALAR value, LIS_MATRIX A, LIS_INTEGER ierr)
```

を用いる. マルチプロセス環境では, 全体行列の第 i 行第 j 列を指定する. flag には

LIS_INS_VALUE 挿入: $A[i,j] = value$

LIS_ADD_VALUE 加算代入: $A[i,j] = A[i,j] + value$

のどちらかを指定する.

行列格納形式の設定

行列の格納形式を設定するには, 関数

- C
```
LIS_INT lis_matrix_set_type(LIS_MATRIX A, LIS_INT matrix_type)
```
- Fortran
```
subroutine lis_matrix_set_type(LIS_MATRIX A, LIS_INTEGER matrix_type,
 LIS_INTEGER ierr)
```

を用いる. 行列作成時の A の matrix_type は LIS_MATRIX_CSR である. 以下に対応する格納形式を示す.

格納形式		matrix_type
Compressed Sparse Row	(CSR)	{LIS_MATRIX_CSR|1}
Compressed Sparse Column	(CSC)	{LIS_MATRIX_CSC|2}
Modified Compressed Sparse Row	(MSR)	{LIS_MATRIX_MSR|3}
Diagonal	(DIA)	{LIS_MATRIX_DIA|4}
Ellpack-Itpack Generalized Diagonal	(ELL)	{LIS_MATRIX_ELL|5}
Jagged Diagonal	(JAD)	{LIS_MATRIX_JAD|6}
Block Sparse Row	(BSR)	{LIS_MATRIX_BSR|7}
Block Sparse Column	(BSC)	{LIS_MATRIX_BSC|8}
Variable Block Row	(VBR)	{LIS_MATRIX_VBR|9}
Coordinate	(COO)	{LIS_MATRIX_COO|10}
Dense	(DNS)	{LIS_MATRIX_DNS|11}

行列の組み立て

行列の要素と格納形式を設定した後, 関数

- C　　LIS_INT lis_matrix_assemble(LIS_MATRIX A)
- Fortran subroutine lis_matrix_assemble(LIS_MATRIX A, LIS_INTEGER ierr)

を呼び出す. lis_matrix_assemble は lis_matrix_set_type で指定された格納形式に組み立てられる.

行列の破棄

不要になった行列をメモリから破棄するには,

- C　　LIS_INT lis_matrix_destroy(LIS_MATRIX A)
- Fortran subroutine lis_matrix_destroy(LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

方法 2: 目的の格納形式の配列を直接定義する場合

(3.2) 式の行列 A を CSR 形式で作成する場合, 逐次, マルチスレッド環境では行列 A そのものを, マルチプロセス環境では各プロセスにプロセス数で行ブロック分割した部分行列を作成する.

行列 A を CSR 形式で作成するプログラムは以下のように記述する. ただし, マルチプロセス環境のプロセス数は 2 とする.

C (逐次・マルチスレッド環境)

```
 1: LIS_INT i,k,n,nnz;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 10; k = 0;
 6: lis_matrix_malloc_csr(n,nnz,&ptr,&index,&value);
 7: lis_matrix_create(0,&A);
 8: lis_matrix_set_size(A,0,n);              /* or lis_matrix_set_size(A,n,0); */
 9:
10: for(i=0;i<n;i++)
11: {
12:     if( i>0   ) {index[k] = i-1; value[k] = 1; k++;}
13:     index[k] = i; value[k] = 2; k++;
14:     if( i<n-1 ) {index[k] = i+1; value[k] = 1; k++;}
15:     ptr[i+1] = k;
16: }
17: ptr[0] = 0;
18: lis_matrix_set_csr(nnz,ptr,index,value,A);
19: lis_matrix_assemble(A);
```

C (マルチプロセス環境)

```
 1: LIS_INT i,k,n,nnz,is,ie;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 2; nnz = 5; k = 0;
 6: lis_matrix_malloc_csr(n,nnz,&ptr,&index,&value);
 7: lis_matrix_create(MPI_COMM_WORLD,&A);
 8: lis_matrix_set_size(A,n,0);
 9: lis_matrix_get_range(A,&is,&ie);
10: for(i=is;i<ie;i++)
11: {
12:     if( i>0   ) {index[k] = i-1; value[k] = 1; k++;}
13:     index[k] = i; value[k] = 2; k++;
14:     if( i<n-1 ) {index[k] = i+1; value[k] = 1; k++;}
15:     ptr[i-is+1] = k;
16: }
17: ptr[0] = 0;
18: lis_matrix_set_csr(nnz,ptr,index,value,A);
19: lis_matrix_assemble(A);
```

配列の関連付け

CSR 形式の配列をライブラリが扱えるよう行列 A に関連付けるには, 関数

- C `LIS_INT lis_matrix_set_csr(LIS_INT nnz, LIS_INT ptr[], LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)`
- Fortran `subroutine lis_matrix_set_csr(LIS_INTEGER nnz, LIS_INTEGER ptr(), LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)`

を用いる. その他の格納形式については 5 節を参照のこと.

方法 3: 外部ファイルから行列, ベクトルデータを読み込む場合

外部ファイルから (3.2) 式の行列 A を CSR 形式で読み込む場合, プログラムは以下のように記述する.

C (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A;
 3: lis_matrix_create(LIS_COMM_WORLD,&A);
 6: lis_matrix_set_type(A,LIS_MATRIX_CSR);
 7: lis_input_matrix(A,"matvec.mtx");
```

Fortran (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A
 3: call lis_matrix_create(LIS_COMM_WORLD,A,ierr)
 6: call lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
 7: call lis_input_matrix(A,'matvec.mtx',ierr)
```

Matrix Market 形式による外部ファイル `matvec.mtx` の記述例を以下に示す.

```
%%MatrixMarket matrix coordinate real general
4 4 10 1 0
1 2  1.0e+00
```

```
1 1  2.0e+00
2 3  1.0e+00
2 1  1.0e+00
2 2  2.0e+00
3 4  1.0e+00
3 2  1.0e+00
3 3  2.0e+00
4 4  2.0e+00
4 3  1.0e+00
```

外部ファイルから (3.2) 式の行列 A を CSR 形式で, また (3.1) 式のベクトル b を読み込む場合のプログラムは以下のように記述する.

C (逐次・マルチスレッド・マルチプロセス環境)

```
1: LIS_MATRIX A;
2: LIS_VECTOR b,x;
3: lis_matrix_create(LIS_COMM_WORLD,&A);
4: lis_vector_create(LIS_COMM_WORLD,&b);
5: lis_vector_create(LIS_COMM_WORLD,&x);
6: lis_matrix_set_type(A,LIS_MATRIX_CSR);
7: lis_input(A,b,x,"matvec.mtx");
```

Fortran (逐次・マルチスレッド・マルチプロセス環境)

```
1: LIS_MATRIX A
2: LIS_VECTOR b,x
3: call lis_matrix_create(LIS_COMM_WORLD,A,ierr)
4: call lis_vector_create(LIS_COMM_WORLD,b,ierr)
5: call lis_vector_create(LIS_COMM_WORLD,x,ierr)
6: call lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
7: call lis_input(A,b,x,'matvec.mtx',ierr)
```

拡張 Matrix Market 形式による外部ファイル `matvec.mtx` の記述例を以下に示す (付録 A を参照).

```
%%MatrixMarket matrix coordinate real general
4 4 10 1 0
1 2  1.0e+00
1 1  2.0e+00
2 3  1.0e+00
2 1  1.0e+00
2 2  2.0e+00
3 4  1.0e+00
3 2  1.0e+00
3 3  2.0e+00
4 4  2.0e+00
4 3  1.0e+00
1  0.0e+00
2  1.0e+00
3  2.0e+00
4  3.0e+00
```

外部ファイルからの読み込み

外部ファイルから行列 A のデータを読み込むには, 関数

- C　　`LIS_INT lis_input_matrix(LIS_MATRIX A, char *filename)`

- Fortran subroutine lis_input_matrix(LIS_MATRIX A,
  character filename, LIS_INTEGER ierr)

を用いる. filename にはファイルパスを指定する. 対応するファイル形式は以下の通りである (ファイル形式については付録 A を参照).

- Matrix Market 形式
- Harwell-Boeing 形式

外部ファイルから行列 A とベクトル b, x のデータを読み込むには, 関数

- C LIS_INT lis_input(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x, char *filename)
- Fortran subroutine lis_input(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
  character filename, LIS_INTEGER ierr)

を用いる. filename にはファイルパスを指定する. 対応するファイル形式は以下の通りである (ファイル形式については付録 A を参照).

- 拡張 Matrix Market 形式
- Harwell-Boeing 形式

3.4 線型方程式の求解

線型方程式 $Ax = b$ を指定された解法で解く場合, プログラムは以下のように記述する.

C (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A;
 2: LIS_VECTOR b,x;
 3: LIS_SOLVER solver;
 4:
 5: /* 行列とベクトルの作成 */
 6:
 7: lis_solver_create(&solver);
 8: lis_solver_set_option("-i bicg -p none",solver);
 9: lis_solver_set_option("-tol 1.0e-12",solver);
10: lis_solve(A,b,x,solver);
```

Fortran (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A
 2: LIS_VECTOR b,x
 3: LIS_SOLVER solver
 4:
 5: /* 行列とベクトルの作成 */
 6:
 7: call lis_solver_create(solver,ierr)
 8: call lis_solver_set_option('-i bicg -p none',solver,ierr)
 9: call lis_solver_set_option('-tol 1.0e-12',solver,ierr)
10: call lis_solve(A,b,x,solver,ierr)
```

ソルバの作成

ソルバ (線型方程式解法の情報を格納する構造体) を作成するには, 関数

- C　　LIS_INT lis_solver_create(LIS_SOLVER *solver)
- Fortran subroutine lis_solver_create(LIS_SOLVER solver, LIS_INTEGER ierr)

を用いる.

残差の監視とカスタム収束判定

アプリケーションは、コマンドラインオプションを変更することなく、線型ソルバに C 言語のコールバック関数を追加できる。残差モニタは以下の関数で登録する。

```
LIS_INT lis_solver_set_monitor(
    LIS_SOLVER solver,
    LIS_SOLVER_MONITOR_CALLBACK monitor,
    void *user_data);
```

また、カスタム収束コールバックは以下の関数で登録する。

```
LIS_INT lis_solver_set_convergence(
    LIS_SOLVER solver,
    LIS_SOLVER_CONVERGENCE_CALLBACK convergence,
    void *user_data);
```

モニタは残差の推移を監視するだけであり、ソルバの意思決定を変更しない。一方で、収束コールバックは、収束候補を受け入れる場合は 0 以外の値を返し、拒否する場合は 0 を返す。コールバックに null を渡すと、デフォルトの挙動に戻る。user_data の所有権はアプリケーション側が保持する。

コールバックは、以下の残差分類のいずれかを受け取る。

```
LIS_SOLVER_RESIDUAL_RECURSIVE
LIS_SOLVER_RESIDUAL_PROJECTED
LIS_SOLVER_RESIDUAL_TRUE
```

LIS_SOLVER_RESIDUAL_PROJECTED はクリロフ部分空間に射影された残差の推定値を表し、LIS_SOLVER_RESIDUAL_TRUE は元の線型方程式から明示的に計算された真の残差を表す。リスタート型 GMRES 法ソルバは、射影残差を使って収束候補を形成し、ソルバレベルで成功（収束）とする前に、真の残差による判定を行う。

精度切り替え型 GMRES の場合、倍精度への切り替え閾値がモニタに報告されるが、これはアルゴリズム上の精度切り替え基準のまま維持される。カスタム収束は最終的な収束フェーズを制御するものであり、倍精度から 4 倍精度への切り替え閾値を再定義することはない。

一般的な初期残差の経路では、真の残差が報告される。また、独立して検証された物理的な残差によって、失敗した反復試行が成功へと変更されるような場面（シーケンス復元時）であっても、シーケンス復元はカスタム収束による拒否を尊重する。

カスタムコールバックを設定した場合でも、lis_solve 内の最終的な安全性検証を回避することはできない。最終的な真の残差が非有限値（NaN や Inf など）であるか、ソルバの許容誤差を超えている場合は、反復計算の結果が成功と判定されていても拒否される。コールバックの設定はソルバオブジェクトの一部であり、lis_solver_reset_sequence の実行後も保持される。

オプションの設定

線型方程式解法をソルバに設定するには, 関数

- C　　LIS_INT lis_solver_set_option(char *text, LIS_SOLVER solver)
- Fortran subroutine lis_solver_set_option(character text, LIS_SOLVER solver, LIS_INTEGER ierr)

または

- C　　LIS_INT lis_solver_set_optionC(LIS_SOLVER solver)
- Fortran subroutine lis_solver_set_optionC(LIS_SOLVER solver, LIS_INTEGER ierr)

を用いる. lis_solver_set_optionC は, ユーザプログラム実行時にコマンドラインで指定されたオプションをソルバに設定する関数である.

以下に指定可能なコマンドラインオプションを示す. -i {cg|1}は-i cg または-i 1 を意味する. -maxiter [1000] は, -maxiter の既定値が 1000 であることを意味する.

線型方程式解法に関するオプション (既定値: `-i bicg`)

線型方程式解法	オプション	補助オプション	
CG	`-i {cg|1}`		
BiCG	`-i {bicg|2}`		
CGS	`-i {cgs|3}`		
BiCGSTAB	`-i {bicgstab|4}`		
BiCGSTAB(l)	`-i {bicgstabl|5}`	`-ell [2]`	次数 l
GPBiCG	`-i {gpbicg|6}`		
TFQMR	`-i {tfqmr|7}`		
Orthomin(m)	`-i {orthomin|8}`	`-restart [40]`	リスタート値 m
GMRES(m)	`-i {gmres|9}`	`-restart [40]`	リスタート値 m
Jacobi	`-i {jacobi|10}`		
Gauss-Seidel	`-i {gs|11}`		
SOR	`-i {sor|12}`	`-omega [1.9]`	緩和係数 ω $(0 < \omega < 2)$
BiCGSafe	`-i {bicgsafe|13}`		
CR	`-i {cr|14}`		
BiCR	`-i {bicr|15}`		
CRS	`-i {crs|16}`		
BiCRSTAB	`-i {bicrstab|17}`		
GPBiCR	`-i {gpbicr|18}`		
BiCRSafe	`-i {bicrsafe|19}`		
FGMRES(m)	`-i {fgmres|20}`	`-restart [40]`	リスタート値 m
IDR(s)	`-i {idrs|21}`	`-irestart [2]`	リスタート値 s
IDR(1)	`-i {idr1|22}`		
MINRES	`-i {minres|23}`		
COCG	`-i {cocg|24}`		
COCR	`-i {cocr|25}`		
GCRO-DR	`-i {gcrodr|26}`	`-gcrodr_k [40]`	GCRO-DR の部分空間次元数
		`-gcrodr_keep [8]`	リサイクルするベクトル数
		`-gcrodr_keep_quality [0.9]`	ベクトルの保持閾値
		`-gcrodr_reject_quality [0.35]`	ベクトルの破棄閾値
FGCRO-DR	`-i {fgcrodr|27}`	`-restart [40]`	リスタート値 m
		`-fgcrodr_k [8]`	部分空間次元数
		`-fgcrodr_keep [0.90]`	空間の保持閾値
		`-fgcrodr_reject [0.10]`	空間の破棄閾値
		`-fgcrodr_update [true]`	リサイクル空間を更新

前処理に関するオプション (既定値: `-p none`)

前処理	オプション	補助オプション	
なし	`-p {none|0}`		
Jacobi	`-p {jacobi|1}`		
ILU(k)	`-p {ilu|2}`	`-ilu_fill [0]`	フィルインレベル k
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
SSOR	`-p {ssor|3}`	`-ssor_omega [1.0]`	緩和係数 ω $(0 < \omega < 2)$
Hybrid	`-p {hybrid|4}`	`-hybrid_i [sor]`	線型方程式解法
		`-hybrid_maxiter [25]`	最大反復回数
		`-hybrid_tol [1.0e-3]`	収束判定基準
		`-hybrid_omega [1.5]`	SOR の緩和係数 ω $(0 < \omega < 2)$
		`-hybrid_ell [2]`	BiCGSTAB(l) の次数 l
		`-hybrid_restart [40]`	GMRES(m), Orthomin(m) の リスタート値 m
I+S	`-p {is|5}`	`-is_alpha [1.0]`	$I + \alpha S^{(m)}$ のパラメータ α
		`-is_m [3]`	$I + \alpha S^{(m)}$ のパラメータ m
SAINV	`-p {sainv|6}`	`-sainv_drop [0.05]`	ドロップ基準
SA-AMG	`-p {saamg|7}`	`-saamg_unsym [false]`	非対称版の選択 (行列構造は対称とする)
		`-saamg_theta [0.05|0.12]`	ドロップ基準 $a_{ij}^2 \leq \theta^2 \lvert a_{ii}\rvert \lvert a_{jj}\rvert$ (対称|非対称)
Crout ILU	`-p {iluc|8}`	`-iluc_drop [0.05]`	ドロップ基準
		`-iluc_rate [5.0]`	最大フィルイン数の倍率
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
ILUT	`-p {ilut|9}`	`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
MFAMG	`-p {mfamg|12}`		
		`-mfamg_block_size [1]`	ローカル連続平滑化ブロックサイズ (1: 重み付きヤコビ 2-: ブロックヤコビ)
Additive Schwarz	`-adds true`	`-adds_iter [1]`	反復回数

その他のオプション

オプション	
-maxiter [1000]	最大反復回数
-maxiter_noimp [0]	残差が改善しない状態が指定した反復回数連続した場合に、計算を早期に打ち切る （デフォルト: 0、早期打ち切りを無効化）
-tol [1.0e-12]	収束判定基準 tol
-tol_w [1.0]	収束判定基準 tol_w
-print [0]	残差履歴の出力
	-print {none|0} 残差履歴を出力しない
	-print {mem|1} 残差履歴をメモリに保存する
	-print {out|2} 残差履歴を標準出力に書き出す
	-print {all|3} 残差履歴をメモリに保存し, 標準出力に書き出す
-scale [0]	スケーリングの選択. 結果は元の行列, ベクトルに上書きされる
	-scale {none|0} スケーリングなし
	-scale {jacobi|1} Jacobi スケーリング $D^{-1}Ax = D^{-1}b$ (D は $A = (a_{ij})$ の対角部分)
	-scale {symm_diag|2} 対角スケーリング $D^{-1/2}AD^{-1/2}x = D^{-1/2}b$ ($D^{-1/2}$ は対角要素の値が $1/\sqrt{a_{ii}}$ である対角行列)
-initx_zeros [1]	初期ベクトル x_0
	-initx_zeros {false|0} 関数 lis_solve() の引数 x により与えられる要素の値を使用
	-initx_zeros {true|1} すべての要素の値を 0 にする
-conv_cond [0]	収束条件
	-conv_cond {nrm2_r|0} $\|\|b - Ax\|\|_2 \leq tol * \|\|b - Ax_0\|\|_2$
	-conv_cond {nrm2_b|1} $\|\|b - Ax\|\|_2 \leq tol * \|\|b\|\|_2$
	-conv_cond {nrm1_b|2} $\|\|b - Ax\|\|_1 \leq tol_w * \|\|b\|\|_1 + tol$
-omp_num_threads [t]	実行スレッド数 (t は最大スレッド数)
-storage [0]	行列格納形式
-storage_block [2]	BSR, BSC 形式のブロックサイズ
-f [0]	線型方程式解法の精度
	-f {double|0} 倍精度
	-f {quad|1} double-double 型 4 倍精度

求解

線型方程式 $Ax = b$ を解くには, 関数

- C
```
LIS_INT lis_solve(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
 LIS_SOLVER solver)
```
- Fortran
```
subroutine lis_solve(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
 LIS_SOLVER solver, LIS_INTEGER ierr)
```

を用いる.

3.5 固有値問題の求解

標準固有値問題 $Ax = \lambda x$ を指定された解法で解く場合, プログラムは以下のように記述する.

C (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A;
 2: LIS_VECTOR x;
 3: LIS_REAL evalue;
 4: LIS_ESOLVER esolver;
 5:
 6: /* 行列とベクトルの作成 */
 7:
 8: lis_esolver_create(&esolver);
 9: lis_esolver_set_option("-e ii -i bicg -p none",esolver);
10: lis_esolver_set_option("-etol 1.0e-12 -tol 1.0e-12",esolver);
11: lis_esolve(A,x,evalue,esolver);
```

Fortran (逐次・マルチスレッド・マルチプロセス環境)

```
 1: LIS_MATRIX A
 2: LIS_VECTOR x
 3: LIS_REAL evalue
 4: LIS_ESOLVER esolver
 5:
 6: /* 行列とベクトルの作成 */
 7:
 8: call lis_esolver_create(esolver,ierr)
 9: call lis_esolver_set_option('-e ii -i bicg -p none',esolver,ierr)
10: call lis_esolver_set_option('-etol 1.0e-12 -tol 1.0e-12',esolver,ierr)
11: call lis_esolve(A,x,evalue,esolver,ierr)
```

ソルバの作成

ソルバ (固有値解法の情報を格納する構造体) を作成するには, 関数

- C　　LIS_INT lis_esolver_create(LIS_ESOLVER *esolver)
- Fortran subroutine lis_esolver_create(LIS_ESOLVER esolver, LIS_INTEGER ierr)

を用いる.

オプションの設定

固有値解法をソルバに設定するには, 関数

- C　　LIS_INT lis_esolver_set_option(char *text, LIS_ESOLVER esolver)
- Fortran subroutine lis_esolver_set_option(character text, LIS_ESOLVER esolver, LIS_INTEGER ierr)

または

- C　　LIS_INT lis_esolver_set_optionC(LIS_ESOLVER esolver)
- Fortran subroutine lis_esolver_set_optionC(LIS_ESOLVER esolver, LIS_INTEGER ierr)

を用いる. lis_esolver_set_optionC は, ユーザプログラム実行時にコマンドラインで指定されたオプションをソルバに設定する関数である.

以下に指定可能なコマンドラインオプションを示す. -e {pi|1}は-e pi または-e 1 を意味する. -emaxiter [1000] は, -emaxiter の既定値が 1000 であることを意味する.

固有値解法に関するオプション (既定値: -e cr)

固有値解法	オプション	補助オプション	
Power	-e {pi|1}		
Inverse	-e {ii|2}	-i [bicg]	線型方程式解法
Rayleigh Quotient	-e {rqi|3}	-i [bicg]	線型方程式解法
CG	-e {cg|4}	-i [cg]	線型方程式解法
CR	-e {cr|5}	-i [bicg]	線型方程式解法
Subspace	-e {si|6}	-ss [1]	部分空間の大きさ
Lanczos	-e {li|7}	-ss [1]	部分空間の大きさ
Arnoldi	-e {ai|8}	-ss [1]	部分空間の大きさ
Generalized Power	-e {gpi|9}	-i [bicg]	線型方程式解法
Generalized Inverse	-e {gii|10}	-i [bicg]	線型方程式解法
Generalized Rayleigh Quotient	-e {gii|11}	-i [bicg]	線型方程式解法
Generalized CG	-e {gcg|12}	-i [cg]	線型方程式解法
Generalized CR	-e {gcr|13}	-i [bicg]	線型方程式解法
Generalized Subspace	-e {gsi|14}	-ss [1]	部分空間の大きさ
Generalized Lanczos	-e {gli|15}	-ss [1]	部分空間の大きさ
Generalized Arnoldi	-e {gai|16}	-ss [1]	部分空間の大きさ
Native FEAST	-e {feast|17}	-ss [1]	探索部分空間の大きさ
		-feast_emin	ターゲット区間の下限
		-feast_emax	ターゲット区間の上限
		-feast_ratio [0.30]	楕円積分経路のアスペクト比
		-feast_points [8]	積分経路上のサンプリング点数
		-feast_solver [gmres]	内部線型方程式解法
Native Krylov	-e {krylov|18}	-ss [1]	結果ベクトルの容量
		-nev [1]	必要な固有対
		-kncv [0]	クリロフ部分空間の大きさ

前処理に関するオプション (既定値: `-p none`)

前処理	オプション	補助オプション	
なし	`-p {none|0}`		
Jacobi	`-p {jacobi|1}`		
ILU(k)	`-p {ilu|2}`	`-ilu_fill [0]`	フィルインレベル k
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
SSOR	`-p {ssor|3}`	`-ssor_omega [1.0]`	緩和係数 ω $(0 < \omega < 2)$
Hybrid	`-p {hybrid|4}`	`-hybrid_i [sor]`	線型方程式解法
		`-hybrid_maxiter [25]`	最大反復回数
		`-hybrid_tol [1.0e-3]`	収束判定基準
		`-hybrid_omega [1.5]`	SOR の緩和係数 ω $(0 < \omega < 2)$
		`-hybrid_ell [2]`	BiCGSTAB(l) の次数 l
		`-hybrid_restart [40]`	GMRES(m), Orthomin(m) の リスタート値 m
I+S	`-p {is|5}`	`-is_alpha [1.0]`	$I + \alpha S^{(m)}$ のパラメータ α
		`-is_m [3]`	$I + \alpha S^{(m)}$ のパラメータ m
SAINV	`-p {sainv|6}`	`-sainv_drop [0.05]`	ドロップ基準
SA-AMG	`-p {saamg|7}`	`-saamg_unsym [false]`	非対称版の選択 (行列構造は対称とする)
		`-saamg_theta [0.05|0.12]`	ドロップ基準 $a_{ij}^2 \leq \theta^2 \lvert a_{ii}\rvert \lvert a_{jj}\rvert$ (対称|非対称)
crout ILU	`-p {iluc|8}`	`-iluc_drop [0.05]`	ドロップ基準
		`-iluc_rate [5.0]`	最大フィルイン数の倍率
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
ILUT	`-p {ilut|9}`	`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
MFAMG	`-p {mfamg|12}`		
		`-mfamg_block_size [1]`	ローカル連続平滑化ブロックサイズ (1: 重み付きヤコビ 2-: ブロックヤコビ)
Additive Schwarz	`-adds true`	`-adds_iter [1]`	反復回数

その他のオプション

オプション	
-emaxiter [1000]	最大反復回数
-etol [1.0e-12]	収束判定基準
-eprint [0]	残差履歴の出力
	-eprint {none|0}　残差履歴を出力しない
	-eprint {mem|1}　残差履歴をメモリに保存する
	-eprint {out|2}　残差履歴を標準出力に書き出す
	-eprint {all|3}　残差履歴をメモリに保存し, 標準出力に書き出す
-ie [ii]	Subspace, Lanczos, Arnoldi の内部で使用する固有値解法の指定
-ige [gii]	Generalized Subspace, Generalized Lanczos, Generalized Arnoldi の内部で使用する固有値解法の指定
-shift [0.0]	$A-\sigma B$ を計算するためのシフト量 σ の実部
-shift_im [0.0]	シフト量 σ の虚部
-initx_ones [1]	初期ベクトル x_0
	-initx_ones {false|0}　関数 lis_esolve() の引数 x により与えられる要素の値を使用
	-initx_ones {true|1}　すべての要素の値を 1 にする
-omp_num_threads [t]	実行スレッド数
	t は最大スレッド数
-estorage [0]	行列格納形式
-estorage_block [2]	BSR, BSC 形式のブロックサイズ
-ef [0]	固有値解法の精度
	-ef {double|0}　倍精度
	-ef {quad|1}　double-double 型 4 倍精度
-rval [0]	Ritz 値
	-rval {false|0}　Ritz 値をもとに固有対を計算
	-rval {true|1}　Ritz 値のみを計算
-nev [1]	必要な固有対の数 ($\leq$ ss)
-etransform [0]	一般化固有値変換の選択
	-etransform {auto|0}　既定の変換を選択
	-etransform {shift-invert|1}　シフト反転 $T=(A-\sigma B)^{-1}B$
	-etransform {buckling|2}　座屈 $T=(A-\sigma B)^{-1}A$

求解

標準固有値問題 $Ax=\lambda x$ を解くには, 関数

- C
```
LIS_INT lis_esolve(LIS_MATRIX A, LIS_VECTOR x,
 LIS_SCALAR evalue, LIS_ESOLVER esolver)
```
- Fortran
```
subroutine lis_esolve(LIS_MATRIX A, LIS_VECTOR x,
 LIS_SCALAR evalue, LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

を用いる.

一般化固有値問題 $Ax = \lambda Bx$ を解くには, 関数

- C　　`LIS_INT lis_gesolve(LIS_MATRIX A, LIS_MATRIX B, LIS_VECTOR x, LIS_SCALAR evalue, LIS_ESOLVER esolver)`
- Fortran subroutine `lis_gesolve(LIS_MATRIX A, LIS_MATRIX B, LIS_VECTOR x, LIS_SCALAR evalue, LIS_ESOLVER esolver, LIS_INTEGER ierr)`

を用いる.

3.6 プログラムの作成

線型方程式 $Ax = b$ を指定された解法で解き, その解を標準出力に書き出すプログラムを以下に示す.
行列 A は次数 12 の 3 重対角行列

$$A = \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}$$

である. 右辺ベクトル b は解 x がすべて 1 となるよう設定される.
プログラムはディレクトリ `lis-($VERSION)/test` に含まれる.

検証プログラム: test4.c

```
 1: #include <stdio.h>
 2: #include "lis.h"
 3: main(LIS_INT argc, char *argv[])
 4: {
 5:     LIS_INT i,n,gn,is,ie,iter;
 6:     LIS_MATRIX A;
 7:     LIS_VECTOR b,x,u;
 8:     LIS_SOLVER solver;
 9:     n = 12;
10:     lis_initialize(&argc,&argv);
11:     lis_matrix_create(LIS_COMM_WORLD,&A);
12:     lis_matrix_set_size(A,0,n);
13:     lis_matrix_get_size(A,&n,&gn);
14:     lis_matrix_get_range(A,&is,&ie);
15:     for(i=is;i<ie;i++)
16:     {
17:         if( i>0 ) lis_matrix_set_value(LIS_INS_VALUE,i,i-1,-1.0,A);
18:         if( i<gn-1 ) lis_matrix_set_value(LIS_INS_VALUE,i,i+1,-1.0,A);
19:         lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0,A);
20:     }
21:     lis_matrix_set_type(A,LIS_MATRIX_CSR);
22:     lis_matrix_assemble(A);
23:
24:     lis_vector_duplicate(A,&u);
25:     lis_vector_duplicate(A,&b);
26:     lis_vector_duplicate(A,&x);
27:     lis_vector_set_all(1.0,u);
28:     lis_matvec(A,u,b);
29:
30:     lis_solver_create(&solver);
31:     lis_solver_set_optionC(solver);
32:     lis_solve(A,b,x,solver);
33:     lis_solver_get_iter(solver,&iter);
34:     printf("number of iterations = %d\n",iter);
35:     lis_vector_print(x);
36:     lis_matrix_destroy(A);
37:     lis_vector_destroy(u);
38:     lis_vector_destroy(b);
39:     lis_vector_destroy(x);
40:     lis_solver_destroy(solver);
41:     lis_finalize();
42:     return 0;
43: }
}
```

検証プログラム: test4f.F

```
 1:      implicit none
 2:
 3:#include "lisf.h"
 4:
 5:      LIS_INTEGER i,n,gn,is,ie,iter,ierr
 6:      LIS_MATRIX A
 7:      LIS_VECTOR b,x,u
 8:      LIS_SOLVER solver
 9:      n  = 12
10:      call lis_initialize(ierr)
11:      call lis_matrix_create(LIS_COMM_WORLD,A,ierr)
12:      call lis_matrix_set_size(A,0,n,ierr)
13:      call lis_matrix_get_size(A,n,gn,ierr)
14:      call lis_matrix_get_range(A,is,ie,ierr)
15:      do i=is,ie-1
16:        if( i>1  ) call lis_matrix_set_value(LIS_INS_VALUE,i,i-1,-1.0d0,
17:     .                                       A,ierr)
18:        if( i<gn ) call lis_matrix_set_value(LIS_INS_VALUE,i,i+1,-1.0d0,
19:     .                                       A,ierr)
20:        call lis_matrix_set_value(LIS_INS_VALUE,i,i,2.0d0,A,ierr)
21:      enddo
22:      call lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
23:      call lis_matrix_assemble(A,ierr)
24:
25:      call lis_vector_duplicate(A,u,ierr)
26:      call lis_vector_duplicate(A,b,ierr)
27:      call lis_vector_duplicate(A,x,ierr)
28:      call lis_vector_set_all(1.0d0,u,ierr)
29:      call lis_matvec(A,u,b,ierr)
30:
31:      call lis_solver_create(solver,ierr)
32:      call lis_solver_set_optionC(solver,ierr)
33:      call lis_solve(A,b,x,solver,ierr)
34:      call lis_solver_get_iter(solver,iter,ierr)
35:      write(*,*) 'number of iterations = ',iter
36:      call lis_vector_print(x,ierr)
37:      call lis_matrix_destroy(A,ierr)
38:      call lis_vector_destroy(b,ierr)
39:      call lis_vector_destroy(x,ierr)
40:      call lis_vector_destroy(u,ierr)
41:      call lis_solver_destroy(solver,ierr)
42:      call lis_finalize(ierr)
43:
44:      stop
45:      end
```

3.7 前処理とソルバの分離

前節では, 前処理行列は lis_solve が呼び出されるたびに更新されていた. 本節では, 前処理とソルバを分離し, 前処理行列をソルバと切り離して更新する方法について述べる. この方法は, 非線型偏微分方程式を含むある種の問題に対して極めて有効である. 実際, Newton-Raphson 法では反復法が用いられ, 各反復での線型方程式の解をもとに解ベクトルが逐次的に求められる. この機能を実現するためには, 以下の関数を実装する必要がある.

- lis_matrix_psd_set_value: 行列要素の値を再定義する.
- lis_matrix_psd_reset_scale: 行列のスケーリングに関する情報を更新する. この関数, 及び次の関数は, 自明でないスケーリングを行う場合にのみ用いる.
- lis_vector_psd_reset_scale: ベクトルのスケーリングに関する情報を更新する.
- lis_solver_set_matrix: 与えられた行列とソルバを関連付ける. この関数は lis_precon_psd_create より前に呼び出されなければならない.
- lis_precon_psd_create: 選択された前処理のためのデータ構造を生成する.
- lis_precon_psd_update: 前処理を評価する.

この方法には現在以下の制限がある.

- 利用可能な行列格納形式は CSR のみである.
- 利用可能なソルバは GMRES のみである.
- 利用可能な前処理は ILU(k) 及び SA-AMG のみである.

実装例を疑似コードにより示す. 詳細は test8f.F90 を参照のこと.

疑似コード

```
 1:      PROGRAM psd_driver
 2:
 3:      implicit none
 4:
 5:#include "lisf.h"
 6:
 7:      LIS_INTEGER i,n,gn,is,ie,iter,ierr
 8:      LIS_MATRIX A
 9:      LIS_VECTOR b,x
10:      LIS_SOLVER solver
11:      REAL :: u(:),du
12:
13:      CALL lis_initialize(ierr)
14:
15:     !=================================================================
16:     ! initialization, only done once
17:     !=================================================================
18:      CALL lis_matrix_create(LIS_COMM_WORLD,A,ierr)
19:      CALL lis_matrix_set_size(A,0,n,ierr)
20:      CALL lis_matrix_get_size(A,n,gn,ierr)
21:      CALL lis_matrix_get_range(A,is,ie,ierr)
22:
23:      CALL UpdateLinearSystem(RHS,LHS)
24:      DO i=is,ie-1
25:         DO j=1,gn
26:             IF (LHS(i,j) exists) THEN
27:                   CALL lis_matrix_set_value(LIS_INS_VALUE,i,j,LHS(i,j),A,ierr)
28:             END IF
29:         END DO
30:      END DO
31:      CALL lis_matrix_set_type(A,LIS_MATRIX_CSR,ierr)
32:      CALL lis_matrix_assemble(A,ierr)
33:
34:      CALL lis_vector_duplicate(A,b,ierr)
35:      CALL lis_vector_duplicate(A,x,ierr)
36:      DO i=is,ie-1
37:           CALL lis_vector_set_value(LIS_INS_VALUE,i,RHS(i),b,ierr)
38:      END DO
39:      u = u_initial
40:
41:      CALL lis_solver_create(solver,ierr)
42:      WRITE(UNIT=options,FMT='(a)') "-p ilu -i gmres -print out -scale none"
43:      CALL lis_solver_set_option(TRIM(options),solver,ierr)
44:
45:     !=================================================================
46:     ! everything up to this point is more or less identical to the standard workflow.
47:     ! Now comes the preconditioner initialization, and the Newton-Raphson
48:     ! iteration.
49:     !=================================================================
50:     CALL lis_solver_set_matrix(A,solver,ierr)
51:     CALL lis_precon_psd_create(solver,precon,ierr)
52:     ! evaluate the preconditioner, at least once . . .
53:     CALL lis_precon_psd_update(solver,precon,ierr)
54:
```

疑似コード（続き）

```
 55:     DO
 56:
 57:         IF (UpdateLHS) THEN
 58:             DO i=is,ie-1
 59:                 DO j=1,gn
 60:                     IF (component (i,j) exists) THEN
 61:                         CALL lis_matrix_psd_set_value(LIS_INS_VALUE,i,j,LHS(i,j),A,ierr)
 62:                     END IF
 63:                 END DO
 64:              END DO
 65:              CALL lis_matrix_psd_reset_scale(A,ierr)
 66:         END IF
 67:
 68:         ! update RHS every iteration
 69:         DO i=is,ie-1
 70:             CALL lis_vector_set_value(LIS_INS_VALUE,i,RHS(i),b,ierr)
 71:         END DO
 72:         CALL lis_vector_psd_reset_scale(A,ierr)
 73:
 74:         IF (UpdateLHS) THEN
 75:             CALL lis_precon_psd_update(solver,precon,ierr)
 76:         END IF
 77:         CALL lis_solve_kernel(A,b,x,solver,precon,ierr)
 78:         CALL lis_solver_get_iter(solver,iter,ierr)
 79:         write(*,*) 'number of iterations = ',iter
 80:         CALL lis_vector_print(x,ierr)
 81:
 82:         ! update the solution
 83:         DO i=is,ie-1
 84:             CALL lis_vector_get_value(x,i,du,ierr)
 85:             u(i)=u(i)-du
 86:         END DO
 87:
 88:         CALL UpdateLinearSystem(RHS,LHS)
 89:
 90:         IF (termination criteria satisfied) EXIT
 91:
 92:     END DO
 93:
 94:
 95:      CALL lis_matrix_destroy(A,ierr)
 96:      CALL lis_vector_destroy(b,ierr)
 97:      CALL lis_vector_destroy(x,ierr)
 98:      CALL lis_vector_destroy(u,ierr)
 99:      CALL lis_solver_destroy(solver,ierr)
100:
101:      CALL lis_finalize(ierr)
102:
103:      END PROGRAM psd_driver
```

3.8 マトリックスフリー計算

バージョン 2.1.13 より、Lis はマトリックスフリー計算をサポートするため、新たに LIS_MATRIX_USER および LIS_MATRIX_OPERATOR を導入した。

3.8.1 ユーザ定義シェル行列 (LIS_MATRIX_USER)

このインタフェースは、アプリケーション独自の行列データ構造を保持したまま、行列ベクトル積のコールバック関数を登録することを可能にする。ユーザ定義行列の対角成分を抽出するために、オプションの対角成分コールバック関数を登録する以下の関数が提供されている:LIS_INT lis_matrix_set_user_diagonal(LIS_MATRIX A, LIS_INT (*callback)(LIS_MATRIX A, LIS_VECTOR d))

- A: LIS_MATRIX_USER として定義された対象の行列
- callback: 対角成分を計算し、ベクトル d に格納するユーザ定義の関数ポインタ

3.8.2 線型結合行列演算子 (LIS_MATRIX_OPERATOR)

この演算子は、元の行列を変更することなく、2 つの行列の線型結合 $C = \alpha A + \beta B$ を表す。演算子行列に対して、対角成分は以下のように自動的かつ再帰的に評価される：

$$\mathrm{diag}(\alpha A + \beta B) = \alpha \cdot \mathrm{diag}(A) + \beta \cdot \mathrm{diag}(B)$$

3.8.3 行列演算子を用いたシフト付き一般化固有値ソルバ

シフト付き一般化固有値問題 $(A - \sigma B)x = \lambda Bx$ に対して、一般化べき乗法（GPI)、一般化逆べき乗法（GII)、および一般化共役残差法（GCR）ソルバは、LIS_MATRIX_OPERATOR を利用して $A - \sigma B$ に対する一時的な演算子を動的に生成する。これは元の行列が元の状態のまま残ることを保証し、ユーザ定義のカスタム行列（LIS_MATRIX_USER）に対する完全なサポートを可能にする。シフト付き LIS_MATRIX_USER は、Lanczos 法のような他の手法に対しては依然として非対応のままである。

3.8.4 ネイティブリスタート付きクリロフソルバ

ネイティブ対称リスタート付きクリロフソルバは、-e krylov（数値識別子 18）で選択される。本ソルバは、実対称標準固有値問題 $Ax = \lambda x$ および一般化固有値問題 $Ax = \lambda Bx$ の複数の固有対を、レイリー・リッツ射影、シック・リスタート、計量再直交化、および明示的な残差検証を用いて計算する。これは追加のソルバであり、既存の部分空間反復法、ランチョス法、アーノルディ法、FEAST 法を置き換えるものではない。その特徴的な機能は、独立したコールバック API、オプションで利用可能な半正定値計量およびハードロッキング、そして慣性に基づく区間ヘルパである。他ソルバに対する性能上の優位性は主張されていない。

標準 Lis 固有値解法インタフェース

本手法は、LIS_MATRIX_USER を含むローカル行列を対象として、lis_esolve および lis_gesolve を通じて利用可能である。以下の C 言語の例では、アプリケーション側で A、B、および初期ベクトル x が準備されているものとする。

```
LIS_ESOLVER esolver;
LIS_SCALAR lambda0;
LIS_INT status;
lis_esolver_create(&esolver);
lis_esolver_set_option(
"-e krylov -ss 4 -nev 4 -kncv 20 "
"-kwhich LM -etol 1e-9 -emaxiter 500",
esolver);
status = lis_gesolve(A, B, x, &lambda0, esolver);
/* Check status and residuals; then destroy esolver. */
```

標準問題の場合は、代わりに `lis_esolve(A,x,&lambda0,esolver)` を使用する。最も関連性の高いオプションは以下の通りである。

- `-ss n` は結果ベクトルの容量を指定し、`-nev k` は要求する固有対の数を指定する（$1 \leq k \leq n_{\mathrm{ss}}$、$k < n$）。
- `-kncv n` は内部クリロフ部分空間のサイズを選択する。これは k を超える必要があり、ゼロを指定した場合は $\min(n, 3k+8)$ が選択される。
- `-kwhich LM`、`SM`、`LA`、または `SA` は、変換されたリッツ値の絶対値の最大／最小、または代数的な順序を選択する。
- `-etol t` と `-emaxiter m` は、残差の許容誤差と反復/再開始の制限回数を設定する。
- `-kresidual_scale a` は、残差スケーリングのための $\|A\|$ の見積もりを提供する。ゼロを指定した場合は、自動的にサンプリングされた見積もりが使用される。
- `-kinner_tol t` が正の値の場合、追加のコストを伴うが、シフト付き線形方程式の解の明示的な検証を有効にする。
- `-ksemidefinite 1` は、一般化シフト反転モードにおいて半正定値 B 計量を有効にする。これはオプトインである。
- `-khardlock 1` は、収束した固有対の永続的なロッキングと計量デフレーションを有効にし、アクティブな部分空間の消去と再初期化を行う。デフォルトはオフである。
- `-shift sigma` と `-etransform shift-invert` はシフトを設定する。座屈変換には、ゼロでないシフトと正定値 A が必要である。

既存の Lis 線形ソルバおよび前処理のオプションは、シフト付き線形計算を設定するために用いられる。前処理のセットアップはクリロフ反復間で再利用される。

変換及び計量再直交化

サポートされている演算、およびリッツ値 μ から物理的な固有値 λ への写像は以下の通りである。

- 標準: $\mathcal{T} = A$。$\lambda = \mu$ であり、ユークリッド正規化を行う。
- 一般化シフト: $\mathcal{T} = (A - \sigma B)^{-1} B$。$\lambda = \sigma + 1/\mu$ であり、正定値 B 計量（または有効化されている場合は半正定値 B）を用いる。

- 一般化座屈: $\mathcal{T} = (A - \sigma B)^{-1} A$。$\lambda = \sigma\mu/(\mu - 1)$ であり、正定値 A 計量を用いる。ここで B は不定値であってもよい。

シフトされた演算子は非特異でなければならず、また逆行列の適用は十分に正確かつほぼ対称でなければならない。逆変換の極（分母がゼロになる点）に近すぎるリッツ値は、有効な物理的固有値として表現できない。半正定値モードでは、開始方向と補充方向は $(A - \sigma B)^{-1} B$ を通して写像される。計量がゼロとなる方向は拒絶される。このモードは B ノルムがゼロでない有限の固有対を探索する。ランク落ちの計量では、部分的な収束に留まる可能性がある。任意の特異行列ペンシルはサポートされていない。

3.8.5 不完全 LU 分解前処理の半精度保持

Lis は、オプションとして ILU(k) および ILUT 前処理の L および U 要素の値を IEEE 754 binary16（半精度浮動小数点）形式で保持することができる。この機能により、数値要素に必要となる記憶容量を削減できる。ただし、行列インデックス、対角成分の逆数、ベクトル、算術演算、残差、および収束判定は、通常の Lis スカラ精度（単精度または倍精度）のまま維持される。

この機能は、ビルド時に `--enable-half` を指定することで有効になる。実行時のオプションは以下の通りである。

```
-precon_half {0|1|2}
```

ここで、各値の意味は以下の通りである。

- `0` は、前処理の半精度保持を無効にする。
- `1` は、品質検証を伴う binary16 保持を有効にする。圧縮された要素が品質基準を満たさない場合、通常の数値要素へ自動的に切り替えられる。
- `2` は、品質検証を伴わない binary16 保持を有効にする。ただし、厳格な妥当性チェックおよび有限値チェックは依然として適用され、半精度保持を安全に使用できない場合は通常の要素が維持される。

半精度保持モードは、ポリシ設定（環境に応じた切り替え）として動作する。選択されたビルドまたはスカラ構成において半精度保持が利用できない場合、Lis は通常の前処理要素を維持し、エラーを出さずに処理を継続する。

ILU(k) では、PSD による数値前処理の更新後に、半精度の数値要素が再計算される。ILUT でも、前処理が通常通り生成された場合に、同じ binary16 保持および適用カーネルが使用される。

品質検証のデフォルトの相対許容誤差は 5×10^{-2} である。これは環境変数 `LIS_HALF_PROBE_TOL` を設定することで上書きできる。

サポートされている x86 システムでは、Lis は実行時に binary16 要素の値をデコードするために F16C 命令セット（SIMD）を使用する場合がある。参照用および切り替え用の実装として、スカラデコーダも引き続き利用可能である。環境変数 `LIS_HALF_NO_SIMD=1` を設定すると、強制的にスカラデコーダが使用される。

3.8.6 ユーザ定義行列における前処理の再利用と互換性

バージョン 2.1.15 より、Lis は前処理オブジェクトの所有権の明確化と、ユーザ定義行列（`LIS_MATRIX_USER`）を用いた繰り返し計算のサポートを強化した。これにより、互換性のある構築済み前処理が与えられている場合、従来の制限が緩和され正常に計算を実行できるようになった。

3.8.7 ソルバ管理機能の拡張

バージョン 2.1.16 より、元の行列構造や右辺ベクトルを直接変更することなく、シェル行列の対称対角スケーリングを行うことができるようになった。さらに、入れ子状の反復における収束の堅牢性を向上させるため、FGMRES ソルバに対してソルバ管理のニア零空間サポートと 2 レベルデフレーション補正が組み込まれた。

3.8.8 MFAMG 前処理

MFAMG は、マトリックスフリーアグリゲーションマルチグリッド前処理である。ユーザから提供される 1 つまたは複数の近似零空間ベクトルから、マルチレベルのアグリゲーション階層を構築する。MFAMG は、逐次環境では CSR 行列および LIS_MATRIX_USER 行列をサポートする。MPI 環境においては、現時点では CSR 行列が必須である。また、倍精度、-scale none の指定が必要である。オプション -mfamg_block_size は、局所平滑化のブロックサイズを制御する。デフォルト値の 1 は、従来の重み付きヤコビ平滑化を維持する。2 から 16 までの値は、連続するローカルな未知数を用いたブロックヤコビ法を有効にする。ブロックがプロセス境界をまたぐことはなく、最後のローカルブロックに含まれる未知数の数は、要求されたブロックサイズよりも少なくなることがある。利用可能な各対角ブロックに対しては、ピボッティングを伴う行列分解が行われる。ブロックが数値的に特異である場合、そのスカラ対角成分が利用可能であれば、MFAMG はそのブロックに対して重み付きヤコビ補正へと切り替える。既存の MFAMG 数値更新パスが階層構造を再利用する場合、分解されたブロックの行列は更新された行列の値に基づいて再計算される。

3.8.9 ネイティブ FEAST 固有値ソルバ

ネイティブ FEAST 固有値ソルバは、-feast_emin および -feast_emax で指定された実数区間の内部にある固有対を計算する。区間の両端の境界値はどちらも必須であり、-feast_emin < -feast_emax を満たす必要がある。部分空間の次元は -ss で選択する。周回積分には、-feast_points で指定された数値積分点と、-feast_ratio で指定された正の楕円アスペクト比が使用される。シフト付き線型方程式の解法には、-feast_solver オプションとして gmres、fgmres、gcrodr、または fgcrodr を指定できる。デフォルトでは GMRES が使用される。ネイティブ FEAST は標準および一般化固有値問題の両方で利用可能であり、実数スカラビルドと複素数スカラビルドの双方をサポートしている。

3.9 実行ファイルの生成

test4.c から実行ファイルを生成する方法について述べる．ディレクトリ lis-($VERSION)/test にある検証プログラム test4.c を SGI Altix 3700 上の Intel C Compiler (icc), Intel Fortran Compiler (ifort) でコンパイルする場合の例を以下に示す．SA-AMG 前処理には Fortran 90 で記述されたコードが含まれるため, SA-AMG 前処理を使用する場合には Fortran 90 コンパイラでリンクしなければならない．また, マルチプロセス環境ではプリプロセッサマクロ USE_MPI が定義されなければならない．64bit 整数型を使用する場合は, C プログラムではプリプロセッサマクロ_LONG__LONG, Fortran プログラムではプリプロセッサマクロ LONG__LONG が定義されなければならない．

逐次環境

コンパイル
```
> icc -c -I($INSTALLDIR)/include test4.c
```
リンク
```
> icc -o test4 test4.o -L($INSTALLDIR)/lib -llis
```
リンク (--enable-saamg)
```
> ifort -nofor_main -o test4 test4.o -L($INSTALLDIR)/lib -llis
```

マルチスレッド環境

コンパイル
```
> icc -c -openmp -I($INSTALLDIR)/include test4.c
```
リンク
```
> icc -openmp -o test4 test4.o -L($INSTALLDIR)/lib -llis
```
リンク (--enable-saamg)
```
> ifort -nofor_main -openmp -o test4 test4.o -L($INSTALLDIR)/lib -llis
```

マルチプロセス環境

コンパイル
```
> icc -c -DUSE_MPI -I($INSTALLDIR)/include test4.c
```
リンク
```
> icc -o test4 test4.o -L($INSTALLDIR)/lib -llis -lmpi
```
リンク (--enable-saamg)
```
> ifort -nofor_main -o test4 test4.o -L($INSTALLDIR)/lib -llis -lmpi
```

マルチスレッド・マルチプロセス環境

コンパイル
```
> icc -c -openmp -DUSE_MPI -I($INSTALLDIR)/include test4.c
```
リンク
```
> icc -openmp -o test4 test4.o -L($INSTALLDIR)/lib -llis -lmpi
```
リンク (--enable-saamg)
```
> ifort -nofor_main -openmp -o test4 test4.o -L($INSTALLDIR)/lib -llis -lmpi
```

次に, test4f.F から実行ファイルを生成する方法について述べる．ディレクトリ lis-($VERSION)/test にある検証プログラム test4f.F を SGI Altix 3700 上の Intel Fortran Compiler (ifort) でコンパイルする場合の例を以下に示す．Fortran のユーザプログラムにはコンパイラ指示文が含まれるため, プリプロセッサを使用するようコンパイラに指示しなければならない．ifort の場合は, オプション -fpp が必要である．

逐次環境

コンパイル

```
> ifort -c -fpp -I($INSTALLDIR)/include test4f.F
```

リンク

```
> ifort -o test4f test4f.o -L($INSTALLDIR)/lib -llis
```

マルチスレッド環境

コンパイル

```
> ifort -c -fpp -openmp -I($INSTALLDIR)/include test4f.F
```

リンク

```
> ifort -openmp -o test4f test4f.o -L($INSTALLDIR)/lib -llis
```

マルチプロセス環境

コンパイル

```
> ifort -c -fpp -DUSE_MPI -I($INSTALLDIR)/include test4f.F
```

リンク

```
> ifort -o test4f test4f.o -L($INSTALLDIR)/lib -llis -lmpi
```

マルチスレッド・マルチプロセス環境

コンパイル

```
> ifort -c -fpp -openmp -DUSE_MPI -I($INSTALLDIR)/include test4f.F
```

リンク

```
> ifort -openmp -o test4f test4f.o -L($INSTALLDIR)/lib -llis -lmpi
```

3.10 実行

ディレクトリ lis-($VERSION)/test にある検証プログラム test4 または test4f を SGI Altix 3700 上のそれぞれの環境で

逐次環境

```
> ./test4 -i bicgstab
```

マルチスレッド環境

```
> env OMP_NUM_THREADS=2 ./test4 -i bicgstab
```

マルチプロセス環境

```
> mpirun -np 2 ./test4 -i bicgstab
```

マルチスレッド・マルチプロセス環境

```
> mpirun -np 2 env OMP_NUM_THREADS=2 ./test4 -i bicgstab
```

と入力して実行すると, 以下のように解が標準出力に書き出される.

```
initial vector x      : all components set to 0
precision             : double
linear solver         : BiCGSTAB
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
```

```
matrix storage format : CSR
linear solver status  : normal end

     0  1.000000e-00
     1  1.000000e+00
     2  1.000000e-00
     3  1.000000e+00
     4  1.000000e-00
     5  1.000000e+00
     6  1.000000e+00
     7  1.000000e-00
     8  1.000000e+00
     9  1.000000e-00
    10  1.000000e+00
    11  1.000000e-00
```

3.11 プロセス上での自由度について

MPI プロセスを複数使用する場合には, $global_n = 0$, $local_n \geq 0$ であってもよい. すなわち, 一つ以上のプロセスにおいて自由度を 0 とすることができる. ただし, その場合においても $global_n = 0$ ならば $local_n$ の総和は 0 より大きくなければならない.

4　4倍精度演算

反復法の計算では, 丸め誤差の影響によって収束が停滞することがある. 本ライブラリでは, long double 型及び倍精度浮動小数点数を 2 個用いた”double-double”[55, 56] 型の 4 倍精度演算を用いることにより, 収束を改善することが可能である. double-double 型演算では, 浮動小数 a を $a = a.hi + a.lo$, $\frac{1}{2}\mathrm{ulp}(a.hi) \geq |a.lo|$ (上位 $a.hi$ と下位 $a.lo$ は倍精度浮動小数) により定義し, Dekker[57] と Knuth[58] のアルゴリズムに基づいて倍精度の四則演算の組み合わせにより 4 倍精度演算を実現する. double-double 型の演算は一般に Fortran の 4 倍精度演算より高速である [59] が, Fortran の表現形式 [60] では仮数部が 112 ビットであるのに対して, 倍精度浮動小数を 2 個使用するため, 仮数部が 104 ビットとなり, 8 ビット少ない. また, 指数部は倍精度浮動小数と同じ 11 ビットである.

double-double 型演算では, 入力として与えられる行列, ベクトル, 及び出力の解は倍精度である. ユーザプログラムは 4 倍精度変数を直接扱うことはなく, オプションとして 4 倍精度演算を使用するかどうかを指定するだけでよい. なお, Intel 系のアーキテクチャに対しては Streaming SIMD Extensions (SSE) 命令を用いて高速化を行う [61].

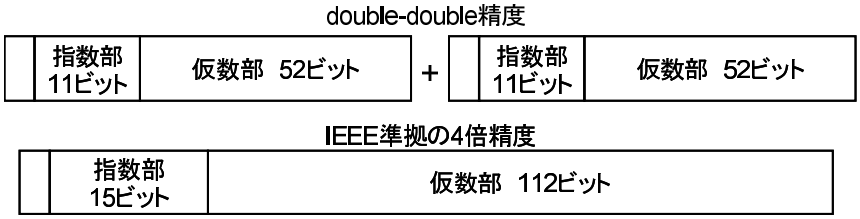


図 2: double-double 型のビット数

4.1　4倍精度演算の使用

Toepliz 行列

$$A = \begin{pmatrix} 2 & 1 & & & & \\ 0 & 2 & 1 & & & \\ \gamma & 0 & 2 & 1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & \gamma & 0 & 2 & 1 \\ & & & \gamma & 0 & 2 \end{pmatrix}$$

に対する線型方程式 $Ax = b$ を指定された解法で解き, 解を標準出力に書き出す検証プログラムが `test5.c` である. 右辺ベクトル b は解 x がすべて 1 となるよう設定される. n は行列 A の次数である. `test5` において,

倍精度の場合

```
> ./test5 200 2.0 -f double
```

または

```
> ./test5 200 2.0
```

と入力して実行すると, 以下の結果が得られる.

```
n = 200, gamma = 2.000000

initial vector x      : all components set to 0
precision             : double
linear solver         : BiCG
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
matrix storage format : CSR
linear solver status  : normal end

BiCG: number of iterations = 1001 (double = 1001, quad = 0)
BiCG: elapsed time         = 2.044368e-02 sec.
BiCG:   preconditioner     = 4.768372e-06 sec.
BiCG:     matrix creation  = 4.768372e-06 sec.
BiCG:   linear solver      = 2.043891e-02 sec.
BiCG: relative residual    = 8.917591e+01
```

4 倍精度の場合

```
> ./test5 200 2.0 -f quad
```

と入力して実行すると, 以下の結果が得られる.

```
n = 200, gamma = 2.000000

initial vector x      : all components set to 0
precision             : quad
linear solver         : BiCG
preconditioner        : none
convergence condition : ||b-Ax||_2 <= 1.0e-12 * ||b-Ax_0||_2
matrix storage format : CSR
linear solver status  : normal end

BiCG: number of iterations = 230 (double = 230, quad = 0)
BiCG: elapsed time         = 2.267408e-02 sec.
BiCG:   preconditioner     = 4.549026e-04 sec.
BiCG:     matrix creation  = 5.006790e-06 sec.
BiCG:   linear solver      = 2.221918e-02 sec.
BiCG: relative residual    = 6.499145e-11
```

5 行列格納形式

本節では, ライブラリで使用できる行列の格納形式について述べる. 行列の行 (列) 番号は 0 から始まるものとする. 次数 $n \times n$ の行列 $A = (a_{ij})$ の非零要素数を nnz とする.

5.1 Compressed Sparse Row (CSR)

CSR 形式では, データを 3 つの配列 (ptr,index,value) に格納する.

- 長さ nnz の倍精度配列 value は, 行列 A の非零要素の値を行方向に沿って格納する.
- 長さ nnz の整数配列 index は, 配列 value に格納された非零要素の列番号を格納する.
- 長さ $n+1$ の整数配列 ptr は, 配列 value と index の各行の開始位置を格納する.

5.1.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の CSR 形式での格納方法を図 3 に示す. この行列を CSR 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

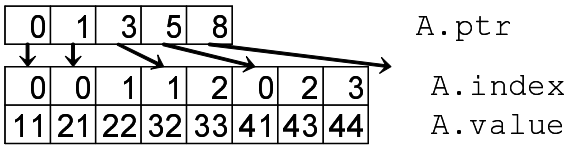


図 3: CSR 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,nnz;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8;
 6: ptr   = (LIS_INT *)malloc( (n+1)*sizeof(int) );
 7: index = (LIS_INT *)malloc( nnz*sizeof(int) );
 8: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
 9: lis_matrix_create(0,&A);
10: lis_matrix_set_size(A,0,n);
11:
12: ptr[0] = 0; ptr[1] = 1; ptr[2] = 3; ptr[3] = 5; ptr[4] = 8;
13: index[0] =  0; index[1] =  0; index[2] =  1; index[3] =  1;
14: index[4] =  2; index[5] =  0; index[6] =  2; index[7] =  3;
15: value[0] = 11; value[1] = 21; value[2] = 22; value[3] = 32;
16: value[4] = 33; value[5] = 41; value[6] = 43; value[7] = 44;
17:
18: lis_matrix_set_csr(nnz,ptr,index,value,A);
19: lis_matrix_assemble(A);
```

5.1.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の CSR 形式での格納方法を図 4 に示す. 2 プロセス上にこの行列を CSR 形式で作成する場合, プログラムは以下のように記述する.

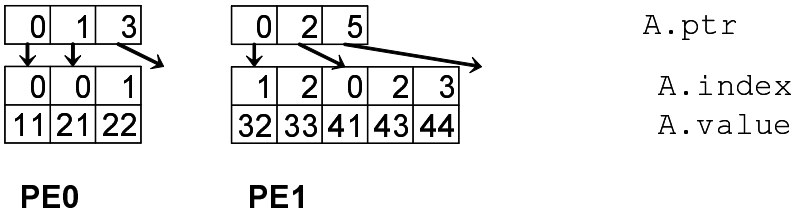


図 4: CSR 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,k,n,nnz,my_rank;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 3;}
 7: else             {n = 2; nnz = 5;}
 8: ptr   = (LIS_INT *)malloc( (n+1)*sizeof(int) );
 9: index = (LIS_INT *)malloc( nnz*sizeof(int) );
10: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
11: lis_matrix_create(MPI_COMM_WORLD,&A);
12: lis_matrix_set_size(A,n,0);
13: if( my_rank==0 ) {
14:     ptr[0] = 0; ptr[1] = 1; ptr[2] = 3;
15:     index[0] =  0; index[1] =  0; index[2] =  1;
16:     value[0] = 11; value[1] = 21; value[2] = 22;}
17: else {
18:     ptr[0] = 0; ptr[1] = 2; ptr[2] = 5;
19:     index[0] =  1; index[1] =  2; index[2] =  0; index[3] =  2; index[4] =  3;
20:     value[0] = 32; value[1] = 33; value[2] = 41; value[3] = 43; value[4] = 44;}
21: lis_matrix_set_csr(nnz,ptr,index,value,A);
22: lis_matrix_assemble(A);
```

5.1.3 関連する関数

配列の関連付け

CSR 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_csr(LIS_INT nnz, LIS_INT ptr[], LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_csr(LIS_INTEGER nnz, LIS_INTEGER ptr(), LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.2 Compressed Sparse Column (CSC)

CSC 形式では, データを 3 つの配列 (ptr,index,value) に格納する.

- 長さ nnz の倍精度配列 value は, 行列 A の非零要素の値を列方向に沿って格納する.
- 長さ nnz の整数配列 index は, 配列 value に格納された非零要素の行番号を格納する.
- 長さ $n+1$ の整数配列 ptr は, 配列 value と index の各列の開始位置を格納する.

5.2.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の CSC 形式での格納方法を図 5 に示す. この行列を CSC 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

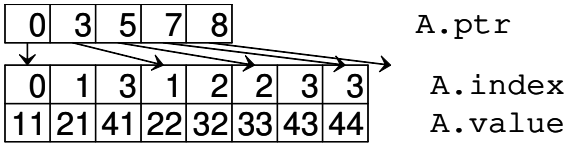


図 5: CSC 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,nnz;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8;
 6: ptr   = (LIS_INT *)malloc( (n+1)*sizeof(int) );
 7: index = (LIS_INT *)malloc( nnz*sizeof(int) );
 8: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
 9: lis_matrix_create(0,&A);
10: lis_matrix_set_size(A,0,n);
11:
12: ptr[0] = 0; ptr[1] = 3; ptr[2] = 5; ptr[3] = 7; ptr[4] = 8;
13: index[0] =  0; index[1] =  1; index[2] =  3; index[3] =  1;
14: index[4] =  2; index[5] =  2; index[6] =  3; index[7] =  3;
15: value[0] = 11; value[1] = 21; value[2] = 41; value[3] = 22;
16: value[4] = 32; value[5] = 33; value[6] = 43; value[7] = 44;
17:
18: lis_matrix_set_csc(nnz,ptr,index,value,A);
19: lis_matrix_assemble(A);
```

5.2.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の CSC 形式での格納方法を図 6 に示す. 2 プロセス上にこの行列を CSC 形式で作成する場合, プログラムは以下のように記述する.

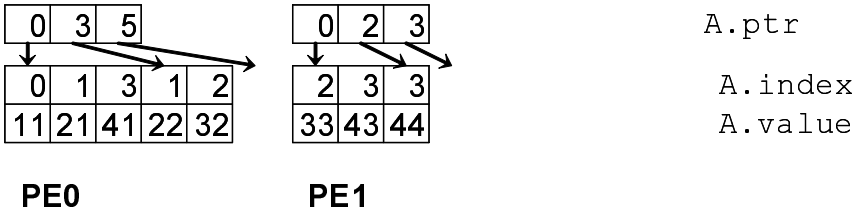


図 6: CSC 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,k,n,nnz,my_rank;
 2: LIS_INT *ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 3;}
 7: else             {n = 2; nnz = 5;}
 8: ptr   = (LIS_INT *)malloc( (n+1)*sizeof(int) );
 9: index = (LIS_INT *)malloc( nnz*sizeof(int) );
10: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
11: lis_matrix_create(MPI_COMM_WORLD,&A);
12: lis_matrix_set_size(A,n,0);
13: if( my_rank==0 ) {
14:     ptr[0] = 0; ptr[1] = 3; ptr[2] = 5;
15:     index[0] =  0; index[1] =  1; index[2] =  3; index[3] =  1; index[4] =  2;
16:     value[0] = 11; value[1] = 21; value[2] = 41; value[3] = 22; value[4] = 32}
17: else {
18:     ptr[0] = 0; ptr[1] = 2; ptr[2] = 3;
19:     index[0] =  2; index[1] =  3; index[2] =  3;
20:     value[0] = 33; value[1] = 43; value[2] = 44;}
21: lis_matrix_set_csc(nnz,ptr,index,value,A);
22: lis_matrix_assemble(A);
```

5.2.3 関連する関数

配列の関連付け

CSC 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_csc(LIS_INT nnz, LIS_INT ptr[], LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_csc(LIS_INTEGER nnz, LIS_INTEGER ptr(), LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.3 Modified Compressed Sparse Row (MSR)

MSR 形式では, データを 2 つの配列 (index,value) に格納する. ndz を対角部分の零要素数とする.

- 長さ $nnz + ndz + 1$ の倍精度配列 value は, 第 n 要素までは行列 A の対角部分を格納する. 第 $n+1$ 要素は使用しない. 第 $n+2$ 要素からは行列 A の対角部分以外の非零要素の値を行方向に沿って格納する.
- 長さ $nnz + ndz + 1$ の整数配列 index は, 第 $n+1$ 要素までは行列 A の非対角部分の各行の開始位置を格納する. 第 $n+2$ 要素からは行列 A の非対角部分の配列 value に格納された非零要素の列番号を格納する.

5.3.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の MSR 形式での格納方法を図 7 に示す. この行列を MSR 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

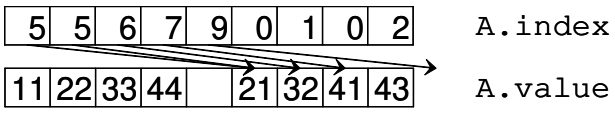


図 7: MSR 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,nnz,ndz;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8; ndz = 0;
 6: index = (LIS_INT *)malloc( (nnz+ndz+1)*sizeof(int) );
 7: value = (LIS_SCALAR *)malloc( (nnz+ndz+1)*sizeof(LIS_SCALAR) );
 8: lis_matrix_create(0,&A);
 9: lis_matrix_set_size(A,0,n);
10:
11: index[0] =  5; index[1] =  5; index[2] =  6; index[3] =  7;
12: index[4] =  9; index[5] =  0; index[6] =  1; index[7] =  0; index[8] =  2;
13: value[0] = 11; value[1] = 22; value[2] = 33; value[3] = 44;
14: value[4] =  0; value[5] = 21; value[6] = 32; value[7] = 41; value[8] = 43;
15:
16: lis_matrix_set_msr(nnz,ndz,index,value,A);
17: lis_matrix_assemble(A);
```

5.3.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の MSR 形式での格納方法を図 8 に示す. 2 プロセス上にこの行列を MSR 形式で作成する場合, プログラムは以下のように記述する.

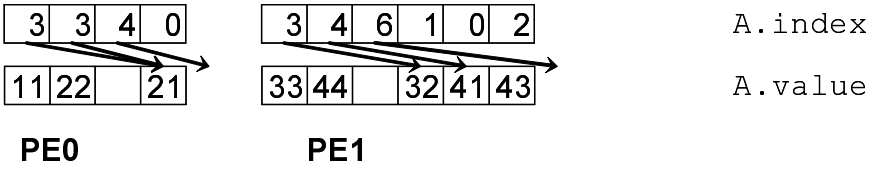


図 8: MSR 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,k,n,nnz,ndz,my_rank;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 3; ndz = 0;}
 7: else             {n = 2; nnz = 5; ndz = 0;}
 8: index = (LIS_INT *)malloc( (nnz+ndz+1)*sizeof(int) );
 9: value = (LIS_SCALAR *)malloc( (nnz+ndz+1)*sizeof(LIS_SCALAR) );
10: lis_matrix_create(MPI_COMM_WORLD,&A);
11: lis_matrix_set_size(A,n,0);
12: if( my_rank==0 ) {
13:     index[0] =  3; index[1] =  3; index[2] =  4; index[3] =  0;
14:     value[0] = 11; value[1] = 22; value[2] =  0; value[3] = 21;}
15: else {
16:     index[0] =  3; index[1] =  4; index[2] =  6; index[3] =  1;
17:     index[4] =  0; index[5] =  2;
18:     value[0] = 33; value[1] = 44; value[2] =  0; value[3] = 32;
19:     value[4] = 41; value[5] = 43;}
20: lis_matrix_set_msr(nnz,ndz,index,value,A);
21: lis_matrix_assemble(A);
```

5.3.3 関連する関数

配列の関連付け

MSR 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_msr(LIS_INT nnz, LIS_INT ndz, LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_msr(LIS_INTEGER nnz, LIS_INTEGER ndz, LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.4 Diagonal (DIA)

DIA 形式では, データを 2 つの配列 (index, value) に格納する. nnd を行列 A の非零な対角要素の本数とする.

- 長さ $nnd \times n$ の倍精度配列 value は, 行列 A の非零な対角要素の値を格納する.
- 長さ nnd の整数配列 index は, 主対角要素から各対角要素へのオフセットを格納する.

マルチスレッド環境では以下のように格納する.
データを 2 つの配列 (index, value) に格納する. $nprocs$ をスレッド数とする. nnd_p を行列 A を行ブロック分割した部分行列の非零な対角部分の本数とする. $maxnnd$ を nnd_p の値の最大値とする.

- 長さ $maxnnd \times n$ の倍精度配列 value は, 行列 A を行ブロック分割した部分行列の非零な対角要素の値を格納する.
- 長さ $nprocs \times maxnnd$ の整数配列 index は, 主対角要素から各対角要素へのオフセットを格納する.

5.4.1 行列の作成 (逐次環境)

行列 A の DIA 形式での格納方法を図 9 に示す. この行列を DIA 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

-3	-1	0										A.index
0	0	0	41	0	21	32	43	11	22	33	44	A.value

図 9: DIA 形式のデータ構造 (逐次環境)

逐次環境

```
 1: LIS_INT n,nnd;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnd = 3;
 6: index = (LIS_INT *)malloc( nnd*sizeof(int) );
 7: value = (LIS_SCALAR *)malloc( n*nnd*sizeof(LIS_SCALAR) );
 8: lis_matrix_create(0,&A);
 9: lis_matrix_set_size(A,0,n);
10:
11: index[0] = -3; index[1] = -1; index[2] =  0;
12: value[0] =  0; value[1] =  0; value[2] =  0; value[3] = 41;
13: value[4] =  0; value[5] = 21; value[6] = 32; value[7] = 43;
14: value[8] = 11; value[9] = 22; value[10]= 33; value[11]= 44;
15:
16: lis_matrix_set_dia(nnd,index,value,A);
17: lis_matrix_assemble(A);
```

5.4.2 行列の作成 (マルチスレッド環境)

2 スレッド上への行列 A の DIA 形式での格納方法を図 10 に示す. 2 スレッド上にこの行列を DIA 形式で作成する場合, プログラムは以下のように記述する.

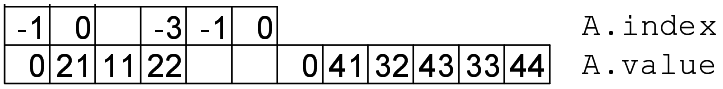


図 10: DIA 形式のデータ構造 (マルチスレッド環境)

マルチスレッド環境

```
 1: LIS_INT n,maxnnd,nprocs;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; maxnnd = 3; nprocs = 2;
 6: index = (LIS_INT *)malloc( maxnnd*sizeof(int) );
 7: value = (LIS_SCALAR *)malloc( n*maxnnd*sizeof(LIS_SCALAR) );
 8: lis_matrix_create(0,&A);
 9: lis_matrix_set_size(A,0,n);
10:
11: index[0] = -1; index[1] =  0; index[2] =  0; index[3] = -3; index[4] = -1; index[5] =  0;
12: value[0] =  0; value[1] = 21; value[2] = 11; value[3] = 22; value[4] =  0; value[5] =  0;
13: value[6] =  0; value[7] = 41; value[8] = 32; value[9] = 43; value[10]= 33; value[11]= 44;
14:
15: lis_matrix_set_dia(maxnnd,index,value,A);
16: lis_matrix_assemble(A);
```

5.4.3 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の DIA 形式での格納方法を図 11 に示す. 2 プロセス上にこの行列を DIA 形式で作成する場合, プログラムは以下のように記述する.

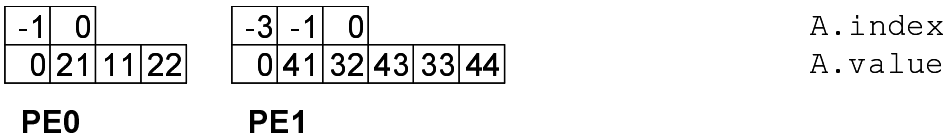


図 11: DIA 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,n,nnd,my_rank;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnd = 2;}
 7: else             {n = 2; nnd = 3;}
 8: index = (LIS_INT *)malloc( nnd*sizeof(int) );
 9: value = (LIS_SCALAR *)malloc( n*nnd*sizeof(LIS_SCALAR) );
10: lis_matrix_create(MPI_COMM_WORLD,&A);
11: lis_matrix_set_size(A,n,0);
12: if( my_rank==0 ) {
13:     index[0] = -1; index[1] =  0;
14:     value[0] =  0; value[1] = 21; value[2] = 11; value[3] = 22;}
15: else {
16:     index[0] = -3; index[1] = -1; index[2] =  0;
17:     value[0] =  0; value[1] = 41; value[2] = 32; value[3] = 43; value[4] = 33;
18:     value[5] = 44;}
19: lis_matrix_set_dia(nnd,index,value,A);
20: lis_matrix_assemble(A);
```

5.4.4 関連する関数

配列の関連付け

DIA 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_dia(LIS_INT nnd, LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_dia(LIS_INTEGER nnd, LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.5 Ellpack-Itpack Generalized Diagonal (ELL)

ELL 形式では, データを 2 つの配列 (index, value) に格納する. $maxnzr$ を行列 A の各行での非零要素数の最大値とする.

- 長さ $maxnzr \times n$ の倍精度配列 value は, 行列 A の各行の非零要素の値を列方向に沿って格納する. 最初の列は各行の最初の非零要素からなる. ただし, 格納する非零要素がない場合は 0 を格納する.
- 長さ $maxnzr \times n$ の整数配列 index は, 配列 value に格納された非零要素の列番号を格納する. ただし, 第 i 行の非零要素数を nnz とすると index[$nnz \times n + i$] にはその行番号 i を格納する.

5.5.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の ELL 形式での格納方法を図 12 に示す. この行列を ELL 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

0	0	1	0	0	1	2	2	0	1	2	3	A.index
11	21	32	41	0	22	33	43	0	0	0	44	A.value

図 12: ELL 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,maxnzr;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; maxnzr = 3;
 6: index = (LIS_INT *)malloc( n*maxnzr*sizeof(int) );
 7: value = (LIS_SCALAR *)malloc( n*maxnzr*sizeof(LIS_SCALAR) );
 8: lis_matrix_create(0,&A);
 9: lis_matrix_set_size(A,0,n);
10:
11: index[0] =  0; index[1] =  0; index[2] =  1; index[3] =  0; index[4] =  0; index[5] =  1;
12: index[6] =  2; index[7] =  2; index[8] =  0; index[9] =  1; index[10]=  2; index[11]=  3;
13: value[0] = 11; value[1] = 21; value[2] = 32; value[3] = 41; value[4] =  0; value[5] = 22;
14: value[6] = 33; value[7] = 43; value[8] =  0; value[9] =  0; value[10]=  0; value[11]= 44;
15:
16: lis_matrix_set_ell(maxnzr,index,value,A);
17: lis_matrix_assemble(A);
```

5.5.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の ELL 形式での格納方法を図 13 に示す. 2 プロセス上にこの行列を ELL 形式で作成する場合, プログラムは以下のように記述する.

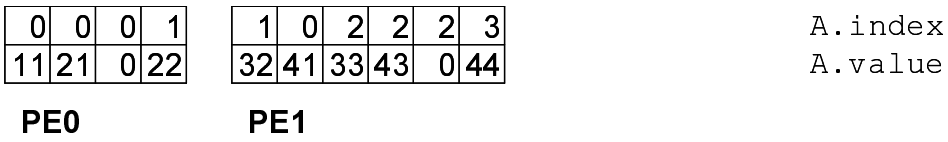


図 13: ELL 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,n,maxnzr,my_rank;
 2: LIS_INT *index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; maxnzr = 2;}
 7: else             {n = 2; maxnzr = 3;}
 8: index = (LIS_INT *)malloc( n*maxnzr*sizeof(int) );
 9: value = (LIS_SCALAR *)malloc( n*maxnzr*sizeof(LIS_SCALAR) );
10: lis_matrix_create(MPI_COMM_WORLD,&A);
11: lis_matrix_set_size(A,n,0);
12: if( my_rank==0 ) {
13:     index[0] =  0; index[1] =  0; index[2] =  0; index[3] =  1;
14:     value[0] = 11; value[1] = 21; value[2] =  0; value[3] = 22;}
15: else {
16:     index[0] =  1; index[1] =  0; index[2] =  2; index[3] =  2; index[4] =  2;
17:     index[5] =  3;
18:     value[0] = 32; value[1] = 41; value[2] = 33; value[3] = 43; value[4] =  0;
19:     value[5] = 44;}
20: lis_matrix_set_ell(maxnzr,index,value,A);
21: lis_matrix_assemble(A);
```

5.5.3 関連する関数

配列の関連付け

ELL 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_ell(LIS_INT maxnzr, LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_ell(LIS_INTEGER maxnzr, LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.6 Jagged Diagonal (JAD)

JAD 形式では, 最初に各行の非零要素数の大きい順に行の並び替えを行い, 各行の非零要素を列方向に沿って格納する. JAD 形式では, データを 4 つの配列 (perm, ptr, index, value) に格納する. $maxnzr$ を行列 A の各行での非零要素数の最大値とする.

- 長さ n の整数配列 perm は, 並び替えた行番号を格納する.
- 長さ nnz の倍精度配列 value は, 並び替えられた行列 A の鋸歯状対角要素の値を格納する. 最初の鋸歯状対角要素は各行の第 1 非零要素からなる. 次の鋸歯状対角要素は各行の第 2 非零要素からなる. これを順次繰り返していく.
- 長さ nnz の整数配列 index は, 配列 value に格納された非零要素の列番号を格納する.
- 長さ $maxnzr + 1$ の整数配列 ptr は, 各鋸歯状対角要素の開始位置を格納する.

マルチスレッド環境では以下のように格納する.
データを 4 つの配列 (perm, ptr, index, value) に格納する. $nprocs$ をスレッド数とする. $maxnzr_p$ を行列 A を行ブロック分割した部分行列の各行での非零要素数の最大値とする. $maxmaxnzr$ は配列 $maxnzr_p$ の値の最大値である.

- 長さ n の整数配列 perm は, 行列 A を行ブロック分割した部分行列を並び替えた行番号を格納する.
- 長さ nnz の倍精度配列 value は, 並び替えられた行列 A の鋸歯状対角要素の値を格納する. 最初の鋸歯状対角要素は各行の第 1 非零要素からなる. 次の鋸歯状対角要素は各行の第 2 非零要素からなる. これを順次繰り返していく.
- 長さ nnz の整数配列 index は, 配列 value に格納された非零要素の列番号を格納する.
- 長さ $nprocs \times (maxmaxnzr + 1)$ の整数配列 ptr は, 行列 A を行ブロック分割した部分行列の各鋸歯状対角要素の開始位置を格納する.

5.6.1 行列の作成 (逐次環境)

行列 A の JAD 形式での格納方法を図 14 に示す. この行列を JAD 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

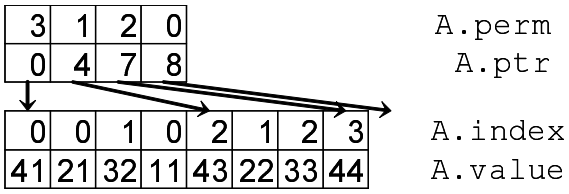


図 14: JAD 形式のデータ構造 (逐次環境)

逐次環境

```
 1: LIS_INT n,nnz,maxnzr;
 2: LIS_INT *perm,*ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8; maxnzr = 3;
 6: perm  = (LIS_INT *)malloc( n*sizeof(int) );
 7: ptr   = (LIS_INT *)malloc( (maxnzr+1)*sizeof(int) );
 8: index = (LIS_INT *)malloc( nnz*sizeof(int) );
 9: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
10: lis_matrix_create(0,&A);
11: lis_matrix_set_size(A,0,n);
12:
13: perm[0] = 3; perm[1] = 1; perm[2] = 2; perm[3] = 0;
14: ptr[0]  = 0; ptr[1]  = 4; ptr[2]  = 7; ptr[3]  = 8;
15: index[0] =  0; index[1] =  0; index[2] =  1; index[3] =  0;
16: index[4] =  2; index[5] =  1; index[6] =  2; index[7] =  3;
17: value[0] = 41; value[1] = 21; value[2] = 32; value[3] = 11;
18: value[4] = 43; value[5] = 22; value[6] = 33; value[7] = 44;
19:
20: lis_matrix_set_jad(nnz,maxnzr,perm,ptr,index,value,A);
21: lis_matrix_assemble(A);
```

5.6.2 行列の作成 (マルチスレッド環境)

2 スレッド上への行列 A の JAD 形式での格納方法を図 15 に示す. 2 スレッド上にこの行列を JAD 形式で作成する場合, プログラムは以下のように記述する.

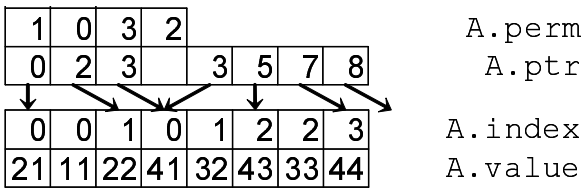


図 15: JAD 形式のデータ構造 (マルチスレッド環境)

マルチスレッド環境

```
 1: LIS_INT n,nnz,maxmaxnzr,nprocs;
 2: LIS_INT *perm,*ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8; maxmaxnzr = 3; nprocs = 2;
 6: perm  = (LIS_INT *)malloc( n*sizeof(int) );
 7: ptr   = (LIS_INT *)malloc( nprocs*(maxmaxnzr+1)*sizeof(int) );
 8: index = (LIS_INT *)malloc( nnz*sizeof(int) );
 9: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
10: lis_matrix_create(0,&A);
11: lis_matrix_set_size(A,0,n);
12:
13: perm[0] = 1; perm[1] = 0; perm[2] = 3; perm[3] = 2;
14: ptr[0]  = 0; ptr[1]  = 2; ptr[2]  = 3; ptr[3]  = 0;
15: ptr[4]  = 3; ptr[5]  = 5; ptr[6]  = 7; ptr[7]  = 8;
16: index[0] =  0; index[1] =  0; index[2] =  1; index[3] =  0;
17: index[4] =  1; index[5] =  2; index[6] =  2; index[7] =  3;
18: value[0] = 21; value[1] = 11; value[2] = 22; value[3] = 41;
19: value[4] = 32; value[5] = 43; value[6] = 33; value[7] = 44;
20:
21: lis_matrix_set_jad(nnz,maxmaxnzr,perm,ptr,index,value,A);
22: lis_matrix_assemble(A);
```

5.6.3 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の JAD 形式での格納方法を図 16 に示す. 2 プロセス上にこの行列を JAD 形式で作成する場合, プログラムは以下のように記述する.

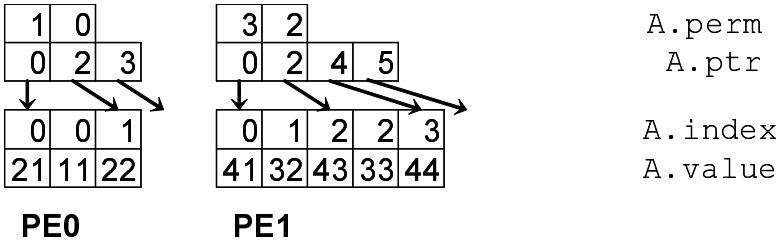


図 16: JAD 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT i,n,nnz,maxnzr,my_rank;
 2: LIS_INT *perm,*ptr,*index;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 3; maxnzr = 2;}
 7: else             {n = 2; nnz = 5; maxnzr = 3;}
 8: perm  = (LIS_INT *)malloc( n*sizeof(int) );
 9: ptr   = (LIS_INT *)malloc( (maxnzr+1)*sizeof(int) );
10: index = (LIS_INT *)malloc( nnz*sizeof(int) );
11: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
12: lis_matrix_create(MPI_COMM_WORLD,&A);
13: lis_matrix_set_size(A,n,0);
14: if( my_rank==0 ) {
15:     perm[0] = 1; perm[1] = 0;
16:     ptr[0]  = 0; ptr[1]  = 2; ptr[2]  = 3;
17:     index[0] =  0; index[1] =  0; index[2] =  1;
18:     value[0] = 21; value[1] = 11; value[2] = 22;}
19: else {
20:     perm[0] = 3; perm[1] = 2;
21:     ptr[0]  = 0; ptr[1]  = 2; ptr[2]  = 4; ptr[3]  = 5;
22:     index[0] =  0; index[1] =  1; index[2] =  2; index[3] =  2; index[4] =  3;
23:     value[0] = 41; value[1] = 32; value[2] = 43; value[3] = 33; value[4] = 44;}
24: lis_matrix_set_jad(nnz,maxnzr,perm,ptr,index,value,A);
25: lis_matrix_assemble(A);
```

5.6.4 関連する関数

配列の関連付け

JAD 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_jad(LIS_INT nnz, LIS_INT maxnzr, LIS_INT perm[], LIS_INT ptr[], LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)

- Fortran subroutine lis_matrix_set_jad(LIS_INTEGER nnz, LIS_INTEGER maxnzr, LIS_INTEGER perm(), LIS_INTEGER ptr(), LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.7 Block Sparse Row (BSR)

BSR 形式では, 行列を $r \times c$ の大きさの部分行列 (ブロックと呼ぶ) に分解する. BSR 形式では, CSR 形式と同様の手順で非零ブロック (少なくとも 1 つの非零要素が存在する) を格納する. $nr = n/r$, $bnnz$ を A の非零ブロック数とする. BSR 形式では, データを 3 つの配列 (bptr, bindex, value) に格納する.

- 長さ $bnnz \times r \times c$ の倍精度配列 value は, 非零ブロックの全要素の値を格納する.
- 長さ $bnnz$ の整数配列 bindex は, 非零ブロックのブロック列番号を格納する.
- 長さ $nr + 1$ の整数配列 bptr は, 配列 bindex のブロック行の開始位置を格納する.

5.7.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の BSR 形式での格納方法を図 17 に示す. この行列を BSR 形式で作成する場合, プログラムは以下のように記述する.

$$A = \left(\begin{array}{cc|cc} 11 & & & \\ 21 & 22 & & \\ \hline & 32 & 33 & \\ 41 & & 43 & 44 \end{array}\right)$$

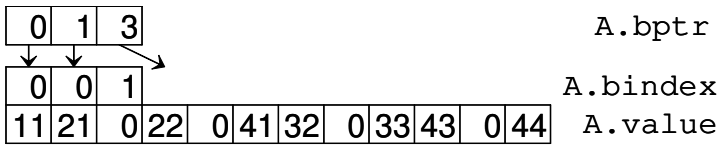


図 17: BSR 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,bnr,bnc,nr,nc,bnnz;
 2: LIS_INT *bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; bnr = 2; bnc = 2; bnnz = 3; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;
 6: bptr   = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 7: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
 8: value  = (LIS_SCALAR *)malloc( bnr*bnc*bnnz*sizeof(LIS_SCALAR) );
 9: lis_matrix_create(0,&A);
10: lis_matrix_set_size(A,0,n);
11:
12: bptr[0] = 0; bptr[1] = 1; bptr[2] = 3;
13: bindex[0] =  0; bindex[1] =  0; bindex[2] =  1;
14: value[0]  = 11; value[1] = 21; value[2] =  0; value[3] = 22;
15: value[4]  =  0; value[5] = 41; value[6] = 32; value[7] =  0;
16: value[8]  = 33; value[9] = 43; value[10]=  0; value[11]= 44;
17:
18: lis_matrix_set_bsr(bnr,bnc,bnnz,bptr,bindex,value,A);
19: lis_matrix_assemble(A);
```

5.7.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の BSR 形式での格納方法を図 18 に示す. 2 プロセス上にこの行列を BSR 形式で作成する場合, プログラムは以下のように記述する.

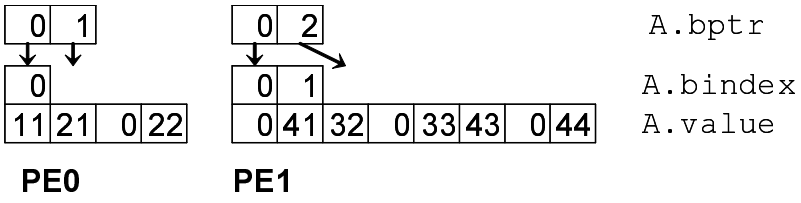


図 18: BSR 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT n,bnr,bnc,nr,nc,bnnz,my_rank;
 2: LIS_INT *bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; bnr = 2; bnc = 2; bnnz = 1; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;}
 7: else             {n = 2; bnr = 2; bnc = 2; bnnz = 2; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;}
 8: bptr   = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 9: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
10: value  = (LIS_SCALAR *)malloc( bnr*bnc*bnnz*sizeof(LIS_SCALAR) );
11: lis_matrix_create(MPI_COMM_WORLD,&A);
12: lis_matrix_set_size(A,n,0);
13: if( my_rank==0 ) {
14:     bptr[0] = 0; bptr[1] = 1;
15:     bindex[0] =  0;
16:     value[0]  = 11; value[1] = 21; value[2] =  0; value[3] = 22;}
17: else {
18:     bptr[0] = 0; bptr[1] = 2;
19:     bindex[0] =  0; bindex[1] =  1;
20:     value[0]  =  0; value[1]  = 41; value[2] = 32; value[3] =  0;
21:     value[4]  = 33; value[5]  = 43; value[6] =  0; value[7] = 44;}
22: lis_matrix_set_bsr(bnr,bnc,bnnz,bptr,bindex,value,A);
23: lis_matrix_assemble(A);
```

5.7.3 関連する関数

配列の関連付け

BSR 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_bsr(LIS_INT bnr, LIS_INT bnc, LIS_INT bnnz, LIS_INT bptr[], LIS_INT bindex[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_bsr(LIS_INTEGER bnr, LIS_INTEGER bnc, LIS_INTEGER bnnz, LIS_INTEGER bptr(), LIS_INTEGER bindex(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.8 Block Sparse Column (BSC)

BSC 形式では, 行列を $r \times c$ の大きさの部分行列 (ブロックと呼ぶ) に分解する. BSC 形式では, CSC 形式と同様の手順で非零ブロック (少なくとも 1 つの非零要素が存在する) を格納する. $nc = n/c$, $bnnz$ を A の非零ブロック数とする. BSC 形式では, データを 3 つの配列 (bptr, bindex, value) に格納する.

- 長さ $bnnz \times r \times c$ の倍精度配列 value は, 非零ブロックの全要素の値を格納する.
- 長さ $bnnz$ の整数配列 bindex は, 非零ブロックのブロック行番号を格納する.
- 長さ $nc + 1$ の整数配列 bptr は, 配列 bindex のブロック列の開始位置を格納する.

5.8.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の BSC 形式での格納方法を図 19 に示す. この行列を BSC 形式で作成する場合, プログラムは以下のように記述する.

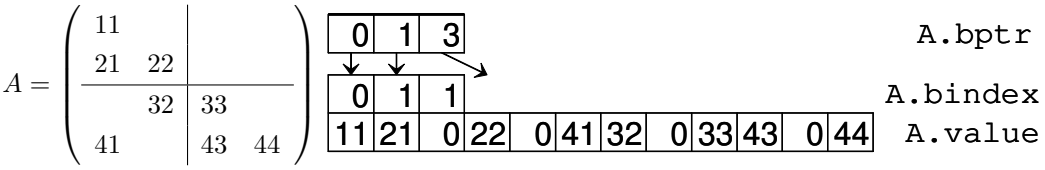


図 19: BSC 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,bnr,bnc,nr,nc,bnnz;
 2: LIS_INT *bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; bnr = 2; bnc = 2; bnnz = 3; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;
 6: bptr   = (LIS_INT *)malloc( (nc+1)*sizeof(int) );
 7: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
 8: value  = (LIS_SCALAR *)malloc( bnr*bnc*bnnz*sizeof(LIS_SCALAR) );
 9: lis_matrix_create(0,&A);
10: lis_matrix_set_size(A,0,n);
11:
12: bptr[0] = 0; bptr[1] = 1; bptr[2] = 3;
13: bindex[0] =  0; bindex[1] =  1; bindex[2] =  1;
14: value[0]  = 11; value[1] = 21; value[2] =  0; value[3] = 22;
15: value[4]  =  0; value[5] = 41; value[6] = 32; value[7] =  0;
16: value[8]  = 33; value[9] = 43; value[10]=  0; value[11]= 44;
17:
18: lis_matrix_set_bsc(bnr,bnc,bnnz,bptr,bindex,value,A);
19: lis_matrix_assemble(A);
```

5.8.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の BSC 形式での格納方法を図 20 に示す. 2 プロセス上にこの行列を BSC 形式で作成する場合, プログラムは以下のように記述する.

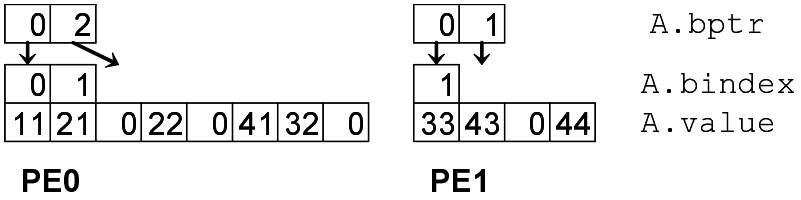


図 20: BSC 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT n,bnr,bnc,nr,nc,bnnz,my_rank;
 2: LIS_INT *bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; bnr = 2; bnc = 2; bnnz = 2; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;}
 7: else             {n = 2; bnr = 2; bnc = 2; bnnz = 1; nr = (n-1)/bnr+1; nc = (n-1)/bnc+1;}
 8: bptr   = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 9: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
10: value  = (LIS_SCALAR *)malloc( bnr*bnc*bnnz*sizeof(LIS_SCALAR) );
11: lis_matrix_create(MPI_COMM_WORLD,&A);
12: lis_matrix_set_size(A,n,0);
13: if( my_rank==0 ) {
14:     bptr[0] = 0; bptr[1] = 2;
15:     bindex[0] =  0; bindex[1] =  1;
16:     value[0]  = 11; value[1]  = 21; value[2] =  0; value[3] = 22;
17:     value[4]  =  0; value[5]  = 41; value[6] = 32; value[7] =  0;}
18: else {
19:     bptr[0] = 0; bptr[1] = 1;
20:     bindex[0] =  1;
21:     value[0]  = 33; value[1]  = 43; value[2] =  0; value[3] = 44;}
22: lis_matrix_set_bsc(bnr,bnc,bnnz,bptr,bindex,value,A);
23: lis_matrix_assemble(A);
```

5.8.3 関連する関数

配列の関連付け

BSC 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_bsc(LIS_INT bnr, LIS_INT bnc, LIS_INT bnnz, LIS_INT bptr[], LIS_INT bindex[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_bsc(LIS_INTEGER bnr, LIS_INTEGER bnc, LIS_INTEGER bnnz, LIS_INTEGER bptr(), LIS_INTEGER bindex(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.9 Variable Block Row (VBR)

VBR 形式は BSR 形式を一般化したものである. 行と列の分割位置は配列 (`row`, `col`) で与えられる. VBR 形式では, CSR 形式と同様の手順で非零ブロック (少なくとも 1 つの非零要素が存在する) を格納する. nr, nc をそれぞれ行分割数, 列分割数とする. $bnnz$ を A の非零ブロック数, nnz を非零ブロックの全要素数とする. VBR 形式では, データを 6 つの配列 (`bptr`, `bindex`, `row`, `col`, `ptr`, `value`) に格納する.

- 長さ $nr + 1$ の整数配列 `row` は, ブロック行の開始行番号を格納する.
- 長さ $nc + 1$ の整数配列 `col` は, ブロック列の開始列番号を格納する.
- 長さ $bnnz$ の整数配列 `bindex` は, 非零ブロックのブロック列番号を格納する.
- 長さ $nr + 1$ の整数配列 `bptr` は, 配列 `bindex` のブロック行の開始位置を格納する.
- 長さ nnz の倍精度配列 `value` は, 非零ブロックの全要素の値を格納する.
- 長さ $bnnz + 1$ の整数配列 `ptr` は, 配列 `value` の非零ブロックの開始位置を格納する.

5.9.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の VBR 形式での格納方法を図 21 に示す. この行列を VBR 形式で作成する場合, プログラムは以下のように記述する.

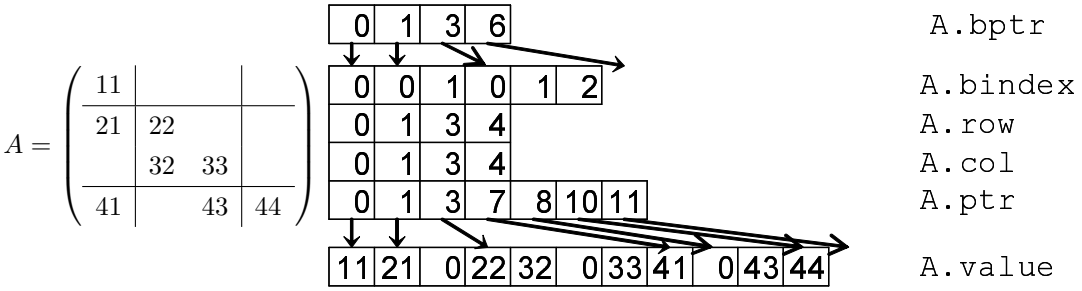


図 21: VBR 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,nnz,nr,nc,bnnz;
 2: LIS_INT *row,*col,*ptr,*bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 11; bnnz = 6; nr = 3; nc = 3;
 6: bptr   = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 7: row    = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 8: col    = (LIS_INT *)malloc( (nc+1)*sizeof(int) );
 9: ptr    = (LIS_INT *)malloc( (bnnz+1)*sizeof(int) );
10: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
11: value  = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
12: lis_matrix_create(0,&A);
13: lis_matrix_set_size(A,0,n);
14:
15: bptr[0] = 0; bptr[1] = 1; bptr[2] = 3; bptr[3] = 6;
16: row[0]  = 0; row[1]  = 1; row[2]  = 3; row[3] = 4;
17: col[0]  = 0; col[1]  = 1; col[2]  = 3; col[3] = 4;
18: bindex[0] =  0; bindex[1] =  0; bindex[2] =  1; bindex[3] =  0;
19: bindex[4] =  1; bindex[5] =  2;
20: ptr[0]    =  0; ptr[1]    =  1; ptr[2]    =  3; ptr[3]    =  7;
21: ptr[4]    =  8; ptr[5]    = 10; ptr[6]    = 11;
22: value[0]  = 11; value[1]  = 21; value[2]  =  0; value[3]  = 22;
23: value[4]  = 32; value[5]  =  0; value[6]  = 33; value[7]  = 41;
24: value[8]  =  0; value[9]  = 43; value[10] = 44;
25:
26: lis_matrix_set_vbr(nnz,nr,nc,bnnz,row,col,ptr,bptr,bindex,value,A);
27: lis_matrix_assemble(A);
```

5.9.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の VBR 形式での格納方法を図 22 に示す. 2 プロセス上にこの行列を VBR 形式で作成する場合, プログラムは以下のように記述する.

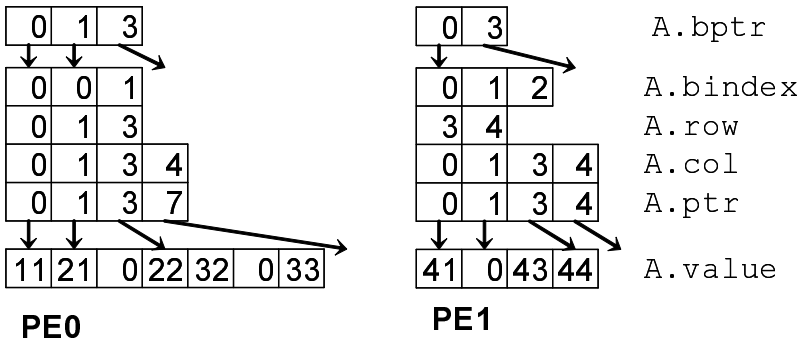


図 22: VBR 形式のデータ構造 (逐次・マルチスレッド環境)

マルチプロセス環境

```
 1: LIS_INT n,nnz,nr,nc,bnnz,my_rank;
 2: LIS_INT *row,*col,*ptr,*bptr,*bindex;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 7; bnnz = 3; nr = 2; nc = 3;}
 7: else             {n = 2; nnz = 4; bnnz = 3; nr = 1; nc = 3;}
 8: bptr   = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
 9: row    = (LIS_INT *)malloc( (nr+1)*sizeof(int) );
10: col    = (LIS_INT *)malloc( (nc+1)*sizeof(int) );
11: ptr    = (LIS_INT *)malloc( (bnnz+1)*sizeof(int) );
12: bindex = (LIS_INT *)malloc( bnnz*sizeof(int) );
13: value  = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
14: lis_matrix_create(MPI_COMM_WORLD,&A);
15: lis_matrix_set_size(A,n,0);
16: if( my_rank==0 ) {
17:     bptr[0] = 0; bptr[1] = 1; bptr[2] = 3;
18:     row[0]  = 0; row[1]  = 1; row[2]  = 3;
19:     col[0]  = 0; col[1]  = 1; col[2]  = 3; col[3] = 4;
20:     bindex[0] =  0; bindex[1] =  0; bindex[2] =  1;
21:     ptr[0]    =  0; ptr[1]    =  1; ptr[2]    =  3; ptr[3]    =  7;
22:     value[0]  = 11; value[1] = 21; value[2] =  0; value[3] = 22;
23:     value[4]  = 32; value[5] =  0; value[6] = 33;}
24: else {
25:     bptr[0] = 0; bptr[1] = 3;
26:     row[0]  = 3; row[1]  = 4;
27:     col[0]  = 0; col[1]  = 1; col[2]  = 3; col[3] = 4;
28:     bindex[0] =  0; bindex[1] =  1; bindex[2] =  2;
29:     ptr[0]    =  0; ptr[1]    =  1; ptr[2]    =  3; ptr[3]    =  4;
30:     value[0]  = 41; value[1]  =  0; value[2]  = 43; value[3]  = 44;}
31: lis_matrix_set_vbr(nnz,nr,nc,bnnz,row,col,ptr,bptr,bindex,value,A);
32: lis_matrix_assemble(A);
```

5.9.3 関連する関数

配列の関連付け

VBR 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_vbr(LIS_INT nnz, LIS_INT nr, LIS_INT nc, LIS_INT bnnz, LIS_INT row[], LIS_INT col[], LIS_INT ptr[], LIS_INT bptr[], LIS_INT bindex[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_vbr(LIS_INTEGER nnz, LIS_INTEGER nr, LIS_INTEGER nc, LIS_INTEGER bnnz, LIS_INTEGER row(), LIS_INTEGER col(), LIS_INTEGER ptr(), LIS_INTEGER bptr(), LIS_INTEGER bindex(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.10 Coordinate (COO)

COO 形式では, データを 3 つの配列 (row, col, value) に格納する.

- 長さ nnz の倍精度配列 value は, 非零要素の値を格納する.
- 長さ nnz の整数配列 row は, 非零要素の行番号を格納する.
- 長さ nnz の整数配列 col は, 非零要素の列番号を格納する.

5.10.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の COO 形式での格納方法を図 23 に示す. この行列を COO 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

0	1	3	1	2	2	3	3	A.row
0	0	0	1	1	2	2	3	A.col
11	21	41	22	32	33	43	44	A.value

図 23: COO 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n,nnz;
 2: LIS_INT *row,*col;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: n = 4; nnz = 8;
 6: row   = (LIS_INT *)malloc( nnz*sizeof(int) );
 7: col   = (LIS_INT *)malloc( nnz*sizeof(int) );
 8: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
 9: lis_matrix_create(0,&A);
10: lis_matrix_set_size(A,0,n);
11:
12: row[0] = 0; row[1] = 1; row[2] = 3; row[3] = 1;
13: row[4] = 2; row[5] = 2; row[6] = 3; row[7] = 3;
14: col[0] = 0; col[1] = 0; col[2] = 0; col[3] = 1;
15: col[4] = 1; col[5] = 2; col[6] = 2; col[7] = 3;
16: value[0] = 11; value[1] = 21; value[2] = 41; value[3] = 22;
17: value[4] = 32; value[5] = 33; value[6] = 43; value[7] = 44;
18:
19: lis_matrix_set_coo(nnz,row,col,value,A);
20: lis_matrix_assemble(A);
```

5.10.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の COO 形式での格納方法を図 24 に示す. 2 プロセス上にこの行列を COO 形式で作成する場合, プログラムは以下のように記述する.

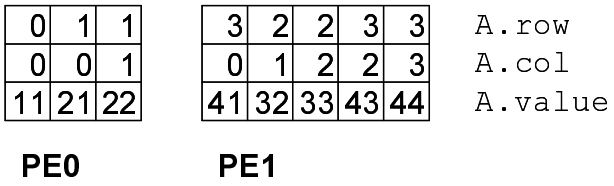


図 24: COO 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT n,nnz,my_rank;
 2: LIS_INT *row,*col;
 3: LIS_SCALAR *value;
 4: LIS_MATRIX A;
 5: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 6: if( my_rank==0 ) {n = 2; nnz = 3;}
 7: else             {n = 2; nnz = 5;}
 8: row   = (LIS_INT *)malloc( nnz*sizeof(int) );
 9: col   = (LIS_INT *)malloc( nnz*sizeof(int) );
10: value = (LIS_SCALAR *)malloc( nnz*sizeof(LIS_SCALAR) );
11: lis_matrix_create(MPI_COMM_WORLD,&A);
12: lis_matrix_set_size(A,n,0);
13: if( my_rank==0 ) {
14:     row[0] = 0; row[1] = 1; row[2] = 1;
15:     col[0] = 0; col[1] = 0; col[2] = 1;
16:     value[0] = 11; value[1] = 21; value[2] = 22;}
17: else {
18:     row[0] = 3; row[1] = 2; row[2] = 2; row[3] = 3; row[4] = 3;
19:     col[0] = 0; col[1] = 1; col[2] = 2; col[3] = 2; col[4] = 3;
20:     value[0] = 41; value[1] = 32; value[2] = 33; value[3] = 43; value[4] = 44;}
21: lis_matrix_set_coo(nnz,row,col,value,A);
22: lis_matrix_assemble(A);
```

5.10.3 関連する関数

配列の関連付け

COO 形式の配列を行列 A に関連付けるには, 関数

- C　　LIS_INT lis_matrix_set_coo(LIS_INT nnz, LIS_INT row[], LIS_INT col[], LIS_SCALAR value[], LIS_MATRIX A)
- Fortran subroutine lis_matrix_set_coo(LIS_INTEGER nnz, LIS_INTEGER row(), LIS_INTEGER col(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)

を用いる.

5.11 Dense (DNS)

DNS 形式では, データを 1 つの配列 (value) に格納する.

- 長さ $n \times n$ の倍精度配列 value は, 列優先で要素の値を格納する.

5.11.1 行列の作成 (逐次・マルチスレッド環境)

行列 A の DNS 形式での格納方法を図 25 に示す. この行列を DNS 形式で作成する場合, プログラムは以下のように記述する.

$$A = \begin{pmatrix} 11 & & & \\ 21 & 22 & & \\ & 32 & 33 & \\ 41 & & 43 & 44 \end{pmatrix}$$

11	21	0	41	0	22	32	0
0	0	33	43	0	0	0	44

A.Value

図 25: DNS 形式のデータ構造 (逐次・マルチスレッド環境)

逐次・マルチスレッド環境

```
 1: LIS_INT n;
 2: LIS_SCALAR *value;
 3: LIS_MATRIX A;
 4: n = 4;
 5: value = (LIS_SCALAR *)malloc( n*n*sizeof(LIS_SCALAR) );
 6: lis_matrix_create(0,&A);
 7: lis_matrix_set_size(A,0,n);
 8:
 9: value[0] = 11; value[1] = 21; value[2] =  0; value[3] = 41;
10: value[4] =  0; value[5] = 22; value[6] = 32; value[7] =  0;
11: value[8] =  0; value[9] =  0; value[10]= 33; value[11]= 43;
12: value[12]=  0; value[13]=  0; value[14]=  0; value[15]= 44;
13:
14: lis_matrix_set_dns(value,A);
15: lis_matrix_assemble(A);
```

5.11.2 行列の作成 (マルチプロセス環境)

2 プロセス上への行列 A の DNS 形式での格納方法を図 26 に示す. 2 プロセス上にこの行列を DNS 形式で作成する場合, プログラムは以下のように記述する.

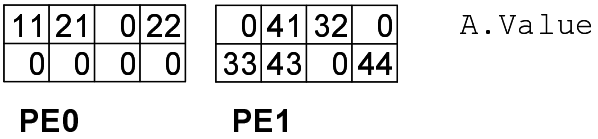


図 26: DNS 形式のデータ構造 (マルチプロセス環境)

マルチプロセス環境

```
 1: LIS_INT n,my_rank;
 2: LIS_SCALAR *value;
 3: LIS_MATRIX A;
 4: MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);
 5: if( my_rank==0 ) {n = 2;}
 6: else             {n = 2;}
 7: value = (LIS_SCALAR *)malloc( n*n*sizeof(LIS_SCALAR) );
 8: lis_matrix_create(MPI_COMM_WORLD,&A);
 9: lis_matrix_set_size(A,n,0);
10: if( my_rank==0 ) {
11:     value[0] = 11; value[1] = 21; value[2] =  0; value[3] = 22;
12:     value[4] =  0; value[5] =  0; value[6] =  0; value[7] =  0;}
13: else {
14:     value[0] =  0; value[1] = 41; value[2] = 32; value[3] =  0;
15:     value[4] = 33; value[5] = 43; value[6] =  0; value[7] = 44;}
16: lis_matrix_set_dns(value,A);
17: lis_matrix_assemble(A);
```

5.11.3 関連する関数

配列の関連付け

DNS 形式の配列を行列 A に関連付けるには, 関数

- C `LIS_INT lis_matrix_set_dns(LIS_SCALAR value[], LIS_MATRIX A)`
- Fortran `subroutine lis_matrix_set_dns(LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)`

を用いる.

6 関数

本節では, ユーザが使用できる関数について述べる. 関数の解説は C を基準に記述する. 配列の要素番号は, C では 0 から, Fortran では 1 から始まるものとする. なお, 各ソルバの状態は以下のように定義する.

LIS_SUCCESS(0)	正常終了
LIS_ILL_OPTION(1)	オプション不正
LIS_BREAKDOWN(2)	ブレイクダウン（ゼロ除算）
LIS_OUT_OF_MEMORY(3)	メモリ不足
LIS_MAXITER(4)	最大反復回数超過
LIS_NOT_IMPLEMENTED(5)	未実装
LIS_ERR_FILE_IO(6)	ファイル I/O エラー

6.1 ベクトルの操作

ベクトル v の次数を $global_n$ とする. ベクトル v を $nprocs$ 個のプロセスで行ブロック分割する場合の各ブロックの行数を $local_n$ とする. $global_n$ をグローバルな次数, $local_n$ をローカルな次数と呼ぶ.

6.1.1 lis_vector_create

```
C       LIS_INT lis_vector_create(LIS_Comm comm, LIS_VECTOR *v)
Fortran subroutine lis_vector_create(LIS_Comm comm, LIS_VECTOR v,
        LIS_INTEGER ierr)
```

機能

ベクトルを作成する.

入力

comm	MPI コミュニケータ

出力

v	ベクトル
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, comm の値は無視される.

6.1.2 lis_vector_destroy

```
C       LIS_INT lis_vector_destroy(LIS_VECTOR v)
Fortran subroutine lis_vector_destroy(LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

不要になったベクトルをメモリから破棄する.

入力

v	メモリから破棄するベクトル

出力

ierr	リターンコード

6.1.3 lis_vector_duplicate

```
C       LIS_INT lis_vector_duplicate(void *vin, LIS_VECTOR *vout)
Fortran subroutine lis_vector_duplicate(LIS_VECTOR vin, LIS_VECTOR vout,
         LIS_INTEGER ierr)
```

機能

既存のベクトルまたは行列と同じ情報を持つベクトルを作成する.

入力

vin	複製元のベクトルまたは行列

出力

vout	複製先のベクトル
ierr	リターンコード

注釈

vin には LIS_VECOTR または LIS_MATRIX を指定することが可能である. 関数 lis_vector_duplicate は要素の値は複製せず, メモリ領域のみ確保する. 値も複製する場合は, この関数の後に関数 lis_vector_copy を呼び出す.

6.1.4 lis_vector_set_size

```
C       LIS_INT lis_vector_set_size(LIS_VECTOR v, LIS_INT local_n,
         LIS_INT global_n)
Fortran subroutine lis_vector_set_size(LIS_VECTOR v, LIS_INTEGER local_n,
         LIS_INTEGER global_n, LIS_INTEGER ierr)
```

機能

ベクトルの次数を設定する.

入力

v	ベクトル
local_n	ベクトルのローカルな次数
global_n	ベクトルのグローバルな次数

出力

ierr	リターンコード

注釈

$local_n$ か $global_n$ のどちらか一方を与えなければならない.

逐次, マルチスレッド環境では, $local_n$ は $global_n$ に等しい. したがって, `lis_vector_set_size(v,n,0)` と `lis_vector_set_size(v,0,n)` は, いずれも次数 n のベクトルを作成する.

マルチプロセス環境においては, `lis_vector_set_size(v,n,0)` は各プロセス上に次数 n の部分ベクトルを作成する. 一方, `lis_vector_set_size(v,0,n)` は各プロセス p 上に次数 m_p の部分ベクトルを作成する. m_p はライブラリ側で決定される.

6.1.5 lis_vector_get_size

```
C       LIS_INT lis_vector_get_size(LIS_VECTOR v, LIS_INT *local_n,
         LIS_INT *global_n)
Fortran subroutine lis_vector_get_size(LIS_VECTOR v, LIS_INTEGER local_n,
         LIS_INTEGER global_n, LIS_INTEGER ierr)
```

機能

ベクトル v の次数を得る.

入力

v	ベクトル

出力

local_n	ベクトルのローカルな次数
global_n	ベクトルのグローバルな次数
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, $local_n$ は $global_n$ に等しい.

6.1.6 lis_vector_get_range

```
C       LIS_INT lis_vector_get_range(LIS_VECTOR v, LIS_INT *is, LIS_INT *ie)
Fortran subroutine lis_vector_get_range(LIS_VECTOR v, LIS_INTEGER is,
         LIS_INTEGER ie, LIS_INTEGER ierr)
```

機能

部分ベクトル v が全体ベクトルのどこに位置するのかを調べる.

入力

v	部分ベクトル

出力

is	部分ベクトル v の全体ベクトル中での開始位置
ie	部分ベクトル v の全体ベクトル中での終了位置+1
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, ベクトルの次数が n ならば, C 版では $is = 0$, $ie = n$, Fortran 版では $is = 1$, $ie = n + 1$ である.

6.1.7 lis_vector_set_value

```
C       LIS_INT lis_vector_set_value(LIS_INT flag, LIS_INT i, LIS_SCALAR value,
         LIS_VECTOR v)
Fortran subroutine lis_vector_set_value(LIS_INTEGER flag, LIS_INTEGER i,
         LIS_SCALAR value, LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

ベクトル v の第 i 行にスカラ値を代入する.

入力

flag	LIS_INS_VALUE 挿入 : $v[i] = value$ LIS_ADD_VALUE 加算代入: $v[i] = v[i] + value$
i	代入する場所
value	代入するスカラ値
v	ベクトル

出力

v	第 i 行にスカラ値が代入されたベクトル
ierr	リターンコード

注釈

マルチプロセス環境では, 全体ベクトルの第 i 行を指定する.

6.1.8 lis_vector_get_value

```
C       LIS_INT lis_vector_get_value(LIS_VECTOR v, LIS_INT i, LIS_SCALAR *value)
Fortran subroutine lis_vector_get_value(LIS_VECTOR v, LIS_INTEGER i,
         LIS_SCALAR value, LIS_INTEGER ierr)
```

機能

ベクトル v の第 i 行の値を取得する.

入力

i	取得する場所
v	値を取得するベクトル

出力

value	スカラ値
ierr	リターンコード

注釈

マルチプロセス環境では, 全体ベクトルの第 i 行を指定する.

6.1.9 lis_vector_set_values

```
C       LIS_INT lis_vector_set_values(LIS_INT flag, LIS_INT count, LIS_INT index[],
         LIS_SCALAR value[], LIS_VECTOR v)
Fortran subroutine lis_vector_set_values(LIS_INTEGER flag, LIS_INTEGER count,
         LIS_INTEGER index(), LIS_SCALAR value(), LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

ベクトル v の第 $index[i]$ 行にスカラ値 $value[i]$ を代入する.

入力

flag	LIS_INS_VALUE 挿入 : $v[index[i]] = value[i]$ LIS_ADD_VALUE 加算代入: $v[index[i]] = v[index[i]] + value[i]$
count	代入するスカラ値を格納する配列の要素数
index	代入する場所を格納する配列
value	代入するスカラ値を格納する配列
v	ベクトル

出力

v	第 $index[i]$ 行にスカラ値 $value[i]$ が代入されたベクトル
ierr	リターンコード

注釈

マルチプロセス環境では, 全体ベクトルの第 $index[i]$ 行を指定する.

6.1.10 lis_vector_get_values

```
C       LIS_INT lis_vector_get_values(LIS_VECTOR v, LIS_INT start, LIS_INT count,
         LIS_SCALAR value[])
Fortran subroutine lis_vector_get_values(LIS_VECTOR v, LIS_INTEGER start,
         LIS_INTEGER count, LIS_SCALAR value(), LIS_INTEGER ierr)
```

機能

ベクトル v の第 $start + i$ 行の値 $(i = 0, 1, ..., count - 1)$ を $value[i]$ に格納する.

入力

start	取得する場所の始点
count	取得するスカラ値の個数
v	値を取得するベクトル

出力

value	取得したスカラ値を格納する配列
ierr	リターンコード

注釈

マルチプロセス環境では, 全体ベクトルの第 $start + i$ 行を指定する.

6.1.11 lis_vector_scatter

```
C       LIS_INT lis_vector_scatter(LIS_SCALAR value[], LIS_VECTOR v)
Fortran subroutine lis_vector_scatter(LIS_SCALAR value(), LIS_VECTOR v,
         LIS_INTEGER ierr)
```

機能

ベクトル v の第 i 行の値 $(i = 0, 1, ..., global_n - 1)$ を $value[i]$ から取得する.

入力

value	取得するスカラ値を格納する配列

出力

v	値を取得したベクトル
ierr	リターンコード

マルチプロセス環境では, 本関数は MPI_Scatterv により実装される.

6.1.12 lis_vector_gather

```
C       LIS_INT lis_vector_gather(LIS_VECTOR v, LIS_SCALAR value[])
Fortran subroutine lis_vector_gather(LIS_VECTOR v, LIS_SCALAR value(),
         LIS_INTEGER ierr)
```

機能

ベクトル v の第 i 行の値 $(i = 0, 1, ..., global_n - 1)$ を $value[i]$ に格納する.

入力

v	値を取得するベクトル

出力

value	取得したスカラ値を格納する配列
ierr	リターンコード

マルチプロセス環境では, 本関数は MPI_Allgatherv により実装される.

6.1.13 lis_vector_is_null

```
C       LIS_INT lis_vector_is_null(LIS_VECTOR v)
Fortran subroutine lis_vector_is_null(LIS_VECTOR v,LIS_INTEGER ierr)
```

機能

ベクトル v が使用可能かどうかを調べる.

入力

v	ベクトル

出力

ierr	リターンコード	
	LIS_TRUE	使用可能
	LIS_FALSE	使用不可

6.2 行列の操作

行列 A の次数を $global_n \times global_n$ とする. 行列 A を $nprocs$ 個のプロセスで行ブロック分割する場合の各部分行列の行数を $local_n$ とする. $global_n$ をグローバルな行数, $local_n$ をローカルな行数と呼ぶ.

6.2.1 lis_matrix_create

```
C       LIS_INT lis_matrix_create(LIS_Comm comm, LIS_MATRIX *A)
Fortran subroutine lis_matrix_create(LIS_Comm comm, LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

行列を作成する.

入力

comm	MPI コミュニケータ

出力

A	行列
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, comm の値は無視される.

6.2.2 lis_matrix_destroy

```
C       LIS_INT lis_matrix_destroy(LIS_MATRIX A)
Fortran subroutine lis_matrix_destroy(LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

不要になった行列をメモリから破棄する.

入力

A	メモリから破棄する行列

出力

ierr	リターンコード

注釈

関数 lis_matrix_destroy により, 行列 A に関連付けられた配列も破棄される.

6.2.3 lis_matrix_duplicate

```
C       LIS_INT lis_matrix_duplicate(LIS_MATRIX Ain, LIS_MATRIX *Aout)
Fortran subroutine lis_matrix_duplicate(LIS_MATRIX Ain, LIS_MATRIX Aout,
         LIS_INTEGER ierr)
```

機能

既存の行列と同じ情報を持つ行列を作成する.

入力

Ain	複製元の行列

出力

Aout	複製先の行列
ierr	リターンコード

注釈

関数 lis_matrix_duplicate は行列の要素の値は複製せず, メモリ領域のみ確保する. 値も複製する場合は, この関数の後に関数 lis_matrix_copy を呼び出す.

6.2.4 lis_matrix_malloc

```
C       LIS_INT lis_matrix_malloc(LIS_MATRIX A, LIS_INT nnz_row, LIS_INT nnz[])
Fortran subroutine lis_matrix_malloc(LIS_MATRIX A, LIS_INTEGER nnz_row,
         LIS_INTEGER nnz[], LIS_INTEGER ierr)
```

機能

行列のメモリ領域を確保する.

入力

A	行列
nnz_row	各行の平均非零要素数
nnz	各行の非零要素数を格納した配列

出力

ierr	リターンコード

注釈

nnz_row または nnz のどちらか一方を指定する. この関数は, lis_matrix_set_value のためにメモリ領域を確保する.

6.2.5 lis_matrix_set_value

```
C       LIS_INT lis_matrix_set_value(LIS_INT flag, LIS_INT i, LIS_INT j,
         LIS_SCALAR value, LIS_MATRIX A)
Fortran subroutine lis_matrix_set_value(LIS_INTEGER flag, LIS_INTEGER i,
         LIS_INTEGER j, LIS_SCALAR value, LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

行列 A の第 i 行第 j 列に値を代入する.

入力

flag	LIS_INS_VALUE 挿入 : $A[i,j] = value$
	LIS_ADD_VALUE 加算代入: $A[i,j] = A[i,j] + value$
i	行列の行番号
j	行列の列番号
value	代入するスカラ値
A	行列

出力

A	第 i 行第 j 列に値が代入された行列
ierr	リターンコード

注釈

マルチプロセス環境では, 全体行列の第 i 行第 j 列を指定する.

関数 lis_matrix_set_value は代入された値を一時的な内部形式で格納するため, lis_matrix_set_value を用いた後には関数 lis_matrix_assemble を呼び出さなければならない.

大規模な行列に対しては, 関数 lis_matrix_set_type の導入を検討すべきである. 詳細については lis-($VERSION)/test/test2.c 及び lis-($VERSION)/test/test2f.F90 を参照のこと.

6.2.6 lis_matrix_assemble

```
C       LIS_INT lis_matrix_assemble(LIS_MATRIX A)
Fortran subroutine lis_matrix_assemble(LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

行列をライブラリで使用可能にする.

入力

A	行列

出力

A	使用可能になった行列
ierr	リターンコード

6.2.7 lis_matrix_set_size

```
LIS_INT lis_matrix_set_size(LIS_MATRIX A, LIS_INT local_n, LIS_INT global_n)
Fortran subroutine lis_matrix_set_size(LIS_MATRIX A, LIS_INTEGER local_n,
         LIS_INTEGER global_n, LIS_INTEGER ierr)
```

機能

行列の次数を設定する.

入力

A	行列
local_n	行列 A のローカルな行数
global_n	行列 A のグローバルな行数

出力

ierr	リターンコード

注釈

$local_n$ か $global_n$ のどちらか一方を与えなければならない.

逐次, マルチスレッド環境では, $local_n$ は $global_n$ に等しい. したがって, `lis_matrix_set_size(A,n,0)` と `lis_matrix_set_size(A,0,n)` は, いずれも次数 $n \times n$ の行列を作成する.

マルチプロセス環境においては, `lis_matrix_set_size(A,n,0)` は各プロセス上に次数 $n \times N$ の部分行列を作成する. N は n の総和である.
一方, `lis_matrix_set_size(A,0,n)` は各プロセス p 上に次数 $m_p \times n$ の部分行列を作成する. m_p はライブラリ側で決定される.

6.2.8 lis_matrix_get_size

```
C       LIS_INT lis_matrix_get_size(LIS_MATRIX A, LIS_INT *local_n,
         LIS_INT *global_n)
Fortran subroutine lis_matrix_get_size(LIS_MATRIX A, LIS_INTEGER local_n,
         LIS_INTEGER global_n, LIS_INTEGER ierr)
```

機能

行列の次数を取得する.

入力

A	行列

出力

local_n	行列 A のローカルな行数
global_n	行列 A のグローバルな行数
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, $local_n$ は $global_n$ に等しい.

6.2.9 lis_matrix_get_range

```
C       LIS_INT lis_matrix_get_range(LIS_MATRIX A, LIS_INT *is, LIS_INT *ie)
Fortran subroutine lis_matrix_get_range(LIS_MATRIX A, LIS_INTEGER is,
         LIS_INTEGER ie, LIS_INTEGER ierr)
```

機能

部分行列 A が全体行列のどこに位置するのかを調べる.

入力

A	部分行列

出力

is	部分行列 A の全体行列中での開始位置
ie	部分行列 A の全体行列中での終了位置+1
ierr	リターンコード

注釈

逐次, マルチスレッド環境では, 行列の次数が $n \times n$ ならば, C 版では $is = 0$, $ie = n$, Fortran 版では $is = 1$, $ie = n + 1$ である.

6.2.10 lis_matrix_get_nnz

```
C       LIS_INT lis_matrix_get_nnz(LIS_MATRIX A, LIS_INT *nnz)
Fortran subroutine lis_matrix_get_nnz(LIS_MATRIX A, LIS_INTEGER nnz,
         LIS_INTEGER ierr)
```

機能

行列 A の非零要素数を取得する.

入力

A	行列

出力

nnz	行列 A の非零要素数
ierr	リターンコード

注釈

マルチプロセス環境では, 部分行列 A の非零要素数を取得する.

6.2.11 lis_matrix_set_type

```
C       LIS_INT lis_matrix_set_type(LIS_MATRIX A, LIS_INT matrix_type)
Fortran subroutine lis_matrix_set_type(LIS_MATRIX A, LIS_INT matrix_type,
         LIS_INTEGER ierr)
```

機能

行列の格納形式を設定する.

入力

A	行列
matrix_type	行列の格納形式

出力

ierr	リターンコード

注釈

行列作成時の A の matrix_type は LIS_MATRIX_CSR である. 以下に matrix_type に指定可能な格納形式を示す.

格納形式		matrix_type
Compressed Sparse Row	(CSR)	{LIS_MATRIX_CSR|1}
Compressed Sparse Column	(CSC)	{LIS_MATRIX_CSC|2}
Modified Compressed Sparse Row	(MSR)	{LIS_MATRIX_MSR|3}
Diagonal	(DIA)	{LIS_MATRIX_DIA|4}
Ellpack-Itpack Generalized Diagonal	(ELL)	{LIS_MATRIX_ELL|5}
Jagged Diagonal	(JAD)	{LIS_MATRIX_JAD|6}
Block Sparse Row	(BSR)	{LIS_MATRIX_BSR|7}
Block Sparse Column	(BSC)	{LIS_MATRIX_BSC|8}
Variable Block Row	(VBR)	{LIS_MATRIX_VBR|9}
Coordinate	(COO)	{LIS_MATRIX_COO|10}
Dense	(DNS)	{LIS_MATRIX_DNS|11}

6.2.12 lis_matrix_get_type

```
C       LIS_INT lis_matrix_get_type(LIS_MATRIX A, LIS_INT *matrix_type)
Fortran subroutine lis_matrix_get_type(LIS_MATRIX A, LIS_INTEGER matrix_type,
         LIS_INTEGER ierr)
```

機能

行列の格納形式を取得する.

入力

A	行列

出力

matrix_type	行列の格納形式
ierr	リターンコード

6.2.13 lis_matrix_set_csr

```
C       LIS_INT lis_matrix_set_csr(LIS_INT nnz, LIS_INT ptr[], LIS_INT index[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_csr(LIS_INTEGER nnz, LIS_INTEGER ptr(),
         LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

CSR 形式の配列を行列 A に関連付ける.

入力

nnz	非零要素数
ptr, index, value	CSR 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_csr を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.14 lis_matrix_set_csc

```
C       LIS_INT lis_matrix_set_csc(LIS_INT nnz, LIS_INT ptr[], LIS_INT index[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_csc(LIS_INTEGER nnz, LIS_INTEGER ptr(),
         LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

CSC 形式の配列を行列 A に関連付ける.

入力

nnz	非零要素数
ptr, index, value	CSC 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_csc を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.15 lis_matrix_set_msr

```
C       LIS_INT lis_matrix_set_msr(LIS_INT nnz, LIS_INT ndz, LIS_INT index[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_msr(LIS_INTEGER nnz, LIS_INTEGER ndz,
         LIS_INTEGER index(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

MSR 形式の配列を行列 A に関連付ける.

入力

nnz	非零要素数
ndz	対角部分の零要素数
index, value	MSR 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_msr を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.16 lis_matrix_set_dia

```
C       LIS_INT lis_matrix_set_dia(LIS_INT nnd, LIS_INT index[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_dia(LIS_INTEGER nnd, LIS_INTEGER index(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

DIA 形式の配列を行列 A に関連付ける.

入力

nnd	非零な対角要素の本数
index, value	DIA 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_dia を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.17 lis_matrix_set_ell

```
C       LIS_INT lis_matrix_set_ell(LIS_INT maxnzr, LIS_INT index[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_ell(LIS_INTEGER maxnzr, LIS_INTEGER index(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

ELL 形式の配列を行列 A に関連付ける.

入力

maxnzr	各行の非零要素数の最大値
index, value	ELL 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_ell を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.18 lis_matrix_set_jad

```
C       LIS_INT lis_matrix_set_jad(LIS_INT nnz, LIS_INT maxnzr, LIS_INT perm[],
         LIS_INT ptr[], LIS_INT index[], LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_jad(LIS_INTEGER nnz, LIS_INTEGER maxnzr,
         LIS_INTEGER perm(), LIS_INTEGER ptr(), LIS_INTEGER index(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

JAD 形式の配列を行列 A に関連付ける.

入力

nnz	非零要素数
maxnzr	各行の非零要素数の最大値
perm, ptr, index, value	JAD 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_jad を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.19 lis_matrix_set_bsr

```
C       LIS_INT lis_matrix_set_bsr(LIS_INT bnr, LIS_INT bnc,
         LIS_INT bnnz, LIS_INT bptr[], LIS_INT bindex[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_bsr(LIS_INTEGER bnr, LIS_INTEGER bnc,
         LIS_INTEGER bnnz, LIS_INTEGER bptr(), LIS_INTEGER bindex(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

BSR 形式の配列を行列 A に関連付ける.

入力

bnr	行ブロックサイズ
bnc	列ブロックサイズ
bnnz	非零ブロック数
bptr, bindex, value	BSR 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_bsr を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.20 lis_matrix_set_bsc

```
C       LIS_INT lis_matrix_set_bsc(LIS_INT bnr, LIS_INT bnc, LIS_INT bnnz,
         LIS_INT bptr[], LIS_INT bindex[], LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_bsc(LIS_INTEGER bnr, LIS_INTEGER bnc,
         LIS_INTEGER bnnz, LIS_INTEGER bptr(), LIS_INTEGER bindex(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

BSC 形式の配列を行列 A に関連付ける.

入力

bnr	行ブロックサイズ
bnc	列ブロックサイズ
bnnz	非零ブロック数
bptr, bindex, value	BSC 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_bsc を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.21 lis_matrix_set_vbr

```
C       LIS_INT lis_matrix_set_vbr(LIS_INT nnz, LIS_INT nr, LIS_INT nc,
         LIS_INT bnnz, LIS_INT row[], LIS_INT col[], LIS_INT ptr[],
         LIS_INT bptr[], LIS_INT bindex[], LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_vbr(LIS_INTEGER nnz, LIS_INTEGER nr,
         LIS_INTEGER nc, LIS_INTEGER bnnz, LIS_INTEGER row(), LIS_INTEGER col(),
         LIS_INTEGER ptr(), LIS_INTEGER bptr(), LIS_INTEGER bindex(),
         LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

VBR 形式の配列を行列 A に関連付ける.

入力

nnz	非零ブロックの全要素数
nr	行ブロック数
nc	列ブロック数
bnnz	非零ブロック数
row, col, ptr, bptr, bindex, value	VBR 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_vbr を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.22 lis_matrix_set_coo

```
C       LIS_INT lis_matrix_set_coo(LIS_INT nnz, LIS_INT row[], LIS_INT col[],
         LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_coo(LIS_INTEGER nnz, LIS_INTEGER row(),
         LIS_INTEGER col(), LIS_SCALAR value(), LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

COO 形式の配列を行列 A に関連付ける.

入力

nnz	非零要素数
row, col, value	COO 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_coo を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.23 lis_matrix_set_dns

```
C       LIS_INT lis_matrix_set_dns(LIS_SCALAR value[], LIS_MATRIX A)
Fortran subroutine lis_matrix_set_dns(LIS_SCALAR value(), LIS_MATRIX A,
         LIS_INTEGER ierr)
```

機能

DNS 形式の配列を行列 A に関連付ける.

入力

value	DNS 形式の配列
A	行列

出力

A	関連付けられた行列

注釈

lis_matrix_set_dns を用いた後には, lis_matrix_assemble を呼び出さなければならない. また, Fortran 版の配列データは 0 を起点としなければならない.

6.2.24 lis_matrix_unset

```
C       LIS_INT lis_matrix_unset(LIS_MATRIX A)
Fortran subroutine lis_matrix_unset(LIS_MATRIX A, LIS_INTEGER ierr)
```

機能

行列のメモリ領域を確保したまま, 配列の行列 A への関連付けを解除する.

入力

A	関連付けられた行列

出力

A	関連付けを解除された行列

注釈

lis_matrix_unset を用いた後には, lis_matrix_destroy を呼び出さなければならない.

6.3 ベクトルと行列を用いた計算

6.3.1 lis_vector_swap

```
C       LIS_INT lis_vector_swap(LIS_VECTOR x, LIS_VECTOR y)
Fortran subroutine lis_vector_swap(LIS_VECTOR x, LIS_VECTOR y, LIS_INTEGER ierr)
```

機能

ベクトル x, y の要素を交換する.

入力

x, y	ベクトル

出力

x, y	交換後のベクトル
ierr	リターンコード

6.3.2 lis_vector_copy

```
C       LIS_INT lis_vector_copy(LIS_VECTOR x, LIS_VECTOR y)
Fortran subroutine lis_vector_copy(LIS_VECTOR x, LIS_VECTOR y, LIS_INTEGER ierr)
```

機能

ベクトル x の要素をベクトル y に複製する.

入力

x	複製元のベクトル

出力

y	複製先のベクトル
ierr	リターンコード

6.3.3 lis_vector_axpy

```
C       LIS_INT lis_vector_axpy(LIS_SCALAR alpha, LIS_VECTOR x, LIS_VECTOR y)
Fortran subroutine lis_vector_axpy(LIS_SCALAR alpha, LIS_VECTOR x, LIS_VECTOR y,
         LIS_INTEGER ierr)
```

機能

ベクトル和 $y = \alpha x + y$ を計算する.

入力

alpha	スカラ値
x, y	ベクトル

出力

y	$\alpha x + y$ (ベクトル y の値は上書きされる)
ierr	リターンコード

6.3.4 lis_vector_xpay

```
C       LIS_INT lis_vector_xpay(LIS_VECTOR x, LIS_SCALAR alpha, LIS_VECTOR y)
Fortran subroutine lis_vector_xpay(LIS_VECTOR x, LIS_SCALAR alpha, LIS_VECTOR y,
         LIS_INTEGER ierr)
```

機能

ベクトル和 $y = x + \alpha y$ を計算する.

入力

alpha	スカラ値
x, y	ベクトル

出力

y	$x + \alpha y$ (ベクトル y の値は上書きされる)
ierr	リターンコード

6.3.5 lis_vector_axpyz

```
C       LIS_INT lis_vector_axpyz(LIS_SCALAR alpha, LIS_VECTOR x, LIS_VECTOR y,
         LIS_VECTOR z)
Fortran subroutine lis_vector_axpyz(LIS_SCALAR alpha, LIS_VECTOR x, LIS_VECTOR y,
         LIS_VECTOR z, LIS_INTEGER ierr)
```

機能

ベクトル和 $z = \alpha x + y$ を計算する.

入力

alpha	スカラ値
x, y	ベクトル

出力

z	$\alpha x + y$
ierr	リターンコード

6.3.6 lis_vector_scale

```
C       LIS_INT lis_vector_scale(LIS_SCALAR alpha, LIS_VECTOR x)
Fortran subroutine lis_vector_scale(LIS_SCALAR alpha, LIS_VECTOR x,
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素を α 倍する.

入力

alpha	スカラ値
x	ベクトル

出力

x	αx (ベクトル x の値は上書きされる)
ierr	リターンコード

6.3.7 lis_vector_pmul

```
C       LIS_INT lis_vector_pmul(LIS_VECTOR x, LIS_VECTOR y, LIS_VECTOR z)
Fortran subroutine lis_vector_pmul(LIS_VECTOR x, LIS_VECTOR y, LIS_VECTOR z,
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素にベクトル y の対応する要素を掛ける.

入力

x, y	ベクトル

出力

z	計算結果を格納したベクトル
ierr	リターンコード

6.3.8 lis_vector_pdiv

```
C       LIS_INT lis_vector_pdiv(LIS_VECTOR x, LIS_VECTOR y, LIS_VECTOR z)
Fortran subroutine lis_vector_pdiv(LIS_VECTOR x, LIS_VECTOR y, LIS_VECTOR z,
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素をベクトル y の対応する要素で割る.

入力

x, y	ベクトル

出力

z	計算結果を格納したベクトル
ierr	リターンコード

6.3.9 lis_vector_set_all

```
C       LIS_INT lis_vector_set_all(LIS_SCALAR value, LIS_VECTOR x)
Fortran subroutine lis_vector_set_all(LIS_SCALAR value, LIS_VECTOR x,
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素にスカラ値を代入する.

入力

value	代入するスカラ値
x	ベクトル

出力

x	各要素にスカラ値が代入されたベクトル
ierr	リターンコード

6.3.10 lis_vector_abs

```
C       LIS_INT lis_vector_abs(LIS_VECTOR x)
Fortran subroutine lis_vector_abs(LIS_VECTOR x, LIS_INTEGER ierr)
```

機能

ベクトル x の要素の絶対値を求める.

入力

x	ベクトル

出力

x	各要素の絶対値を格納したベクトル
ierr	リターンコード

6.3.11 lis_vector_reciprocal

```
C       LIS_INT lis_vector_reciprocal(LIS_VECTOR x)
Fortran subroutine lis_vector_reciprocal(LIS_VECTOR x, LIS_INTEGER ierr)
```

機能

ベクトル x の要素の逆数を求める.

入力

x	ベクトル

出力

x	各要素の逆数を格納したベクトル
ierr	リターンコード

6.3.12 lis_vector_conjugate

```
C       LIS_INT lis_vector_conjugate(LIS_VECTOR x)
Fortran subroutine lis_vector_conjugate(LIS_VECTOR x, LIS_INTEGER ierr)
```

機能

ベクトル x の要素の共役複素数を求める.

入力

x	ベクトル

出力

x	各要素の共役複素数を格納したベクトル
ierr	リターンコード

6.3.13 lis_vector_shift

```
C       LIS_INT lis_vector_shift(LIS_SCALAR sigma, LIS_VECTOR x)
Fortran subroutine lis_vector_shift(LIS_SCALAR sigma, LIS_VECTOR x,
         LIS_INTEGER ierr)
```

機能

ベクトル x をシフトする.

入力

sigma	シフト量
x	ベクトル

出力

x	各要素のシフト後の値 $x_i - \sigma$ を格納したベクトル
ierr	リターンコード

6.3.14 lis_vector_dot

```
C       LIS_INT lis_vector_dot(LIS_VECTOR x, LIS_VECTOR y, LIS_SCALAR *value)
Fortran subroutine lis_vector_dot(LIS_VECTOR x, LIS_VECTOR y, LIS_SCALAR value,
         LIS_INTEGER ierr)
```

機能

エルミート内積 $x^H y$ を計算する.

入力

x, y	ベクトル

出力

value	エルミート内積
ierr	リターンコード

6.3.15 lis_vector_nhdot

```
C       LIS_INT lis_vector_nhdot(LIS_VECTOR x, LIS_VECTOR y, LIS_SCALAR *value)
Fortran subroutine lis_vector_nhdot(LIS_VECTOR x, LIS_VECTOR y, LIS_SCALAR value,
         LIS_INTEGER ierr)
```

機能

非エルミート内積 $x^T y$ を計算する.

入力

x, y	ベクトル

出力

value	非エルミート内積
ierr	リターンコード

6.3.16 lis_vector_nrm1

```
C       LIS_INT lis_vector_nrm1(LIS_VECTOR x, LIS_REAL *value)
Fortran subroutine lis_vector_nrm1(LIS_VECTOR x, LIS_REAL value, LIS_INTEGER ierr)
```

機能

ベクトル x の 1 ノルムを計算する.

入力

x	ベクトル

出力

value	ベクトルの 1 ノルム
ierr	リターンコード

6.3.17 lis_vector_nrm2

```
C       LIS_INT lis_vector_nrm2(LIS_VECTOR x, LIS_REAL *value)
Fortran subroutine lis_vector_nrm2(LIS_VECTOR x, LIS_REAL value, LIS_INTEGER ierr)
```

機能

ベクトル x の 2 ノルムを計算する.

入力

x	ベクトル

出力

value	ベクトルの 2 ノルム
ierr	リターンコード

6.3.18 lis_vector_nrmi

```
C       LIS_INT lis_vector_nrmi(LIS_VECTOR x, LIS_REAL *value)
Fortran subroutine lis_vector_nrmi(LIS_VECTOR x, LIS_REAL value, LIS_INTEGER ierr)
```

機能

ベクトル x の無限大ノルムを計算する.

入力

x	ベクトル

出力

value	ベクトルの無限大ノルム
ierr	リターンコード

6.3.19 lis_vector_sum

```
C       LIS_INT lis_vector_sum(LIS_VECTOR x, LIS_SCALAR *value)
Fortran subroutine lis_vector_sum(LIS_VECTOR x, LIS_SCALAR value, LIS_INTEGER ierr)
```

機能

ベクトル x の要素の和を計算する.

入力

x	ベクトル

出力

value	ベクトルの要素の和
ierr	リターンコード

6.3.20 lis_matrix_set_blocksize

```
C       LIS_INT lis_matrix_set_blocksize(LIS_MATRIX A, LIS_INT bnr, LIS_INT bnc,
         LIS_INT row[], LIS_INT col[])
Fortran subroutine lis_matrix_set_blocksize(LIS_MATRIX A, LIS_INTEGER bnr,
         LIS_INTEGER bnc, LIS_INTEGER row[], LIS_INTEGER col[], LIS_INTEGER ierr)
```

機能

BSR, BSC, VBR 形式のブロックサイズ, 分割情報を設定する.

入力

A	行列
bnr	BSR(BSC) 形式の行ブロックサイズ, または VBR 形式の行ブロック数
bnc	BSR(BSC) 形式の列ブロックサイズ, または VBR 形式の列ブロック数
row	VBR 形式の行分割情報の配列
col	VBR 形式の列分割情報の配列

出力

ierr	リターンコード

6.3.21 lis_matrix_convert

```
C       LIS_INT lis_matrix_convert(LIS_MATRIX Ain, LIS_MATRIX Aout)
Fortran subroutine lis_matrix_convert(LIS_MATRIX Ain, LIS_MATRIX Aout,
         LIS_INTEGER ierr)
```

機能

行列 A_{in} を指定した格納形式に変換し，行列 A_{out} に格納する．

入力

Ain	変換元の行列

出力

Aout	指定の格納形式に変換された行列
ierr	リターンコード

注釈

変換先の格納形式の指定は lis_matrix_set_type を用いて A_{out} に対して行う．BSR, BSC, VBR 形式のブロックサイズ等の情報は，lis_matrix_set_blocksize を用いて A_{out} に対して設定する．
lis-($VERSION)/test/test2.c 及び lis-($VERSION)/test/test2f.F90 を参照のこと．

指定された格納形式への変換のうち，以下の表において丸印を付したものは，直接変換される．それ以外のものは，記載される形式を経由した後，指定の格納形式に変換される．表に記載されないものは，CSR 形式を経由した後，指定の格納形式に変換される．

元＼先	CSR	CSC	MSR	DIA	ELL	JAD	BSR	BSC	VBR	COO	DNS
CSR		○	○	○	○	○	○	CSC	○	○	○
COO	○	○	○	CSR	CSR	CSR	CSR	CSC	CSR		CSR

6.3.22 lis_matrix_copy

```
C       LIS_INT lis_matrix_copy(LIS_MATRIX Ain, LIS_MATRIX Aout)
Fortran subroutine lis_matrix_copy(LIS_MATRIX Ain, LIS_MATRIX Aout,
         LIS_INTEGER ierr)
```

機能

行列 A_{in} の要素を行列 A_{out} に複製する.

入力

Ain	複製元の行列

出力

Aout	複製先の行列
ierr	リターンコード

6.3.23 lis_matrix_axpy

```
C       LIS_INT lis_matrix_axpy(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B)
Fortran subroutine lis_matrix_axpy(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B,
         LIS_INTEGER ierr)
```

機能

行列和 $B = \alpha A + B$ を計算する.

入力

alpha	スカラ値
A, B	行列

出力

B	$\alpha A + B$ (行列 B の値は上書きされる)
ierr	リターンコード

注釈

行列 A, B は DNS 形式でなければならない.

6.3.24 lis_matrix_xpay

```
C       LIS_INT lis_matrix_xpay(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B)
Fortran subroutine lis_matrix_xpay(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B,
         LIS_INTEGER ierr)
```

機能

行列和 $B = A + \alpha B$ を計算する.

入力

alpha	スカラ値
A, B	行列

出力

B	$A + \alpha B$ (行列 B の値は上書きされる)
ierr	リターンコード

注釈

行列 A, B は DNS 形式でなければならない.

6.3.25 lis_matrix_axpyz

```
C       LIS_INT lis_matrix_axpyz(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B,
         LIS_MATRIX C)
Fortran subroutine lis_matrix_axpyz(LIS_SCALAR alpha, LIS_MATRIX A, LIS_MATRIX B,
         LIS_MATRIX C, LIS_INTEGER ierr)
```

機能

行列和 $C = \alpha A + B$ を計算する.

入力

alpha	スカラ値
A, B	行列

出力

C	$\alpha A + B$
ierr	リターンコード

注釈

行列 A, B, C は DNS 形式でなければならない.

6.3.26 lis_matrix_scale

```
C       LIS_INT lis_matrix_scale(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR d,
         LIS_INT action)
Fortran subroutine lis_matrix_scale(LIS_MATRIX A, LIS_VECTOR b,
         LIS_VECTOR d, LIS_INTEGER action, LIS_INTEGER ierr)
```

機能

行列 A 及びベクトル b のスケーリングを行う.

入力

A	行列
b	ベクトル
action	LIS_SCALE_JACOBI Jacobi スケーリング $D^{-1}Ax = D^{-1}b$ (D は $A = (a_{ij})$ の対角部分) LIS_SCALE_SYMM_DIAG 対角スケーリング $D^{-1/2}AD^{-1/2}x = D^{-1/2}b$ ($D^{-1/2}$ は対角要素の値が $1/\sqrt{a_{ii}}$ である対角行列)

出力

A	スケーリングによって得られる行列
b	スケーリングによって得られるベクトル
d	D^{-1} または $D^{-1/2}$ の対角部分を格納したベクトル
ierr	リターンコード

6.3.27 lis_matrix_get_diagonal

```
C       LIS_INT lis_matrix_get_diagonal(LIS_MATRIX A, LIS_VECTOR d)
Fortran subroutine lis_matrix_get_diagonal(LIS_MATRIX A, LIS_VECTOR d,
         LIS_INTEGER ierr)
```

機能

行列 A の対角部分をベクトル d に複製する.

入力

A	行列

出力

d	対角要素が格納されたベクトル
ierr	リターンコード

6.3.28 lis_matrix_shift_diagonal

```
C       LIS_INT lis_matrix_shift_diagonal(LIS_MATRIX A, LIS_SCALAR sigma)
Fortran subroutine lis_matrix_shift_diagonal(LIS_MATRIX A, LIS_SCALAR sigma,
         LIS_INTEGER ierr)
```

機能

行列 A をシフトする.

入力

sigma	シフト量
A	行列

出力

A	シフト後の行列 $A - \sigma E$
ierr	リターンコード

6.3.29 lis_matrix_shift_matrix

```
C       LIS_INT lis_matrix_shift_matrix(LIS_MATRIX A, LIS_MATRIX B,
         LIS_SCALAR sigma)
Fortran subroutine lis_matrix_shift_matrix(LIS_MATRIX A, LIS_MATRIX B,
         LIS_SCALAR sigma, LIS_INTEGER ierr)
```

機能

行列 A をシフトする.

入力

sigma	シフト量
A	行列
B	行列

出力

A	シフト後の行列 $A - \sigma B$
ierr	リターンコード

6.3.30 lis_matvec

```
C       LIS_INT lis_matvec(LIS_MATRIX A, LIS_VECTOR x, LIS_VECTOR y)
Fortran subroutine lis_matvec(LIS_MATRIX A, LIS_VECTOR x, LIS_VECTOR y,
         LIS_INTEGER ierr)
```

機能

行列ベクトル積 $y = Ax$ を計算する.

入力

A	行列
x	ベクトル

出力

y	Ax
ierr	リターンコード

6.3.31 lis_matvech

```
C       LIS_INT lis_matvech(LIS_MATRIX A, LIS_VECTOR x, LIS_VECTOR y)
Fortran subroutine lis_matvech(LIS_MATRIX A, LIS_VECTOR x, LIS_VECTOR y,
         LIS_INTEGER ierr)
```

機能

行列ベクトル積 $y = A^H x$ を計算する.

入力

A	行列
x	ベクトル

出力

y	$A^H x$
ierr	リターンコード

6.4 線型方程式の求解

6.4.1 lis_solver_create

```
C       LIS_INT lis_solver_create(LIS_SOLVER *solver)
Fortran subroutine lis_solver_create(LIS_SOLVER solver, LIS_INTEGER ierr)
```

機能

ソルバ (線型方程式解法の情報を格納する構造体) を作成する.

入力

なし

出力

solver	ソルバ
ierr	リターンコード

注釈

ソルバは線型方程式解法の情報を持つ.

6.4.2 lis_solver_destroy

```
C       LIS_INT lis_solver_destroy(LIS_SOLVER solver)
Fortran subroutine lis_solver_destroy(LIS_SOLVER solver, LIS_INTEGER ierr)
```

機能

不要になったソルバをメモリから破棄する.

入力

solver	メモリから破棄するソルバ

出力

ierr	リターンコード

6.4.3 lis_precon_create

```
C       LIS_INT lis_precon_create(LIS_SOLVER solver, LIS_PRECON *precon)
Fortran subroutine lis_precon_create(LIS_SOLVER solver, LIS_PRECON precon,
         LIS_INTEGER ierr)
```

機能

前処理行列を作成する.

入力

なし

出力

precon	前処理行列
ierr	リターンコード

6.4.4 lis_precon_destroy

```
C       LIS_INT lis_precon_destroy(LIS_PRECON precon)
Fortran subroutine lis_precon_destroy(LIS_PRECON precon, LIS_INTEGER ierr)
```

機能

不要になった前処理行列をメモリから破棄する.

入力

precon	メモリから破棄する前処理行列

出力

ierr	リターンコード

6.4.5 lis_solver_set_option

```
C       LIS_INT lis_solver_set_option(char *text, LIS_SOLVER solver)
Fortran subroutine lis_solver_set_option(character text, LIS_SOLVER solver,
         LIS_INTEGER ierr)
```

機能

線型方程式解法のオプションをソルバに設定する.

入力

text	コマンドラインオプション

出力

solver	ソルバ
ierr	リターンコード

注釈

以下に指定可能なコマンドラインオプションを示す. -i {cg|1}は -i cg または -i 1 を意味する. -maxiter [1000] は, -maxiter の既定値が 1000 であることを意味する.

線型方程式解法に関するオプション (既定値: `-i bicg`)

線型方程式解法	オプション	補助オプション	
CG	`-i {cg|1}`		
BiCG	`-i {bicg|2}`		
CGS	`-i {cgs|3}`		
BiCGSTAB	`-i {bicgstab|4}`		
BiCGSTAB(l)	`-i {bicgstabl|5}`	`-ell [2]`	次数 l
GPBiCG	`-i {gpbicg|6}`		
TFQMR	`-i {tfqmr|7}`		
Orthomin(m)	`-i {orthomin|8}`	`-restart [40]`	リスタート値 m
GMRES(m)	`-i {gmres|9}`	`-restart [40]`	リスタート値 m
Jacobi	`-i {jacobi|10}`		
Gauss-Seidel	`-i {gs|11}`		
SOR	`-i {sor|12}`	`-omega [1.9]`	緩和係数 ω $(0 < \omega < 2)$
BiCGSafe	`-i {bicgsafe|13}`		
CR	`-i {cr|14}`		
BiCR	`-i {bicr|15}`		
CRS	`-i {crs|16}`		
BiCRSTAB	`-i {bicrstab|17}`		
GPBiCR	`-i {gpbicr|18}`		
BiCRSafe	`-i {bicrsafe|19}`		
FGMRES(m)	`-i {fgmres|20}`	`-restart [40]`	リスタート値 m
IDR(s)	`-i {idrs|21}`	`-irestart [2]`	リスタート値 s
IDR(1)	`-i {idr1|22}`		
MINRES	`-i {minres|23}`		
COCG	`-i {cocg|24}`		
COCR	`-i {cocr|25}`		
GCRO-DR	`-i {gcrodr|26}`	`-gcrodr_k [40]`	GCRO-DR の部分空間次元数
		`-gcrodr_keep [8]`	リサイクルするベクトル数
		`-gcrodr_keep_quality [0.9]`	ベクトルの保持閾値
		`-gcrodr_reject_quality [0.35]`	ベクトルの破棄閾値
FGCRO-DR	`-i {fgcrodr|27}`	`-restart [40]`	リスタート値 m
		`-fgcrodr_k [8]`	部分空間次元数
		`-fgcrodr_keep [0.90]`	空間の保持閾値
		`-fgcrodr_reject [0.10]`	空間の破棄閾値
		`-fgcrodr_update [true]`	リサイクル空間を更新

前処理に関するオプション (既定値: `-p none`)

前処理	オプション	補助オプション	
なし	`-p {none|0}`		
Jacobi	`-p {jacobi|1}`		
ILU(k)	`-p {ilu|2}`	`-ilu_fill [0]`	フィルインレベル k
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
SSOR	`-p {ssor|3}`	`-ssor_omega [1.0]`	緩和係数 ω $(0 < \omega < 2)$
Hybrid	`-p {hybrid|4}`	`-hybrid_i [sor]`	線型方程式解法
		`-hybrid_maxiter [25]`	最大反復回数
		`-hybrid_tol [1.0e-3]`	収束判定基準
		`-hybrid_omega [1.5]`	SOR の緩和係数 ω $(0 < \omega < 2)$
		`-hybrid_ell [2]`	BiCGSTAB(l) の次数 l
		`-hybrid_restart [40]`	GMRES(m), Orthomin(m) の リスタート値 m
I+S	`-p {is|5}`	`-is_alpha [1.0]`	$I + \alpha S^{(m)}$ のパラメータ α
		`-is_m [3]`	$I + \alpha S^{(m)}$ のパラメータ m
SAINV	`-p {sainv|6}`	`-sainv_drop [0.05]`	ドロップ基準
SA-AMG	`-p {saamg|7}`	`-saamg_unsym [false]`	非対称版の選択 (行列構造は対称とする)
		`-saamg_theta [0.05|0.12]`	ドロップ基準 $a_{ij}^2 \leq \theta^2 \lvert a_{ii}\rvert \lvert a_{jj}\rvert$ (対称|非対称)
Crout ILU	`-p {iluc|8}`	`-iluc_drop [0.05]`	ドロップ基準
		`-iluc_rate [5.0]`	最大フィルイン数の倍率
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
ILUT	`-p {ilut|9}`	`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
MFAMG	`-p {mfamg|12}`		
		`-mfamg_block_size [1]`	ローカル連続平滑化ブロックサイズ (1: 重み付きヤコビ 2-: ブロックヤコビ)
Additive Schwarz	`-adds true`	`-adds_iter [1]`	反復回数

その他のオプション

オプション	
-maxiter [1000]	最大反復回数
-maxiter_noimp [0]	残差が改善しない状態が指定した反復回数連続した場合に、計算を早期に打ち切る （デフォルト: 0、早期打ち切りを無効化）
-tol [1.0e-12]	収束判定基準 tol
-tol_w [1.0]	収束判定基準 tol_w
-print [0]	残差履歴の出力
	-print {none|0} 残差履歴を出力しない
	-print {mem|1} 残差履歴をメモリに保存する
	-print {out|2} 残差履歴を標準出力に書き出す
	-print {all|3} 残差履歴をメモリに保存し, 標準出力に書き出す
-scale [0]	スケーリングの選択. 結果は元の行列, ベクトルに上書きされる
	-scale {none|0} スケーリングなし
	-scale {jacobi|1} Jacobi スケーリング $D^{-1}Ax = D^{-1}b$ (D は $A = (a_{ij})$ の対角部分)
	-scale {symm_diag|2} 対角スケーリング $D^{-1/2}AD^{-1/2}x = D^{-1/2}b$ ($D^{-1/2}$ は対角要素の値が $1/\sqrt{a_{ii}}$ である対角行列)
-initx_zeros [1]	初期ベクトル x_0
	-initx_zeros {false|0} 関数 lis_solve() の引数 x により与えられる要素の値を使用
	-initx_zeros {true|1} すべての要素の値を 0 にする
-conv_cond [0]	収束条件
	-conv_cond {nrm2_r|0} $\|\|b - Ax\|\|_2 \leq tol * \|\|b - Ax_0\|\|_2$
	-conv_cond {nrm2_b|1} $\|\|b - Ax\|\|_2 \leq tol * \|\|b\|\|_2$
	-conv_cond {nrm1_b|2} $\|\|b - Ax\|\|_1 \leq tol_w * \|\|b\|\|_1 + tol$
-omp_num_threads [t]	実行スレッド数 t は最大スレッド数
-storage [0]	行列格納形式
-storage_block [2]	BSR, BSC のブロックサイズ
-f [0]	線型方程式解法の精度
	-f {double|0} 倍精度
	-f {quad|1} double-double 型 4 倍精度

6.4.6 lis_solver_set_optionC

```
C       LIS_INT lis_solver_set_optionC(LIS_SOLVER solver)
Fortran subroutine lis_solver_set_optionC(LIS_SOLVER solver, LIS_INTEGER ierr)
```

機能

ユーザプログラム実行時にコマンドラインで指定された線型方程式解法のオプションをソルバに設定する.

入力

なし

出力

solver	ソルバ
ierr	リターンコード

6.4.7 lis_solve

```
C       LIS_INT lis_solve(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_SOLVER solver)
Fortran subroutine lis_solve(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_SOLVER solver, LIS_INTEGER ierr)
```

機能

指定された解法で線型方程式 $Ax = b$ を解く.

入力

A	係数行列
b	右辺ベクトル
x	初期ベクトル
solver	ソルバ

出力

x	解
solver	反復回数, 経過時間等の情報
ierr	リターンコード

注釈

オプション -initx_zeros {false|0} が指定された場合, 初期ベクトルは引数 x により与えられる. その他の場合には, 初期ベクトルのすべての要素の値は 0 に設定される.

この関数は, ソルバの状態 (solver->retcode) が LIS_BREAKDOWN または LIS_MAXITER の場合には 0 を返す. 関数 lis_solver_get_status() も参照のこと.

6.4.8 lis_solve_kernel

```
C       LIS_INT lis_solve_kernel(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_SOLVER solver, LIS_PRECON precon)
Fortran subroutine lis_solve_kernel(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_SOLVER solver, LIS_PRECON precon, LIS_INTEGER ierr)
```

機能

指定された解法について, 外部で定義された前処理を用いて線型方程式 $Ax = b$ を解く.

入力

A	係数行列
b	右辺ベクトル
x	初期ベクトル
solver	ソルバ
precon	前処理

出力

x	解
solver	反復回数, 経過時間等の情報
ierr	リターンコード

注釈

lis-($VERSION)/src/esolver/lis_esolver_ii.c などを参照のこと. このプログラムでは, lis_solve_kernel を複数回呼び出して最小固有値を計算する.

この関数は, ソルバの状態 (solver->retcode) が LIS_BREAKDOWN または LIS_MAXITER の場合には 0 を返す. 関数 lis_solver_get_status() も参照のこと.

6.4.9 lis_solver_get_status

```
C       LIS_INT lis_solver_get_status(LIS_SOLVER solver, LIS_INT *status)
Fortran subroutine lis_solver_get_status(LIS_SOLVER solver, LIS_INTEGER status,
         LIS_INTEGER ierr)
```

機能

状態をソルバから取得する.

入力

solver	ソルバ

出力

status	状態
ierr	リターンコード

注釈

この関数は, ソルバの状態 (solver->retcode) を返す.

6.4.10 lis_solver_get_iter

```
C       LIS_INT lis_solver_get_iter(LIS_SOLVER solver, LIS_INT *iter)
Fortran subroutine lis_solver_get_iter(LIS_SOLVER solver, LIS_INTEGER iter,
         LIS_INTEGER ierr)
```

機能

反復回数をソルバから取得する.

入力

solver	ソルバ

出力

iter	反復回数
ierr	リターンコード

6.4.11 lis_solver_get_iterex

```
C       LIS_INT lis_solver_get_iterex(LIS_SOLVER solver, LIS_INT *iter,
         LIS_INT *iter_double, LIS_INT *iter_quad)
Fortran subroutine lis_solver_get_iterex(LIS_SOLVER solver, LIS_INTEGER iter,
         LIS_INTEGER iter_double, LIS_INTEGER iter_quad, LIS_INTEGER ierr)
```

機能

反復回数に関する詳細情報をソルバから取得する.

入力

solver	ソルバ

出力

iter	総反復回数
iter_double	倍精度演算の反復回数
iter_quad	double-double 型 4 倍精度演算の反復回数
ierr	リターンコード

6.4.12 lis_solver_get_time

```
C       LIS_INT lis_solver_get_time(LIS_SOLVER solver, double *time)
Fortran subroutine lis_solver_get_time(LIS_SOLVER solver, real*8 time,
         LIS_INTEGER ierr)
```

機能

経過時間をソルバから取得する.

入力

solver	ソルバ

出力

time	経過時間
ierr	リターンコード

6.4.13 lis_solver_get_timeex

```
C       LIS_INT lis_solver_get_timeex(LIS_SOLVER solver, double *time,
         double *itime, double *ptime, double *p_c_time, double *p_i_time)
Fortran subroutine lis_solver_get_timeex(LIS_SOLVER solver, real*8 time,
         real*8 itime, real*8 ptime, real*8 p_c_time, real*8 p_i_time,
         LIS_INTEGER ierr)
```

機能

経過時間に関する詳細情報をソルバから取得する.

入力

solver	ソルバ

出力

time	itime と ptime の合計
itime	線型方程式解法の経過時間
ptime	前処理の経過時間
p_c_time	前処理行列作成の経過時間
p_i_time	線型方程式解法中の前処理の経過時間
ierr	リターンコード

6.4.14 lis_solver_get_residualnorm

```
C       LIS_INT lis_solver_get_residualnorm(LIS_SOLVER solver, LIS_REAL *residual)
Fortran subroutine lis_solver_get_residualnorm(LIS_SOLVER solver,
         LIS_REAL residual, LIS_INTEGER ierr)
```

機能

相対残差ノルム $||b - Ax||_2/||b||_2$ をソルバから取得する.

入力

solver	ソルバ

出力

residual	相対残差ノルム
ierr	リターンコード

6.4.15 lis_solver_get_rhistory

```
C       LIS_INT lis_solver_get_rhistory(LIS_SOLVER solver, LIS_VECTOR v)
Fortran subroutine lis_solver_get_rhistory(LIS_SOLVER solver,
         LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

残差履歴をソルバから取得する.

入力

solver	ソルバ

出力

v	残差履歴が収められたベクトル
ierr	リターンコード

注釈

ベクトル v はあらかじめ関数 lis_vector_create で作成しておかなければならない. ベクトル v の次数 n が残差履歴の長さよりも小さい場合は残差履歴の最初から n 個までを取得する.

6.4.16 lis_solver_get_solver

```
C       LIS_INT lis_solver_get_solver(LIS_SOLVER solver, LIS_INT *nsol)
Fortran subroutine lis_solver_get_solver(LIS_SOLVER solver, LIS_INTEGER nsol,
         LIS_INTEGER ierr)
```

機能

線型方程式解法の番号をソルバから取得する.

入力

solver	ソルバ

出力

nsol	線型方程式解法の番号
ierr	リターンコード

6.4.17 lis_solver_get_precon

```
C       LIS_INT lis_solver_get_precon(LIS_SOLVER solver, LIS_INT *precon_type)
Fortran subroutine lis_solver_get_precon(LIS_SOLVER solver, LIS_INTEGER precon_type,
         LIS_INTEGER ierr)
```

機能

前処理の番号をソルバから取得する.

入力

solver	ソルバ

出力

precon_type	前処理の番号
ierr	リターンコード

6.4.18 lis_solver_get_solvername

```
C       LIS_INT lis_solver_get_solvername(LIS_INT nsol, char *name)
Fortran subroutine lis_solver_get_solvername(LIS_INTEGER nsol, character name,
         LIS_INTEGER ierr)
```

機能

線型方程式解法の番号から解法名を取得する.

入力

nsol	線型方程式解法の番号

出力

name	線型方程式解法名
ierr	リターンコード

6.4.19 lis_solver_get_preconname

```
C       LIS_INT lis_solver_get_preconname(LIS_INT precon_type, char *name)
Fortran subroutine lis_solver_get_preconname(LIS_INTEGER precon_type,
         character name, LIS_INTEGER ierr)
```

機能

前処理の番号から前処理名を取得する.

入力

precon_type	前処理の番号

出力

name	前処理名
ierr	リターンコード

6.5 固有値問題の求解

6.5.1 lis_esolver_create

```
C       LIS_INT lis_esolver_create(LIS_ESOLVER *esolver)
Fortran subroutine lis_esolver_create(LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

機能

ソルバ (固有値解法の情報を格納する構造体) を作成する.

入力

なし

出力

esolver	ソルバ
ierr	リターンコード

注釈

ソルバは固有値解法の情報を持つ.

6.5.2 lis_esolver_destroy

```
C       LIS_INT lis_esolver_destroy(LIS_ESOLVER esolver)
Fortran subroutine lis_esolver_destroy(LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

機能

不要になったソルバをメモリから破棄する.

入力

esolver	メモリから破棄するソルバ

出力

ierr	リターンコード

6.5.3 lis_esolver_set_option

```
C       LIS_INT lis_esolver_set_option(char *text, LIS_ESOLVER esolver)
Fortran subroutine lis_esolver_set_option(character text, LIS_ESOLVER esolver,
         LIS_INTEGER ierr)
```

機能

固有値解法のオプションをソルバに設定する.

入力

text	コマンドラインオプション

出力

esolver	ソルバ
ierr	リターンコード

注釈

以下に指定可能なコマンドラインオプションを示す. -e {pi|1}は-e pi または-e 1 を意味する. -emaxiter [1000] は-emaxiter の既定値が 1000 であることを意味する.

固有値解法に関するオプション (既定値: `-e cr`)

固有値解法	オプション	補助オプション	
Power	`-e {pi|1}`		
Inverse	`-e {ii|2}`	`-i [bicg]`	線型方程式解法
Rayleigh Quotient	`-e {rqi|3}`	`-i [bicg]`	線型方程式解法
CG	`-e {cg|4}`	`-i [cg]`	線型方程式解法
CR	`-e {cr|5}`	`-i [bicg]`	線型方程式解法
Subspace	`-e {si|6}`	`-ss [1]`	部分空間の大きさ
Lanczos	`-e {li|7}`	`-ss [1]`	部分空間の大きさ
Arnoldi	`-e {ai|8}`	`-ss [1]`	部分空間の大きさ
Generalized Power	`-e {gpi|9}`	`-i [bicg]`	線型方程式解法
Generalized Inverse	`-e {gii|10}`	`-i [bicg]`	線型方程式解法
Generalized Rayleigh Quotient	`-e {gii|11}`	`-i [bicg]`	線型方程式解法
Generalized CG	`-e {gcg|12}`	`-i [cg]`	線型方程式解法
Generalized CR	`-e {gcr|13}`	`-i [bicg]`	線型方程式解法
Generalized Subspace	`-e {gsi|14}`	`-ss [1]`	部分空間の大きさ
Generalized Lanczos	`-e {gli|15}`	`-ss [1]`	部分空間の大きさ
Generalized Arnoldi	`-e {gai|16}`	`-ss [1]`	部分空間の大きさ
`-ss [1]`	探索部分空間の大きさ		
		`-feast_emin`	ターゲット区間の下限
		`-feast_emax`	ターゲット区間の上限
		`-feast_ratio [0.30]`	楕円積分経路のアスペクト比
		`-feast_points [8]`	積分経路上のサンプリング点数
		`-feast_solver [gmres]`	内部線型方程式解法
Native Krylov	`-e {krylov|18}`	`-ss [1]`	結果ベクトルの容量
		`-nev [1]`	必要な固有対
		`-kncv [0]`	クリロフ部分空間の大きさ

前処理に関するオプション (既定値: `-p none`)

前処理	オプション	補助オプション	
なし	`-p {none|0}`		
Jacobi	`-p {jacobi|1}`		
ILU(k)	`-p {ilu|2}`	`-ilu_fill [0]`	フィルインレベル k
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
SSOR	`-p {ssor|3}`	`-ssor_omega [1.0]`	緩和係数 ω $(0 < \omega < 2)$
Hybrid	`-p {hybrid|4}`	`-hybrid_i [sor]`	線型方程式解法
		`-hybrid_maxiter [25]`	最大反復回数
		`-hybrid_tol [1.0e-3]`	収束判定基準
		`-hybrid_omega [1.5]`	SOR の緩和係数 ω $(0 < \omega < 2)$
		`-hybrid_ell [2]`	BiCGSTAB(l) の次数 l
		`-hybrid_restart [40]`	GMRES(m), Orthomin(m) の リスタート値 m
I+S	`-p {is|5}`	`-is_alpha [1.0]`	$I + \alpha S^{(m)}$ のパラメータ α
		`-is_m [3]`	$I + \alpha S^{(m)}$ のパラメータ m
SAINV	`-p {sainv|6}`	`-sainv_drop [0.05]`	ドロップ基準
SA-AMG	`-p {saamg|7}`	`-saamg_unsym [false]`	非対称版の選択 (行列構造は対称とする)
		`-saamg_theta [0.05|0.12]`	ドロップ基準 $a_{ij}^2 \leq \theta^2 \lvert a_{ii}\rvert\lvert a_{jj}\rvert$ (対称|非対称)
Crout ILU	`-p {iluc|8}`	`-iluc_drop [0.05]`	ドロップ基準
		`-iluc_rate [5.0]`	最大フィルイン数の倍率
		`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
ILUT	`-p {ilut|9}`	`-ilu_pivot_tol [0.0]`	相対極小ピボット閾値
MFAMG	`-p {mfamg|12}`		
		`-mfamg_block_size [1]`	ローカル連続平滑化ブロックサイズ (1: 重み付きヤコビ 2-: ブロックヤコビ)
Additive Schwarz	`-adds true`	`-adds_iter [1]`	反復回数

その他のオプション

オプション	
-emaxiter [1000]	最大反復回数
-etol [1.0e-12]	収束判定基準
-eprint [0]	残差履歴の出力
	-eprint {none|0}　残差履歴を出力しない
	-eprint {mem|1}　残差履歴をメモリに保存する
	-eprint {out|2}　残差履歴を標準出力に書き出す
	-eprint {all|3}　残差履歴をメモリに保存し, 標準出力に書き出す
-ie [ii]	Subspace, Lanczos, Arnoldi の内部で使用する固有値解法の指定
-ige [gii]	Generalized Subspace, Generalized Lanczos, Generalized Arnoldi の内部で使用する固有値解法の指定
-shift [0.0]	$A - \sigma B$ を計算するためのシフト量 σ の実部
-shift_im [0.0]	シフト量 σ の虚部
-initx_ones [1]	初期ベクトル x_0
	-initx_ones {false|0}　関数 lis_esolve() の引数 x により与えられる要素の値を使用
	-initx_ones {true|1}　すべての要素の値を 1 にする
-omp_num_threads [t]	実行スレッド数
	t は最大スレッド数
-estorage [0]	行列格納形式
-estorage_block [2]	BSR, BSC 形式のブロックサイズ
-ef [0]	固有値解法の精度
	-ef {double|0}　倍精度
	-ef {quad|1}　double-double 型 4 倍精度
-rval [0]	Ritz 値
	-rval {false|0}　Ritz 値をもとに固有対を計算
	-rval {true|1}　Ritz 値のみを計算
-nev [1]	必要な固有対の数 ($\leq$ ss)
-etransform [0]	一般化固有値変換の選択
	-etransform {auto|0}　既定の変換を選択
	-etransform {shift-invert|1}　シフト反転 $T = (A - \sigma B)^{-1} B$
	-etransform {buckling|2}　座屈 $T = (A - \sigma B)^{-1} A$

6.5.4 lis_esolver_set_optionC

```
C       LIS_INT lis_esolver_set_optionC(LIS_ESOLVER esolver)
Fortran subroutine lis_esolver_set_optionC(LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

機能

ユーザプログラム実行時にコマンドラインで指定された固有値解法のオプションをソルバに設定する.

入力

なし

出力

esolver	ソルバ
ierr	リターンコード

6.5.5 lis_esolve

```
C       LIS_INT lis_esolve(LIS_MATRIX A, LIS_VECTOR x,
         LIS_SCALAR evalue, LIS_ESOLVER esolver)
Fortran subroutine lis_esolve(LIS_MATRIX A, LIS_VECTOR x,
         LIS_SCALAR evalue, LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

機能

指定された解法で標準固有値問題 $Ax = \lambda x$ を解く.

入力

A	係数行列
x	初期ベクトル
esolver	ソルバ

出力

evalue	既定の固有値
x	対応する固有ベクトル
esolver	反復回数, 経過時間等の情報
ierr	リターンコード

注釈

既定の固有値はモード 0 の固有値である. 以下の節では, 既定の固有対とはモード 0 の固有値及び対応する固有ベクトルを指す.

オプション -initx_ones {false|0} が指定された場合, 初期ベクトルは引数 x により与えられる. その他の場合には, 初期ベクトルのすべての要素の値は 0 に設定される.

この関数は, ソルバの状態 (esolver->retcode) が LIS_MAXITER の場合には 0 を返す. 関数 lis_esolver_get_status() も参照のこと.

6.5.6 lis_gesolve

```
C       LIS_INT lis_gesolve(LIS_MATRIX A, LIS_MATRIX B,
         LIS_VECTOR x, LIS_SCALAR evalue, LIS_ESOLVER esolver)
Fortran subroutine lis_gesolve(LIS_MATRIX A, LIS_MATRIX B,
         LIS_VECTOR x, LIS_SCALAR evalue, LIS_ESOLVER esolver, LIS_INTEGER ierr)
```

機能

指定された解法で一般化固有値問題 $Ax = \lambda Bx$ を解く.

入力

A	係数行列
B	係数行列
x	初期ベクトル
esolver	ソルバ

出力

evalue	既定の固有値
x	既定の固有ベクトル
esolver	反復回数, 経過時間等の情報
ierr	リターンコード

注釈

オプション -initx_ones {false|0} が指定された場合, 初期ベクトルは引数 x により与えられる. その他の場合には, 初期ベクトルのすべての要素の値は 0 に設定される.

この関数は, ソルバの状態 (esolver->retcode) が LIS_MAXITER の場合には 0 を返す. 関数 lis_esolver_get_status() も参照のこと.

6.5.7 lis_esolver_get_status

```
C       LIS_INT lis_esolver_get_status(LIS_ESOLVER esolver, LIS_INT *status)
Fortran subroutine lis_esolver_get_status(LIS_ESOLVER esolver, LIS_INTEGER status,
         LIS_INTEGER ierr)
```

機能

状態をソルバから取得する.

入力

esolver	ソルバ

出力

status	状態
ierr	リターンコード

注釈

この関数は, ソルバの状態 (esolver->retcode) を返す.

6.5.8 lis_esolver_get_iter

```
C       LIS_INT lis_esolver_get_iter(LIS_ESOLVER esolver, LIS_INT *iter)
Fortran subroutine lis_esolver_get_iter(LIS_ESOLVER esolver, LIS_INTEGER iter,
         LIS_INTEGER ierr)
```

機能

既定の固有対の反復回数をソルバから取得する.

入力

esolver	ソルバ

出力

iter	反復回数
ierr	リターンコード

6.5.9 lis_esolver_get_iterex

```
C       LIS_INT lis_esolver_get_iterex(LIS_ESOLVER esolver, LIS_INT *iter)
Fortran subroutine lis_esolver_get_iterex(LIS_ESOLVER esolver, LIS_INTEGER iter,
         LIS_INTEGER ierr)
```

機能

既定の固有対の反復回数に関する詳細情報をソルバから取得する.

入力

esolver	ソルバ

出力

iter	総反復回数
iter_double	倍精度演算の反復回数
iter_quad	double-double 型 4 倍精度演算の反復回数
ierr	リターンコード

6.5.10 lis_esolver_get_time

```
C       LIS_INT lis_esolver_get_time(LIS_ESOLVER esolver, double *time)
Fortran subroutine lis_esolver_get_time(LIS_ESOLVER esolver, real*8 time,
         LIS_INTEGER ierr)
```

機能

既定の固有対の経過時間をソルバから取得する.

入力

esolver	ソルバ

出力

time	経過時間
ierr	リターンコード

6.5.11 lis_esolver_get_timeex

```
C       LIS_INT lis_esolver_get_timeex(LIS_ESOLVER esolver, double *time,
         double *itime, double *ptime, double *p_c_time, double *p_i_time)
Fortran subroutine lis_esolver_get_timeex(LIS_ESOLVER esolver, real*8 time,
         real*8 itime, real*8 ptime, real*8 p_c_time, real*8 p_i_time,
         LIS_INTEGER ierr)
```

機能

既定の固有対の経過時間に関する詳細情報をソルバから取得する.

入力

esolver	ソルバ

出力

time	固有値解法の経過時間
itime	固有値解法中の線型方程式解法の経過時間
ptime	固有値解法中の線型方程式解法前処理の経過時間
p_c_time	前処理行列作成の経過時間
p_i_time	線型方程式解法中の前処理の経過時間
ierr	リターンコード

6.5.12 lis_esolver_get_residualnorm

```
C       LIS_INT lis_esolver_get_residualnorm(LIS_ESOLVER esolver,
         LIS_REAL *residual)
Fortran subroutine lis_esolver_get_residualnorm(LIS_ESOLVER esolver,
         LIS_REAL residual, LIS_INTEGER ierr)
```

機能

既定の固有対の相対残差ノルム $||\lambda x - (B^{-1})Ax||_2/||\lambda x||_2$ をソルバから取得する.

入力

esolver	ソルバ

出力

residual	相対残差ノルム
ierr	リターンコード

6.5.13 lis_esolver_get_rhistory

```
C       LIS_INT lis_esolver_get_rhistory(LIS_ESOLVER esolver, LIS_VECTOR v)
Fortran subroutine lis_esolver_get_rhistory(LIS_ESOLVER esolver,
         LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

既定の固有対の残差履歴をソルバから取得する.

入力

esolver	ソルバ

出力

v	残差履歴が収められたベクトル
ierr	リターンコード

注釈

ベクトル v はあらかじめ関数 lis_vector_create で作成しておかなければならない. ベクトル v の次数 n が残差履歴の長さよりも小さい場合は残差履歴の最初から n 個までを取得する.

6.5.14 lis_esolver_get_evalues

```
C       LIS_INT lis_esolver_get_evalues(LIS_ESOLVER esolver, LIS_VECTOR v)
Fortran subroutine lis_esolver_get_evalues(LIS_ESOLVER esolver,
         LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

すべての固有値をソルバから取得する.

入力

esolver	ソルバ

出力

v	固有値が納められたベクトル
ierr	リターンコード

注釈

ベクトル v はあらかじめ関数 lis_vector_create で作成しておかなければならない.

6.5.15 lis_esolver_get_evectors

```
C       LIS_INT lis_esolver_get_evectors(LIS_ESOLVER esolver, LIS_MATRIX M)
Fortran subroutine lis_esolver_get_evectors(LIS_ESOLVER esolver,
         LIS_MATRIX M, LIS_INTEGER ierr)
```

機能

すべての固有ベクトルをソルバから取得し, 行列 M に格納する.

入力

esolver	ソルバ

出力

M	COO 形式で固有ベクトルが納められた行列
ierr	リターンコード

注釈

行列 M はあらかじめ関数 lis_matrix_create で作成しておかなければならない. 第 i 固有ベクトルは行列 M の第 i 列に格納される. lis-($VERSION)/test/etest5.c を参照のこと.

6.5.16 lis_esolver_get_residualnorms

```
C       LIS_INT lis_esolver_get_residualnorms(LIS_ESOLVER esolver, LIS_VECTOR v)
Fortran subroutine lis_esolver_get_residualnorms(LIS_ESOLVER esolver,
         LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

すべての固有対の相対残差ノルム $||\lambda x - (B^{-1})Ax||_2/||\lambda x||_2$ をソルバから取得する.

入力

esolver	ソルバ

出力

v	相対残差ノルムが納められたベクトル
ierr	リターンコード

注釈

ベクトル v はあらかじめ関数 lis_vector_create で作成しておかなければならない.

6.5.17 lis_esolver_get_iters

```
C       LIS_INT lis_esolver_get_iters(LIS_ESOLVER esolver, LIS_VECTOR v)
Fortran subroutine lis_esolver_get_iters(LIS_ESOLVER esolver,
         LIS_VECTOR v, LIS_INTEGER ierr)
```

機能

すべての固有対の反復回数をソルバから取得する.

入力

esolver	ソルバ

出力

v	反復回数が納められたベクトル
ierr	リターンコード

6.5.18 lis_esolver_get_specific_evalue

```
C       LIS_INT lis_esolver_get_specific_evalue(LIS_ESOLVER esolver, LIS_INT mode,
         LIS_SCALAR evalue)
Fortran subroutine lis_esolver_get_specific_evalue(LIS_ESOLVER esolver,
         LIS_INT mode, LIS_SCALAR evalue, LIS_INTEGER ierr)
```

機能

指定された固有値をソルバから取得する.

入力

esolver	ソルバ
mode	固有値のモード番号

出力

evalue	固有値
ierr	リターンコード

6.5.19 lis_esolver_get_specific_evector

```
C       LIS_INT lis_esolver_get_specific_evector(LIS_ESOLVER esolver, LIS_INT mode,
         LIS_VECTOR x)
Fortran subroutine lis_esolver_get_specific_evector(LIS_ESOLVER esolver,
         LIS_INT mode, LIS_VECTOR x, LIS_INTEGER ierr)
```

機能

指定された固有ベクトルをソルバから取得する.

入力

esolver	ソルバ
mode	固有ベクトルのモード番号

出力

x	固有ベクトル
ierr	リターンコード

6.5.20 lis_esolver_get_specific_residualnorm

```
C       LIS_INT lis_esolver_get_specific_residualnorm(LIS_ESOLVER esolver,
         LIS_INT mode, LIS_REAL *residual)
Fortran subroutine lis_esolver_get_specific_residualnorm(LIS_ESOLVER esolver,
         LIS_INT mode, LIS_REAL residual, LIS_INTEGER ierr)
```

機能

指定された固有対の相対残差ノルム $||\lambda x - (B^{-1})Ax||_2/||\lambda x||_2$ をソルバから取得する.

入力

esolver	ソルバ
mode	固有対のモード番号

出力

residual	相対残差ノルム
ierr	リターンコード

6.5.21 lis_esolver_get_specific_iter

```
C       LIS_INT lis_esolver_get_specific_iter(LIS_ESOLVER esolver, LIS_INT mode,
         LIS_INT *iter)
Fortran subroutine lis_esolver_get_specific_iter(LIS_ESOLVER esolver,
         LIS_INT mode, LIS_INT iter, LIS_INTEGER ierr)
```

機能

指定された固有対の反復回数をソルバから取得する.

入力

esolver	ソルバ
mode	固有対のモード番号

出力

iter	反復回数
ierr	リターンコード

6.5.22 lis_esolver_get_esolver

```
C       LIS_INT lis_esolver_get_esolver(LIS_ESOLVER esolver, LIS_INT *nsol)
Fortran subroutine lis_esolver_get_esolver(LIS_ESOLVER esolver, LIS_INTEGER nsol,
         LIS_INTEGER ierr)
```

機能

固有値解法の番号をソルバから取得する.

入力

esolver	ソルバ

出力

nsol	固有値解法の番号
ierr	リターンコード

6.5.23 lis_esolver_get_esolvername

```
C       LIS_INT lis_esolver_get_esolvername(LIS_INT esolver, char *name)
Fortran subroutine lis_esolver_get_esolvername(LIS_INTEGER esolver, character name,
         LIS_INTEGER ierr)
```

機能

固有値解法の番号から解法名を取得する.

入力

nesol	固有値解法の番号

出力

name	固有値解法名
ierr	リターンコード

6.6 配列を用いた計算

以下はローカルな処理のための関数であり, 並列化されない. 配列データは 0 を起点とし, 列優先順序で格納される. lis-($VERSION)/test/test6.c 及び lis-($VERSION)/test/test6f.F90 を参照のこと.

6.6.1 lis_array_swap

```
C       LIS_INT lis_array_swap(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[])
Fortran subroutine lis_array_swap(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_INTEGER ierr)
```

機能

ベクトル x, y の要素を交換する.

入力

n	ベクトルの次数
x, y	次数 n のベクトル x, y を格納する配列

出力

x, y	交換後の配列
ierr	リターンコード

6.6.2 lis_array_copy

```
C       LIS_INT lis_array_copy(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[])
Fortran subroutine lis_array_copy(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素をベクトル y に複製する.

入力

n	ベクトルの次数
x	複製元のベクトル x を格納する配列

出力

y	複製先のベクトル y が格納された配列
ierr	リターンコード

6.6.3 lis_array_axpy

```
C       LIS_INT lis_array_axpy(LIS_INT n, LIS_SCALAR alpha, LIS_SCALAR x[],
         LIS_SCALAR y[])
Fortran subroutine lis_array_axpy(LIS_INTEGER n, LIS_SCALAR alpha, LIS_SCALAR x(),
         LIS_SCALAR y(), LIS_INTEGER ierr)
```

機能

ベクトル和 $y = \alpha x + y$ を計算する.

入力

n	ベクトルの次数
alpha	スカラ値
x, y	ベクトル x, y を格納する配列

出力

y	$\alpha x + y$ (ベクトル y の値は上書きされる)
ierr	リターンコード

6.6.4 lis_array_xpay

```
C       LIS_INT lis_array_xpay(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR alpha,
         LIS_SCALAR y[])
Fortran subroutine lis_array_xpay(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR alpha,
         LIS_SCALAR y(), LIS_INTEGER ierr)
```

機能

ベクトル和 $y = x + \alpha y$ を計算する.

入力

n	ベクトルの次数
alpha	スカラ値
x, y	ベクトル x, y を格納する配列

出力

y	$x + \alpha y$ (ベクトル y の値は上書きされる)
ierr	リターンコード

6.6.5 lis_array_axpyz

```
C       LIS_INT lis_array_axpyz(LIS_INT n, LIS_SCALAR alpha, LIS_SCALAR x[],
         LIS_SCALAR y[], LIS_SCALAR z[])
Fortran subroutine lis_array_axpyz(LIS_INTEGER n, LIS_SCALAR alpha, LIS_SCALAR x(),
         LIS_SCALAR y(), LIS_SCALAR z(), LIS_INTEGER ierr)
```

機能

ベクトル和 $z = \alpha x + y$ を計算する.

入力

n	ベクトルの次数
alpha	スカラ値
x, y	ベクトル x, y を格納する配列

出力

z	$\alpha x + y$
ierr	リターンコード

6.6.6 lis_array_scale

```
C       LIS_INT lis_array_scale(LIS_INT n, LIS_SCALAR alpha, LIS_SCALAR x[])
Fortran subroutine lis_array_scale(LIS_INTEGER n, LIS_SCALAR alpha, LIS_SCALAR x(),
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素を α 倍する.

入力

n	ベクトルの次数
alpha	スカラ値
x	ベクトル x を格納する配列

出力

x	αx (ベクトル x の値は上書きされる)
ierr	リターンコード

6.6.7 lis_array_pmul

```
C       LIS_INT lis_array_pmul(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[],
         LIS_SCALAR z[])
Fortran subroutine lis_array_pmul(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_SCALAR z(), LIS_INTEGER ierr)
```

機能

ベクトル x の要素にベクトル y の対応する要素を掛ける.

入力

n	ベクトルの次数
x, y	ベクトル x, y を格納する配列

出力

z	計算結果が格納された配列
ierr	リターンコード

6.6.8 lis_array_pdiv

```
C       LIS_INT lis_array_pdiv(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[],
         LIS_SCALAR z[])
Fortran subroutine lis_array_pdiv(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_SCALAR z(), LIS_INTEGER ierr)
```

機能

ベクトル x の要素をベクトル y の対応する要素で割る.

入力

n	ベクトルの次数
x, y	ベクトル x, y を格納する配列

出力

z	計算結果が格納された配列
ierr	リターンコード

6.6.9 lis_array_set_all

```
C       LIS_INT lis_array_set_all(LIS_INT n, LIS_SCALAR value, LIS_SCALAR x[])
Fortran subroutine lis_array_set_all(LIS_INTEGER n, LIS_SCALAR value,
         LIS_SCALAR x(), LIS_INTEGER ierr)
```

機能

ベクトル x の要素にスカラ値を代入する.

入力

n	ベクトルの次数
value	代入するスカラ値
x	ベクトル x を格納する配列

出力

x	各要素にスカラ値が代入された配列
ierr	リターンコード

6.6.10 lis_array_abs

```
C       LIS_INT lis_array_abs(LIS_INT n, LIS_SCALAR x[])
Fortran subroutine lis_array_abs(LIS_INTEGER n, LIS_SCALAR x(), LIS_INTEGER ierr)
```

機能

ベクトル x の要素の絶対値を求める.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

x	各要素の絶対値が格納された配列
ierr	リターンコード

6.6.11 lis_array_reciprocal

```
C       LIS_INT lis_array_reciprocal(LIS_INT n, LIS_SCALAR x[])
Fortran subroutine lis_array_reciprocal(LIS_INTEGER n, LIS_SCALAR x(),
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素の逆数を求める.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

x	各要素の逆数が格納された配列
ierr	リターンコード

6.6.12 lis_array_conjugate

```
C       LIS_INT lis_array_conjugate(LIS_INT n, LIS_SCALAR x[])
Fortran subroutine lis_array_conjugate(LIS_INTEGER n, LIS_SCALAR x(),
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素の共役複素数を求める.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

x	各要素の共役複素数が格納された配列
ierr	リターンコード

6.6.13 lis_array_shift

```
C       LIS_INT lis_array_shift(LIS_INT n, LIS_SCALAR sigma, LIS_SCALAR x[])
Fortran subroutine lis_array_shift(LIS_INTEGER n, LIS_SCALAR sigma, LIS_SCALAR x(),
         LIS_INTEGER ierr)
```

機能

ベクトル x をシフトする.

入力

n	ベクトルの次数
sigma	シフト量
x	ベクトル x を格納する配列

出力

x	各要素のシフト後の値 $x_i - \sigma$ が格納された配列
ierr	リターンコード

6.6.14 lis_array_dot

```
C       LIS_INT lis_array_dot(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[],
         LIS_SCALAR *value)
Fortran subroutine lis_array_dot(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_SCALAR value, LIS_INTEGER ierr)
```

機能

エルミート内積 $x^H y$ を計算する.

入力

n	ベクトルの次数
x, y	ベクトル x, y を格納する配列

出力

value	エルミート内積
ierr	リターンコード

6.6.15 lis_array_nhdot

```
C       LIS_INT lis_array_nhdot(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR y[],
         LIS_SCALAR *value)
Fortran subroutine lis_array_nhdot(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR y(),
         LIS_SCALAR value, LIS_INTEGER ierr)
```

機能

非エルミート内積 $x^T y$ を計算する.

入力

n	ベクトルの次数
x, y	ベクトル x, y を格納する配列

出力

value	非エルミート内積
ierr	リターンコード

6.6.16 lis_array_nrm1

```
C       LIS_INT lis_array_nrm1(LIS_INT n, LIS_SCALAR x[], LIS_REAL *value)
Fortran subroutine lis_array_nrm1(LIS_INTEGER n, LIS_SCALAR x(), LIS_REAL value,
         LIS_INTEGER ierr)
```

機能

ベクトル x の 1 ノルムを計算する.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

value	ベクトルの 1 ノルム
ierr	リターンコード

6.6.17 lis_array_nrm2

```
C       LIS_INT lis_array_nrm2(LIS_INT n, LIS_SCALAR x[], LIS_REAL *value)
Fortran subroutine lis_array_nrm2(LIS_INTEGER n, LIS_SCALAR x(), LIS_REAL value,
         LIS_INTEGER ierr)
```

機能

ベクトル x の 2 ノルムを計算する.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

value	ベクトルの 2 ノルム
ierr	リターンコード

6.6.18 lis_array_nrmi

```
C       LIS_INT lis_array_nrmi(LIS_INT n, LIS_SCALAR x[], LIS_REAL *value)
Fortran subroutine lis_array_nrmi(LIS_INTEGER n, LIS_SCALAR x(), LIS_REAL value,
         LIS_INTEGER ierr)
```

機能

ベクトル x の無限大ノルムを計算する.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

value	ベクトルの無限大ノルム
ierr	リターンコード

6.6.19 lis_array_sum

```
C       LIS_INT lis_array_sum(LIS_INT n, LIS_SCALAR x[], LIS_SCALAR *value)
Fortran subroutine lis_array_sum(LIS_INTEGER n, LIS_SCALAR x(), LIS_SCALAR value,
         LIS_INTEGER ierr)
```

機能

ベクトル x の要素の和を計算する.

入力

n	ベクトルの次数
x	ベクトル x を格納する配列

出力

value	ベクトルの要素の和
ierr	リターンコード

6.6.20 lis_array_matvec

```
C       LIS_INT lis_array_matvec(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR x[],
         LIS_SCALAR y[], LIS_INT op)
Fortran subroutine lis_array_matvec(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR x(),
         LIS_SCALAR y(), LIS_INTEGER op, LIS_INTEGER ierr)
```

機能

行列ベクトル積 Ax を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列
x	次数 n のベクトル x を格納する配列
y	次数 n のベクトル y を格納する配列
op	LIS_INS_VALUE 挿入 : $y = Ax$ LIS_SUB_VALUE 減算代入: $y = y - Ax$

出力

y	y
ierr	リターンコード

6.6.21 lis_array_matvech

```
C       LIS_INT lis_array_matvech(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR x[],
         LIS_SCALAR y[], LIS_INT op)
Fortran subroutine lis_array_matvech(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR x(),
         LIS_SCALAR y(), LIS_INTEGER op, LIS_INTEGER ierr)
```

機能

行列ベクトル積 $A^H x$ を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列
x	次数 n のベクトル x を格納する配列
y	次数 n のベクトル y を格納する配列
op	LIS_INS_VALUE 挿入 : $y = A^H x$ LIS_SUB_VALUE 減算代入: $y = y - H^T x$

出力

y	y
ierr	リターンコード

6.6.22 lis_array_matvec_ns

```
C       LIS_INT lis_array_matvec_ns(LIS_INT m, LIS_INT n, LIS_SCALAR a[],
         LIS_INT lda, LIS_SCALAR x[], LIS_SCALAR y[], LIS_INT op)
Fortran subroutine lis_array_matvec_ns(LIS_INTEGER m, LIS_INTEGER n, LIS_SCALAR a(),
         LIS_INTEGER lda, LIS_SCALAR x(), LIS_SCALAR y(), LIS_INTEGER op,
         LIS_INTEGER ierr)
```

機能

行列 A が正方でない場合に行列ベクトル積 Ax を計算する.

入力

m, n	行列の次数
a	次数 $m \times n$ の行列 A を格納する配列
lda	配列 A の第一次元の次数
x	次数 n のベクトル x を格納する配列
y	次数 m のベクトル y を格納する配列
op	LIS_INS_VALUE 挿入 : $y = Ax$ LIS_SUB_VALUE 減算代入: $y = y - Ax$

出力

y	y
ierr	リターンコード

6.6.23 lis_array_matmat

```
C       LIS_INT lis_array_matmat(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR b[],
         LIS_SCALAR c[], LIS_INT op)
Fortran subroutine lis_array_matmat(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR b(),
         LIS_SCALAR c(), LIS_INTEGER op, LIS_INTEGER ierr)
```

機能

行列積 AB を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列
b	次数 $n \times n$ の行列 B を格納する配列
c	次数 $n \times n$ の行列 C を格納する配列
op	LIS_INS_VALUE 挿入 : $C = AB$ LIS_SUB_VALUE 減算代入: $C = C - AB$

出力

c	C
ierr	リターンコード

6.6.24 lis_array_matmat_ns

```
C       LIS_INT lis_array_matmat_ns(LIS_INT l, LIS_INT m, LIS_INT n,
         LIS_SCALAR a[], LIS_INT lda, LIS_SCALAR b[], LIS_INT ldb, LIS_SCALAR c[],
         LIS_INT ldc, LIS_INT op)
Fortran subroutine lis_array_matmat_ns(LIS_INTEGER l, LIS_INTEGER m, LIS_INTEGER n,
         LIS_SCALAR a(), LIS_INTEGER lda, LIS_SCALAR b(), LIS_INTEGER ldb,
         LIS_SCALAR c(), LIS_INTEGER ldc, LIS_INTEGER op, LIS_INTEGER ierr)
```

機能

行列 A, B が正方でない場合に積 AB を計算する.

入力

l, m, n	行列の次数
a	次数 $l \times m$ の行列 A を格納する配列
lda	配列 A の第一次元の次数
b	次数 $m \times n$ の行列 B を格納する配列
ldb	配列 B の第一次元の次数
c	次数 $l \times n$ の行列 C を格納する配列
ldc	配列 C の第一次元の次数
op	LIS_INS_VALUE 挿入 : $C = AB$ LIS_SUB_VALUE 減算代入: $C = C - AB$

出力

c	C
ierr	リターンコード

6.6.25 lis_array_ge

```
C       LIS_INT lis_array_ge(LIS_INT n, LIS_SCALAR a[]
Fortran subroutine lis_array_ge(LIS_INTEGER n, LIS_SCALAR a(), LIS_INTEGER ierr)
```

機能

Gauss の消去法を用いて行列 A の逆を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列

出力

a	A^{-1}
ierr	リターンコード

6.6.26 lis_array_solve

```
C       LIS_INT lis_array_solve(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR b[],
         LIS_SCALAR x[], LIS_SCALAR w[])
Fortran subroutine lis_array_solve(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR b(),
         LIS_SCALAR x(), LIS_SCALAR w(), LIS_INTEGER ierr)
```

機能

直接法を用いて線型方程式 $Ax = b$ を解く.

入力

n	行列の次数
a	次数 $n \times n$ の係数行列 A を格納する配列
b	次数 n の右辺ベクトル b を格納する配列
w	要素数 $n \times n$ の作業配列

出力

x	解 x
ierr	リターンコード

6.6.27 lis_array_cgs

```
C       LIS_INT lis_array_cgs(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR q[],
         LIS_SCALAR r[])
Fortran subroutine lis_array_cgs(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR q(),
         LIS_SCALAR r(), LIS_INTEGER ierr)
```

機能

古典 Gram-Schmidt 法を用いて QR 分解 $QR = A$ を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列

出力

q	次数 $n \times n$ の直交行列 Q が格納された配列
r	次数 $n \times n$ の上三角行列 R が格納された配列
ierr	リターンコード

6.6.28 lis_array_mgs

```
C       LIS_INT lis_array_mgs(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR q[],
         LIS_SCALAR r[])
Fortran subroutine lis_array_mgs(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR q(),
         LIS_SCALAR r(), LIS_INTEGER ierr)
```

機能

修正 Gram-Schmidt 法を用いて QR 分解 $QR = A$ を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列

出力

q	次数 $n \times n$ の直交行列 Q が格納された配列
r	次数 $n \times n$ の上三角行列 R が格納された配列
ierr	リターンコード

6.6.29 lis_array_qr

```
C       LIS_INT lis_array_qr(LIS_INT n, LIS_SCALAR a[], LIS_SCALAR q[],
         LIS_SCALAR r[], LIS_INT *qriter, LIS_REAL *qrerr)
Fortran subroutine lis_array_qr(LIS_INTEGER n, LIS_SCALAR a(), LIS_SCALAR q(),
         LIS_SCALAR r(), LIS_INTEGER qriter, LIS_REAL qrerr, LIS_INTEGER ierr)
```

機能

QR 法を用いて行列 A の固有値を計算する.

入力

n	行列の次数
a	次数 $n \times n$ の行列 A を格納する配列
q	要素数 $n \times n$ の作業配列 Q
r	要素数 $n \times n$ の作業配列 R

出力

a	固有値をブロック対角要素に持つ相似変換後のブロック上三角行列 A が格納された配列
qriter	QR 法の反復回数
qrerr	相似変換後の第 1 劣対角要素 $A(2,1)$ の 2 ノルム
ierr	リターンコード

6.7 ファイルの操作

6.7.1 lis_input

```
C       LIS_INT lis_input(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x, char *filename)
Fortran subroutine lis_input(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         character filename, LIS_INTEGER ierr)
```

機能

外部ファイルから行列, ベクトルデータを読み込む.

入力

filename	ファイル名

出力

A	指定された格納形式の行列
b	右辺ベクトル
x	解ベクトル
ierr	リターンコード

注釈

対応するファイル形式は以下の通りである.

- 拡張 Matrix Market 形式 (ベクトルデータに対応)
- Harwell-Boeing 形式

これらのデータ構造については付録 A を参照のこと.

6.7.2 lis_input_vector

```
C       LIS_INT lis_input_vector(LIS_VECTOR v, char *filename)
Fortran subroutine lis_input_vector(LIS_VECTOR v, character filename,
         LIS_INTEGER ierr)
```

機能

外部ファイルからベクトルデータを読み込む.

入力

filename	ファイル名

出力

v	ベクトル
ierr	リターンコード

注釈

対応するファイル形式は

- PLAIN 形式
- 拡張 Matrix Market 形式 (ベクトルデータに対応)

これらのデータ構造については付録 A を参照のこと.

6.7.3 lis_input_matrix

```
C       LIS_INT lis_input_matrix(LIS_MATRIX A, char *filename)
Fortran subroutine lis_input_matrix(LIS_MATRIX A, character filename,
         LIS_INTEGER ierr)
```

機能

外部ファイルから行列データを読み込む.

入力

filename	ファイル名

出力

A	指定された格納形式の行列
ierr	リターンコード

注釈

対応するファイル形式は以下の通りである.

- Matrix Market 形式
- Harwell-Boeing 形式

これらのデータ構造については付録 A を参照のこと.

6.7.4 lis_output

```
C       LIS_INT lis_output(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_INT format, char *filename)
Fortran subroutine lis_output(LIS_MATRIX A, LIS_VECTOR b, LIS_VECTOR x,
         LIS_INTEGER format, character filename, LIS_INTEGER ierr)
```

機能

行列, ベクトルデータを外部ファイルに書き込む.

入力

A	行列	
b	右辺ベクトル	
x	解ベクトル	
format	ファイル形式	
	LIS_FMT_MM	Matrix Market 形式
filename	ファイル名	

出力

ierr	リターンコード

注釈

ファイル形式のデータ構造については付録 A を参照のこと.

C 言語版において, ベクトルをファイルに書き込まない場合は NULL を代入することができる.

6.7.5 lis_output_vector

```
C       LIS_INT lis_output_vector(LIS_VECTOR v, LIS_INT format, char *filename)
Fortran subroutine lis_output_vector(LIS_VECTOR v, LIS_INTEGER format,
         character filename, LIS_INTEGER ierr)
```

機能

ベクトルデータを外部ファイルに書き込む.

入力

v	ベクトル	
format	ファイル形式	
	LIS_FMT_PLAIN	PLAIN 形式
	LIS_FMT_MM	Matrix Market 形式
filename	ファイル名	

出力

ierr	リターンコード

注釈

ファイル形式のデータ構造については付録 A を参照のこと.

6.7.6 lis_output_matrix

```
C       LIS_INT lis_output_matrix(LIS_MATRIX A, LIS_INT format, char *filename)
Fortran subroutine lis_output_matrix(LIS_MATRIX A, LIS_INTEGER format,
         character filename, LIS_INTEGER ierr)
```

機能

行列データを外部ファイルに書き込む.

入力

A	行列	
format	ファイル形式	
	LIS_FMT_MM	Matrix Market 形式
filename	ファイル名	

出力

ierr	リターンコード

6.8 その他

6.8.1 lis_initialize

```
C       LIS_INT lis_initialize(int* argc, char** argv[])
Fortran subroutine lis_initialize(LIS_INTEGER ierr)
```

機能

MPI の初期化, コマンドライン引数の取得等の初期化処理を行う.

入力

argc	コマンドライン引数の数
argv	コマンドライン引数

出力

ierr	リターンコード

6.8.2 lis_finalize

```
C       LIS_INT lis_finalize()
Fortran subroutine lis_finalize(LIS_INTEGER ierr)
```

機能

終了処理を行う.

入力

なし

出力

ierr	リターンコード

6.8.3 lis_wtime

```
C       double lis_wtime()
Fortran real*8 lis_wtime()
```

機能

経過時間を計測する.

入力

なし

出力

ある時点からの経過時間を double 型の値 (単位は秒) として返す.

注釈

処理時間を測定する場合は, 処理の開始時と終了時の時間を `lis_wtime` により測定し, その差を求める.

6.8.4 CHKERR

```
C       void CHKERR(LIS_INT ierr)
Fortran subroutine CHKERR(LIS_INTEGER ierr)
```

機能

関数が正常に終了したかどうかを判定する.

入力

`ierr`	リターンコード

出力

なし

注釈

正常に終了していない場合は，`lis_finalize` を実行した後，プログラムを強制終了する.

6.8.5 lis_printf

```
C       LIS_INT lis_printf(LIS_Comm comm, const char *mess, ...)
```

機能

プロセス 0 上で書式付きの文字列を出力する.

入力

comm	MPI コミュニケータ

出力

mess	文字列

注釈

文字列において，'%D' は LIS_INT が long long int の場合には'%lld' に，int の場合には'%d' に置き換えられる.

逐次, マルチスレッド環境では, comm の値は無視される.

参考文献

[1] A. Nishida. Experience in Developing an Open Source Scalable Software Infrastructure in Japan. Lecture Notes in Computer Science 6017, pp. 87-98, Springer, 2010.

[2] M. R. Hestenes and E. Stiefel. Methods of Conjugate Gradients for Solving Linear Systems. Journal of Research of the National Bureau of Standards, Vol. 49, No. 6, pp. 409–436, 1952.

[3] C. Lanczos. Solution of Linear Equations by Minimized Iterations. Journal of Research of the National Bureau of Standards, Vol. 49, No. 1, pp. 33–53, 1952.

[4] R. Fletcher. Conjugate Gradient Methods for Indefinite Systems. Lecture Notes in Mathematics 506, pp. 73–89, Springer, 1976.

[5] P. Joly and G. Meurant. Complex Conjugate Gradient Methods. Numerical Algorithms, Vol. 4, pp. 379–406, 1993.

[6] M. D. Pocock and S. P. Walker. The Complex Bi-Conjugate Gradient Solver Applied to Large Electromagnetic Scattering Problems, Computational Costs, and Cost Scalings. IEEE Transactions on Antennas and Propagation, Vol. 45, No. 1, pp. 140–146, 1997.

[7] T. Sogabe, M. Sugihara, and S. Zhang. An Extension of the Conjugate Residual Method to Nonsymmetric Linear Systems. Journal of Computational and Applied Mathematics, Vol. 226, No. 1, pp. 103–113, 2009.

[8] P. Sonneveld. CGS, A Fast Lanczos-Type Solver for Nonsymmetric Linear Systems. SIAM Journal on Scientific and Statistical Computing, Vol. 10, No. 1, pp. 36–52, 1989.

[9] K. Abe, T. Sogabe, S. Fujino, and S. Zhang. A Product-Type Krylov Subspace Method Based on Conjugate Residual Method for Nonsymmetric Coefficient Matrices (in Japanese). IPSJ Transactions on Advanced Computing Systems, Vol. 48, No. SIG8(ACS18), pp. 11–21, 2007.

[10] H. van der Vorst. Bi-CGSTAB: A Fast and Smoothly Converging Variant of Bi-CG for the Solution of Nonsymmetric Linear Systems. SIAM Journal on Scientific and Statistical Computing, Vol. 13, No. 2, pp. 631–644, 1992.

[11] S. Zhang. Generalized Product-Type Methods Preconditionings Based on Bi-CG for Solving Nonsymmetric Linear Systems. SIAM Journal on Scientific Computing, Vol. 18, No. 2, pp. 537–551, 1997.

[12] S. Fujino, M. Fujiwara, and M. Yoshida. A Proposal of Preconditioned BiCGSafe Method with Safe Convergence. Proceedings of The 17th IMACS World Congress on Scientific Computation, Applied Mathematics and Simulation, CD-ROM, 2005.

[13] S. Fujino and Y. Onoue. Estimation of BiCRSafe Method Based on Residual of BiCR Method (in Japanese). IPSJ SIG Technical Report, 2007-HPC-111, pp. 25–30, 2007.

[14] G. L. G. Sleijpen, H. A. van der Vorst, and D. R. Fokkema. BiCGstab(l) and Other Hybrid Bi-CG Methods. Numerical Algorithms, Vol. 7, No. 1, pp. 75–109, 1994.

[15] R. W. Freund. A Transpose-Free Quasi-Minimal Residual Algorithm for Non-Hermitian Linear Systems. SIAM Journal on Scientific Computing, Vol. 14, No. 2, pp. 470–482, 1993.

[16] K. R. Biermann. Eine unveröffentlichte Jugendarbeit C. G. J. Jacobi über wiederholte Funktionen. Journal für die reine und angewandte Mathematik, Vol. 207, pp. 996-112, 1961.

[17] S. C. Eisenstat, H. C. Elman, and M. H. Schultz. Variational Iterative Methods for Nonsymmetric Systems of Linear Equations. SIAM Journal on Numerical Analysis, Vol. 20, No. 2, pp. 345–357, 1983.

[18] C. F. Gauss. Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem. Perthes et Besser, 1809.

[19] L. Seidel. Über ein Verfahren, die Gleichungen, auf welche die Methode der kleinsten Quadrate führt, sowie lineäre Gleichungen überhaupt, durch successive Annäherung aufzulösen. Abhandlungen der Bayerischen Akademie, Vol. 11, pp. 81–108, 1873.

[20] Y. Saad and M. H. Schultz. GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems. SIAM Journal on Scientific and Statistical Computing, Vol. 7, No. 3, pp. 856–869, 1986.

[21] D. M. Young. Iterative Methods for Solving Partial Difference Equations of Elliptic Type. Doctoral Thesis, Harvard University, 1950.

[22] S. P. Frankel. Convergence Rates of Iterative Treatments of Partial Differential Equations. Mathematical Tables and Other Aids to Computation, Vol. 4, No. 30, pp. 65–75, 1950.

[23] Y. Saad. A Flexible Inner-outer Preconditioned GMRES Algorithm. SIAM Journal on Scientific and Statistical Computing, Vol. 14, No. 2, pp. 461–469, 1993.

[24] P. Sonnerveld and M. B. van Gijzen. IDR(s): A Family of Simple and Fast Algorithms for Solving Large Nonsymmetric Systems of Linear Equations. SIAM Journal on Scientific Computing, Vol. 31, No. 2, pp. 1035–1062, 2008.

[25] C. C. Paige and M. A. Saunders. Solution of Sparse Indefinite Systems of Linear Equations. SIAM Journal on Numerical Analysis, Vol. 12, No. 4, pp. 617–629, 1975.

[26] H. A. van der Vorst and J. B. M. Melissen. A Petrov-Galerkin Type Method for Solving $Ax = b$, where A is Symmetric Complex. IEEE Transactioins on Magnetics, Vol. 26, No. 2, pp. 706–708, 1990.

[27] T. Sogabe and S. Zhang. A COCR Method for Solving Complex Symmetric Linear Systems. Journal of Computational and Applied Mathematics, Vol. 199, No. 2, pp. 297–303, 2007.

[28] Q. Niu, L-Z. Lu and G. Liu. Accelerated GCRO-DR method for solving sequences of systems of linear equations. Journal of Computational and Applied Mathematics, Vol. 253, pp. 131–141, 2013.

[29] L. M. Carvalho, S. Gratton, R. Lago and X. Vasseur. A Flexible Generalized Conjugate Residual Method with Inner Orthogonalization and Deflated Restarting. SIAM Journal on Matrix Analysis and Applications, Vol. 32, No. 4, pp. 1212–1235, 2011.

[30] R. von Mises and H. Pollaczek-Geiringer. Praktische Verfahren der Gleichungsauflösung. Zeitschrift für Angewandte Mathematik und Mechanik, Vol. 9, No. 2, pp. 152–164, 1929.

[31] H. Wielandt. Beiträge zur mathematischen Behandlung komplexer Eigenwertprobleme, Teil V: Bestimmung höherer Eigenwerte durch gebrochene Iteration. Bericht B 44/J/37, Aerodynamische Versuchsanstalt Göttingen, 1944.

[32] J. W. S. Rayleigh. Some General Theorems relating to Vibrations. Proceedings of the London Mathematical Society, Vol. 4, No. 1, pp. 357–368, 1873.

[33] A. V. Knyazev. Toward the Optimal Preconditioned Eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient Method. SIAM Journal on Scientific Computing, Vol. 23, No. 2, pp. 517-541, 2001.

[34] E. Suetomi and H. Sekimoto. Conjugate Gradient Like Methods and Their Application to Eigenvalue Problems for Neutron Diffusion Equation. Annals of Nuclear Energy, Vol. 18, No. 4, pp. 205-227, 1991.

[35] G. L. G. Sleijpen and H. A. van der Vorst. A Jacobi-Davidson Iteration Method for Linear Eigenvalue Problems. SIAM Journal on Matrix Analysis and Applications, Vol. 17, No. 2, pp. 401–425, 1996.

[36] H. R. Rutishauser. Computational Aspects of F. L. Bauser's Simultaneous Iteration Method. Numerische Mathematik, Vol. 13, No. 1, pp. 4–13, 1969.

[37] C. Lanczos. An Iteration Method for the Solution of the Eigenvalue Problem of Linear Differential and Integral Operators. Journal of Research of the National Bureau of Standards, Vol. 45, No. 4, pp. 255–282, 1950.

[38] W. E. Arnoldi. The Principle of Minimized Iterations in the Solution of the Matrix Eigenvalue Problems. Quarterly of Applied Mathematics, Vol. 9, No. 17, pp. 17–29, 1951.

[39] E. Polizzi. FEAST Eigenvalue Solver v4.0 User Guide. arXiv:2002.04807, 2020.

[40] O. Axelsson. A Survey of Preconditioned Iterative Methods for Linear Systems of Equations. BIT, Vol. 25, No. 1, pp. 166–187, 1985.

[41] I. Gustafsson. A Class of First Order Factorization Methods. BIT, Vol. 18, No. 2, pp. 142–156, 1978.

[42] K. Nakajima, H. Nakamura, and T. Tanahashi. Parallel Iterative Solvers with Localized ILU Preconditioning. Lecture Notes in Computer Science 1225, pp. 342–350, 1997.

[43] Y. Saad. ILUT: A Dual Threshold Incomplete LU Factorization. Numerical Linear Algebra with Applications, Vol. 1, No. 4, pp. 387–402, 1994.

[44] Y. Saad, et al. ITSOL: ITERATIVE SOLVERS Package.
http://www-users.cs.umn.edu/~saad/software/ITSOL/.

[45] N. Li, Y. Saad, and E. Chow. Crout Version of ILU for General Sparse Matrices. SIAM Journal on Scientific Computing, Vol. 25, No. 2, pp. 716–728, 2003.

[46] T. Kohno, H. Kotakemori, and H. Niki. Improving the Modified Gauss-Seidel Method for Z-matrices. Linear Algebra and its Applications, Vol. 267, pp. 113–123, 1997.

[47] A. Fujii, A. Nishida, and Y. Oyanagi. Evaluation of Parallel Aggregate Creation Orders : Smoothed Aggregation Algebraic Multigrid Method. High Performance Computational Science and Engineering, pp. 99–122, Springer, 2005.

[48] E. N. Epperly, A. T. Barker, and R. D. Falgout. Smoothers for Matrix-Free Algebraic Multigrid Preconditioning of High-Order Finite Elements. LLNL-TR-814531, 2020.

[49] K. Abe, S. Zhang, H. Hasegawa, and R. Himeno. A SOR-base Variable Preconditioned CGR Method (in Japanese). Transactions of the JSIAM, Vol. 11, No. 4, pp. 157–170, 2001.

[50] R. Bridson and W. P. Tang. Refining an Approximate Inverse. Journal of Computational and Applied Mathematics, Vol. 123, No. 1-2, pp. 293–306, 2000.

[51] T. Chan and T. Mathew. Domain Decomposition Algorithms. Acta Numerica, Vol. 3, pp. 61–143, 1994.

[52] M. Dryja and O. B. Widlund. Domain Decomposition Algorithms with Small Overlap. SIAM Journal on Scientific Computing, Vol. 15, No. 3, pp. 604–620, 1994.

[53] H. Kotakemori, H. Hasegawa, and A. Nishida. Performance Evaluation of a Parallel Iterative Method Library using OpenMP. Proceedings of the 8th International Conference on High Performance Computing in Asia Pacific Region, pp. 432–436, IEEE, 2005.

[54] H. Kotakemori, H. Hasegawa, T. Kajiyama, A. Nukada, R. Suda, and A. Nishida. Performance Evaluation of Parallel Sparse Matrix-Vector Products on SGI Altix 3700. Lecture Notes in Computer Science 4315, pp. 153–163, Springer, 2008.

[55] D. H. Bailey. A Fortran-90 Double-Double Library. http://crd-legacy.lbl.gov/~dhbailey/mpdist/.

[56] Y. Hida, X. S. Li, and D. H. Bailey. Algorithms for Quad-Double Precision Floating Point Arithmetic. Proceedings of the 15th Symposium on Computer Arithmetic, pp. 155–162, 2001.

[57] T. Dekker. A Floating-Point Technique for Extending the Available Precision. Numerische Mathematik, Vol. 18, No. 3, pp. 224–242, 1971.

[58] D. E. Knuth. The Art of Computer Programming: Seminumerical Algorithms, Vol. 2. Addison-Wesley, 1969.

[59] D. H. Bailey. High-Precision Floating-Point Arithmetic in Scientific Computation. Computing in Science and Engineering, Vol. 7, No. 3, pp. 54–61, IEEE, 2005.

[60] Intel Fortran Compiler for Linux Systems User's Guide, Vol I. Intel Corporation, 2004.

[61] H. Kotakemori, A. Fujii, H. Hasegawa, and A. Nishida. Implementation of Fast Quad Precision Operation and Acceleration with SSE2 for Iterative Solver Library (in Japanese). IPSJ Transactions on Advanced Computing Systems, Vol. 1, No. 1, pp. 73–84, 2008.

[62] R. Courant and D. Hilbert. Methods of Mathematical Physics. Wiley-VCH, 1989.

[63] C. Lanczos. The Variational Principles of Mechanics, 4th Edition. University of Toronto Press, 1970.

[64] J. H. Wilkinson. The Algebraic Eigenvalue Problem. Oxford University Press, 1988.

[65] D. M. Young. Iterative Solution of Large Linear Systems. Academic Press, 1971.

[66] G. H. Golub and C. F. Van Loan. Matrix Computations, 3rd Edition. The Johns Hopkins University Press, 1996.

[67] J. J. Dongarra, I. S. Duff, D. C. Sorensen, and H. A. van der Vorst. Solving Linear Systems on Vector and Shared Memory Computers. SIAM, 1991.

[68] Y. Saad. Numerical Methods for Large Eigenvalue Problems. Halsted Press, 1992.

[69] R. Barrett, et al. Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods. SIAM, 1994.

[70] Y. Saad. Iterative Methods for Sparse Linear Systems. Second Edition. SIAM, 2003.

[71] A. Greenbaum. Iterative Methods for Solving Linear Systems. SIAM, 1997.

[72] Z. Bai, et al. Templates for the Solution of Algebraic Eigenvalue Problems. SIAM, 2000.

[73] J. H. Wilkinson and C. Reinsch. Handbook for Automatic Computation, Vol. 2: Linear Algebra. Grundlehren Der Mathematischen Wissenschaften, Vol. 186, Springer, 1971.

[74] B. T. Smith, J. M. Boyle, Y. Ikebe, V. C. Klema, and C. B. Moler. Matrix Eigensystem Routines: EISPACK Guide, 2nd ed. Lecture Notes in Computer Science 6, Springer, 1970.

[75] B. S. Garbow, J. M. Boyle, J. J. Dongarra, and C. B. Moler. Matrix Eigensystem Routines: EISPACK Guide Extension. Lecture Notes in Computer Science 51, Springer, 1972.

[76] J. J. Dongarra, J. R. Bunch, G. B. Moler, and G. M. Stewart. LINPACK Users' Guide. SIAM, 1979.

[77] J. R. Rice and R. F. Boisvert. Solving Elliptic Problems Using ELLPACK. Springer, 1985.

[78] E. Anderson, et al. LAPACK Users' Guide. 3rd ed. SIAM, 1987.

[79] J. Dongarra, A. Lumsdaine, R. Pozo, and K. Remington. A Sparse Matrix Library in C++ for High Performance Architectures. Proceedings of the Second Object Oriented Numerics Conference, pp. 214–218, 1992.

[80] I. S. Duff, R. G. Grimes, and J. G. Lewis. Users' Guide for the Harwell-Boeing Sparse Matrix Collection (Release I). Technical Report TR/PA/92/86, CERFACS, 1992.

[81] Y. Saad. SPARSKIT: A Basic Tool Kit for Sparse Matrix Computations, Version 2, 1994. http://www-users.cs.umn.edu/~saad/software/SPARSKIT/.

[82] A. Geist, et al. PVM: Parallel Virtual Machine. MIT Press, 1994.

[83] R. Bramley and X. Wang. SPLIB: A library of Iterative Methods for Sparse Linear System. Technical Report, Department of Computer Science, Indiana University, 1995.

[84] R. F. Boisvert, et al. The Matrix Market Exchange Formats: Initial Design. Technical Report NISTIR 5935, National Institute of Standards and Technology, 1996.

[85] L. S. Blackford, et al. ScaLAPACK Users' Guide. SIAM, 1997.

[86] R. B. Lehoucq, D. C. Sorensen, and C. Yang. ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly-Restarted Arnoldi Methods. SIAM, 1998.

[87] R. S. Tuminaro, et al. Official Aztec User's Guide, Version 2.1. Technical Report SAND99-8801J, Sandia National Laboratories, 1999.

[88] W. Gropp, E. Lusk, and A. Skjellum. Using MPI, 2nd Edition: Portable Parallel Programming with the Message-Passing Interface. MIT Press, 1999.

[89] K. Garatani, H. Nakamura, H. Okuda, and G. Yagawa. GeoFEM: High Performance Parallel FEM for Solid Earth. Lecture Notes in Computer Science 1593, pp. 133–140, Springer, 1999.

[90] S. Balay, et al. PETSc Users Manual. Technical Report ANL-95/11, Argonne National Laboratory, 2004.

[91] V. Hernandez, J. E. Roman, and V. Vidal. SLEPc: A Scalable and Flexible Toolkit for the Solution of Eigenvalue Problems. ACM Transactions on Mathematical Software, Vol. 31, No. 3, pp. 351–362, 2005.

[92] M. A. Heroux, et al. An Overview of the Trilinos Project. ACM Transactions on Mathematical Software, Vol. 31, No. 3, pp. 397–423, 2005.

[93] R. D. Falgout, J. E. Jones, and U. M. Yang. The Design and Implementation of hypre, a Library of Parallel High Performance Preconditioners. Lecture Notes in Computational Science and Engineering 51, pp. 209–236, Springer, 2006.

[94] B. Chapman, G. Jost, and R. van der Pas. Using OpenMP: Portable Shared Memory Parallel Programming. MIT Press, 2007.

[95] J. Dongarra and M. Heroux. Toward a New Metric for Ranking High Performance Computing Systems. Technical Report SAND2013-4744, Sandia National Laboratories, 2013.

A　ファイル形式

本節では, 本ライブラリで使用できるファイル形式について述べる. なお Harwell-Boeing 形式では, 行列が対称である場合にも上三角及び下三角要素の双方を格納する必要がある.

A.1　拡張 Matrix Market 形式

Matrix Market 形式はベクトルデータの格納に対応していないため, 拡張 Matrix Market 形式では行列とベクトルを合わせて格納できるよう仕様を拡張する. $M \times N$ の行列 $A = (a_{ij})$ の非零要素数を L とする. $a_{ij} = A(I, J)$ とする. ファイル形式を以下に示す.

```
%%MatrixMarket matrix coordinate real general  <-- ヘッダ
%                                              <-+
%                                                | 0 行以上のコメント行
%                                              <-+
M N L B X                                      <-- 行数 列数 非零要素数 (0 or 1) (0 or 1)
I1 J1 A(I1,J1)                                 <-+
I2 J2 A(I2,J2)                                   | 行番号 列番号 値
 . . .                                           | 配列起点は 1
IL JL A(IL,JL)                                 <-+
I1 B(I1)                                       <-+
I2 B(I2)                                         | 右辺ベクトル（B=1 の場合のみ存在する）
 . . .                                           | 行番号 値
IM B(IM)                                       <-+
I1 X(I1)                                       <-+
I2 X(I2)                                         | 解（X=1 の場合のみ存在する）
 . . .                                           | 行番号 値
IM X(IM)                                       <-+
```

(A.1) 式の行列 A とベクトル b に対するファイル形式を以下に示す.

$$A = \begin{pmatrix} 2 & 1 & & \\ 1 & 2 & 1 & \\ & 1 & 2 & 1 \\ & & 1 & 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix} \tag{A.1}$$

```
%%MatrixMarket matrix coordinate real general
4 4 10 1 0
1 2  1.00e+00
1 1  2.00e+00
2 3  1.00e+00
2 1  1.00e+00
2 2  2.00e+00
3 4  1.00e+00
3 2  1.00e+00
3 3  2.00e+00
4 4  2.00e+00
4 3  1.00e+00
1  0.00e+00
2  1.00e+00
3  2.00e+00
4  3.00e+00
```

A.2 Harwell-Boeing 形式

Harwell-Boeing 形式では, CSC 形式で行列を格納する. value を行列 A の非零要素の値, index を非零要素の行番号, ptr を value と index の各列の開始位置を格納する配列とする. ファイル形式を以下に示す.

```
第 1 行 (A72,A8)
   1 - 72 Title
  73 - 80 Key
第 2 行 (5I14)
   1 - 14 ヘッダを除く総行数
  15 - 28 ptr の行数
  29 - 42 index の行数
  43 - 56 value の行数
  57 - 70 右辺ベクトルの行数
第 3 行 (A3,11X,4I14)
   1 -  3 行列の種類
            第 1 列:
              R Real matrix
              C Complex matrix
              P Pattern only (非対応)
            第 2 列:
              S Symmetric (非対応)
              U Unsymmetric
              H Hermitian (非対応)
              Z Skew symmetric (非対応)
              R Rectangular (非対応)
            第 3 列:
              A Assembled
              E Elemental matrices (非対応)
   4 - 14 空白
  15 - 28 行数
  29 - 42 列数
  43 - 56 非零要素数
  57 - 70 0
第 4 行 (2A16,2A20)
   1 - 16 ptr の形式
  17 - 32 index の形式
  33 - 52 value の形式
  53 - 72 右辺の形式
第 5 行 (A3,11X,2I14) 右辺ベクトルが存在する場合
   1      右辺ベクトルの種類
            F フルベクトル
            M 行列と同じ形式 (非対応)
   2      初期値が与えられるならば G
   3      解が与えられるならば X
   4 - 14 空白
  15 - 28 右辺ベクトルの数
  29 - 42 非零要素数
```

(A.1) 式の行列 A とベクトル b に対するファイル形式を以下に示す.

```
1--------10--------20--------30--------40--------50--------60--------70--------80
Harwell-Boeing format sample                                          Lis
             8             1             1              4             2
RUA                        4             4             10             4
(11i7)           (13i6)          (3e26.18)           (3e26.18)
F                          1             0
      1      3      6      9
```

```
   1     2     1     2     3     2     3     4     3     4
 2.00000000000000000E+00  1.00000000000000000E+00  1.00000000000000000E+00
 2.00000000000000000E+00  1.00000000000000000E+00  1.00000000000000000E+00
 2.00000000000000000E+00  1.00000000000000000E+00  1.00000000000000000E+00
 2.00000000000000000E+00
 0.00000000000000000E+00  1.00000000000000000E+00  2.00000000000000000E+00
 3.00000000000000000E+00
```

A.3 ベクトル用拡張 Matrix Market 形式

ベクトル用拡張 Matrix Market 形式では, ベクトルデータを格納できるよう Matrix Market 形式の仕様を拡張する. 次数 N のベクトル $b = (b_i)$ に対して $b_i = B(I)$ とする. ファイル形式を以下に示す.

```
%%MatrixMarket vector coordinate real general  <-- ヘッダ
%                                              <-+
%                                                | 0 行以上のコメント行
%                                              <-+
N                                              <-- 行数
I1 B(I1)                                       <-+
I2 B(I2)                                         | 行番号 値
 . . .                                           | 配列起点は 1
IN B(IN)                                       <-+
```

(A.1) 式のベクトル b に対するファイル形式を以下に示す.

```
%%MatrixMarket vector coordinate real general
4
1  0.00e+00
2  1.00e+00
3  2.00e+00
4  3.00e+00
```

A.4 ベクトル用 PLAIN 形式

ベクトル用 PLAIN 形式は, ベクトルの値を第 1 要素から順に書き出したものである. 次数 N のベクトル $b = (b_i)$ に対して $b_i = B(I)$ とする. ファイル形式を以下に示す.

```
B(1)                                           <-+
B(2)                                             | N 個
. . .                                            |
B(N)                                           <-+
```

(A.1) 式のベクトル b に対するファイル形式を以下に示す.

```
0.00e+00
1.00e+00
2.00e+00
3.00e+00
```